INSTITUTE OF TECHNOLOGY

School of Computing

Department of Software Engineering

Transport Scheduling and Ticket Booking System

A Senior Project Documentation Submitted to Woldia University in Partial Fulfillment of the Requirement for the Degree of Bachelor of Science in Software Engineering

Group Members

Name	ID
1. Belete Siyum	145584
2. Abirham Temesgen	145292
3. Natnael Argaw	147000
4. Sisay Bizualew	147285

Adviser: Mr. Zeleke C.

Submission Date: march/25/2026

Woldia , Ethiopia

DECLARATION

We, the undersigned, hereby declare that the project report entitled “Transport Scheduling and Ticket Booking System” is the result of our own investigation and research, and that all sections of the report have been authored by the team members. We declare that this project has not been submitted to any other institution for academic credit, and that each member has actively participated in and contributed to the requirement analysis and system design phases.

1. Belete Siyum ID: 145584 Signature: _____
2. Abirham Temesgen ID: 145292 Signature: _____
3. Natnael Argaw ID: 147000 Signature: _____
4. Sisay Bizualew ID: 147285 Signature: _____

Project Title: Transport Scheduling and Ticket Booking System

Advisor

Zeleke C.

Signature _____

Date _____

Department of Software Engineering

ACKNOWLEDGMENT

First and foremost, we would like to express our sincere gratitude to our advisor, Mr. Zeleke, for his continuous guidance, valuable suggestions, and professional support throughout the development of this project. His expertise and encouragement were instrumental in shaping our work and keeping us motivated as a team.

We extend our heartfelt thanks to the staff at Woldia Menehariya and the Semen Wolo Transport Logistics office for their cooperation and for providing the necessary information and requirements that formed the foundation of this system. We are also grateful to the Woldia University Department of Software Engineering for providing the academic resources and conducive learning environment necessary for this work.

We would like to acknowledge our teammates for their dedication and collaborative effort, as the shared responsibility among group members played a vital role in the successful completion of this documentation. Finally, we express our deepest gratitude to our families and friends for their constant encouragement, patience, and moral support throughout the duration of this project. Their motivation and understanding helped us overcome challenges and stay focused on our goals.

ABSTRACT

The current inter-city transport management in Ethiopia, particularly at regional terminals like Woldia Menehariya, remains heavily reliant on manual, paper-based processes. This traditional workflow leads to critical inefficiencies, including terminal overcrowding, frequent double-bookings, and a complete lack of real-time schedule visibility for passengers. This project presents a digital "Transport Scheduling and Ticket Booking System" designed to modernize these operations. The system features a centralized Web dashboard for administrators developed with Next.js and Spring Boot, alongside a cross-platform mobile application built with Flutter for passengers. Key functional modules include real-time bus searching, interactive seat selection, integrated digital payment gateways (Telebirr/CBE Birr), and secure QR-code-based ticket verification. Developed using the Object-Oriented System Analysis and Design (OOSAD) methodology and the Incremental Development Model, the system aims to enhance operational accountability for the Ethiopian Transport Authority and provide a seamless, convenient travel experience for the public.

ACRONYMS AND ABBREVIATIONS

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
ERD	Entity-Relationship Diagram
ETA	Ethiopian Transport Authority
ETB	Ethiopian Birr
FIFO	First-In, First-Out
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NFR	Non-Functional Requirement
OOSAD	Object-Oriented System Analysis and Design
ORM	Object-Relational Mapping
PK / FK	Primary Key / Foreign Key
POS	Point of Sale
QR	Quick Response (Code)
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
SOA	Service-Oriented Architecture
TLS / SSL	Transport Layer Security / Secure Sockets Layer
UML	Unified Modeling Language

Table of Contents

DECLARATION i

ACKNOWLEDGMENT ii

ABSTRACT iii

ACRONYMS AND ABBREVIATIONS iv

LIST OF FIGURES ix

LIST OF TABLES x

CHAPTER 1: INTRODUCTION 1

1.1 Background of the Organization 1

1.2 Project Overview 1

1.3 Statement of the Problem 1

1.4 Objectives of the Project 2

1.4.1 General Objective 2

1.4.2 Specific Objectives 2

1.5 Scope and Limitation of the Project 2

1.5.1 Scope of the Project 2

1.5.2 Limitation of the Project 3

1.6 Project Significance 3

1.7 Project Beneficiaries 3

1.8 Feasibility Study of the Project 4

1.8.1 Technical Feasibility 4

1.8.2 Economic Feasibility 4

1.8.3 Operational Feasibility 4

1.8.4 Schedule Feasibility 4

1.9 Methodology of the Project 4

1.9.1 Data Gathering Methodologies 4

1.9.2 System Analysis and Design Methodology 5

1.9.3 System Development Model 5

1.10 Development Tools 5
1.10.1 Software Tools 5
1.10.2 Hardware Tools 5
1.11 Team Roles and Responsibilities 5
CHAPTER 2: ANALYSIS OF THE CURRENT SYSTEM 7
2.1 Description of the Current System 7
2.2 Overview of the Current System (Manual Workflow) 7
2.2.1 Queue and Schedule Management 7
2.2.2 Manual Ticketing and Seat Allocation 7
2.2.3 Ticket Collection and Dispatching 8
2.3 Structure of the Existing System (Process Flow) 8
2.4 Key Limitations of the Current System 10
CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION 11
3.1. Proposed System Overview 11
3.2 Functional Requirements 12
3.2.1 FR-URM: User Management & Authentication 12
3.2.2 FR-FLM: Vehicle Fleet & Association Management 13
3.2.3 FR-TKM: Search, Selection & Booking Management 14
3.2.4 FR-PYM: Payment, Ticketing & Notifications 15
3.2.5 FR-OPS: Dispatch & Terminal Operations 16
3.2.6 FR-REP: Reporting & Analytics 16
3.2.7 Functional Requirement Traceability Matrix 17
3.3. Non-Functional Requirements 19
3.3.2 Security Requirements 19
3.3.3 Usability Requirements 19
3.3.4 Reliability Requirements 19
3.3.5 Availability Requirements 20
3.3.6 Scalability Requirements 20
3.3.7 Maintainability Requirements 20

3.3.8 Compatibility Requirements 20
3.3.9 Portability Requirements 20
3.3.10 Data Integrity & Accuracy Requirements 21
3.3.11 Legal & Compliance Requirements 21
3.3.12 Localization & Language Requirements 21
3.3.13 Backup & Recovery Requirements 21
3.3.14 Ethical & Transparency Requirements 21
3.3.15 Operational Constraints 22
3.4 Use Case Models 23
3.4.1 Use Case Diagram 23
3.4.2 Use Case Descriptions 25
3.5 Object Model 50
3.5.1 Class Diagram 50
3.6 Dynamic Models 52
3.6.1 Sequence Diagram 52
3.6.2 Activity Diagram 60
3.6.3 Activity Diagram Descriptions 63
CHAPTER 4: SYSTEM DESIGN 66
4.1 Introduction 66
4.2 System Purpose 66
4.3 Design Goals 67
4.4 Current Software Architecture 67
4.4.1 Overview 67
4.4.2 Process Decomposition 68
4.4.3 Key Limitations 68
4.5 Proposed System Architecture 69
4.5.1 Overview 69
4.5.2 System Modules & Responsibilities 71
4.5.3 Deployment & Infrastructure 71

4.5.4 Database Design 72
4.5.5 Security & Access Control 73
4.5.6 Boundary Conditions 74
4.6 User Interface Design 74
4.6.1 Navigational Paths 74
4.6.2 Screen Mock-ups 75
4.7 Detailed Component Specifications 76
4.7.1 Financial Fare Calculation & Revenue Split Engine 76
4.8 Integration Interfaces 78
Core System API Endpoint Specifications 78
Appendix 80

LIST OF FIGURES

LIST OF TABLES

Table Number	Table Title / Technical Description	Document Section
Table 3.1	Functional Requirement Traceability Matrix	Chapter 3 (Requirements)
Table 3.2	Use Case Description Dictionary: Register Account (UC-01)	Chapter 3 (Use Case Specs)
Table 3.3	Use Case Description Dictionary: Secure Access Authentication (UC-02)	Chapter 3 (Use Case Specs)
Table 3.4	Use Case Description Dictionary: Edit Profile (UC-03)	Chapter 3 (Use Case Specs)
Table 3.5	Use Case Description Dictionary: View Booking History (UC-04)	Chapter 3 (Use Case Specs)
Table 3.6	Use Case Description Dictionary: Manage Users (UC-05)	Chapter 3 (Use Case Specs)
Table 3.7	Use Case Description Dictionary: Logout (UC-06)	Chapter 3 (Use Case Specs)
Table 3.8	Use Case Description Dictionary: Profile Fleet Assets & Drivers (UC-07)	Chapter 3 (Use Case Specs)
Table 3.9	Use Case Description Dictionary: Manage Vehicle Records (UC-08)	Chapter 3 (Use Case Specs)
Table 3.10	Use Case Description Dictionary: Onboard Transport Association (UC-09)	Chapter 3 (Use Case Specs)
Table 4.1	User Interface Navigational Paths Matrix by System Role	Chapter 4 (UI Design)
Table 4.2	Relational Data Dictionary & Third Normal Form (3NF) Schema Definitions	Chapter 4 (Database Design)
Table 4.3	Detailed Core System API Endpoint Specifications and Payload Definitions	Chapter 4 (Component Specs)

CHAPTER 1: INTRODUCTION

1.1 Background of the Organization

The Ethiopian transport sector is primarily overseen by the Ethiopian Transport Authority (ETA), which is responsible for regulating, coordinating, and managing the movement of passengers and freight across the country. In regional hubs and university towns like Woldia, transportation is a critical utility. Currently, the management of vehicle schedules and ticket sales is largely decentralized across various transport associations. These associations operate within designated public transit terminals, serving Level 1, 2, and 3 transport services. The organizational workflow currently relies on manual record-keeping by terminal clerks and dispatchers, who manage vehicle queues and passenger manifests using physical paper ledgers.

1.2 Project Overview

The "Transport Scheduling and Ticket Booking System" is a comprehensive digital solution designed to modernize the way public transport is accessed and managed in Ethiopia. This project integrates a web-based administrative dashboard with a cross-platform mobile application. The system aims to automate vehicle scheduling routines, digitize the ticket issuance process, and provide a real-time platform for passengers to reserve travel seats remotely.

To prevent speculative block-bookings and system manipulation, the core engine strictly enforces a single-ticket constraint per passenger transaction (1 ticket per traveler account hold structure). By leveraging modern web and mobile technologies, the project seeks to eliminate the physical constraints of traditional booking and provide the ETA, drivers, and transport associations with accurate, real-time data for operational decision-making.

1.3 Statement of the Problem

The current manual transport management system in Ethiopia faces several critical challenges:

- Physical Constraints: Passengers are required to travel physically to terminals to check schedules and purchase tickets, leading to wasted time and severe station overcrowding.
- Double-Booking and Human Errors: Manual seat allocation across scattered ticket windows is highly prone to human error, often resulting in seat duplicate assignments or inaccurate passenger manifests.
- Information Asymmetry: There is no centralized platform for passengers to compare vehicle operational levels, dynamic pricing structures, or departure timelines across different independent associations.
- Inefficient Reporting & Manifest Management: Gathering financial data, driver log sheets, and traffic statistics from manual ledgers is slow and prone to data loss. This makes it exceptionally difficult for the ETA to monitor revenue and for dispatchers to securely issue required documents to drivers.

- Security and Boarding Fraud: Physical paper tickets are easily forged, altered, or lost, and there is no secure verification mechanism at the point of boarding to match a traveler against the active driver manifest.

1.4 Objectives of the Project

1.4.1 General Objective

The General objective of this project is to design, develop, and implement an integrated Transport Scheduling and Ticket Booking System that automates vehicle scheduling routines, incorporates verified driver assignment workflows, and enables secure, digital single-ticket reservations for passengers.

1.4.2 Specific Objectives

- To conduct a thorough investigation of the current manual booking, driver staging, and vehicle scheduling process at the terminal level.
- To design a relational database capable of managing structural vehicle seating grid layouts, driver registries, and real-time state synchronization.
- To develop a web-based interface for administrators and association managers to seamlessly oversee routes, vehicle management, drivers, and operational schedules.
- To create a user-friendly mobile application that allows passengers to search for real-time availability, select a single seat from an interactive cabin grid, and execute checkouts digitally.
- To integrate a secure QR-code-based verification system for terminal clerks and drivers to quickly validate passenger tickets at the gate.
- To implement a centralized reporting module that generates automated financial distribution splits and digital passenger manifests streamed directly to drivers.

1.5 Scope and Limitation of the Project

1.5.1 Scope of the Project

The scope of this project focuses on the digital transformation of the ticketing, driver routing, and scheduling lifecycle, specifically:

- Functional Areas: User registration and role-based authentication, real-time schedule searching, interactive seat selection grids, a transactional 10-minute database hold lock, digital payment gateway integration (Telebirr/CBE Birr), automated QR ticket generation, and single-ticket reservation enforcement.
- Administrative Areas: Vehicle Management (covering Level 1, 2, and 3 fleets), driver roster sheets, route and base price configurations, automated $5\% / 95\%$ revenue splits, and dispatcher manifest management.
- Technical Platforms: A responsive web application for administrators and association managers (Next.js/Spring Boot) and a native mobile application for passengers and drivers (Flutter).

- Geographic Focus: The system is designed with a focus on regional transport routes within Ethiopia, starting with terminal operations similar to those in Woldia.

1.5.2 Limitation of the Project

While the system is robust, it has the following limitations:

- Hardware Integration: The system does not include live GPS hardware tracking for vehicles; it relies on manual status updates executed by dispatchers and drivers.
- Connectivity Constraints: The system requires an active internet connection for real-time seat synchronization and database row-locking; it cannot perform offline bookings.
- Refund Automation: While tickets can be cancelled via the app, the physical disbursement of cash refunds remains manual at the terminal based on the generated cancellation report.
- Regional Scope: The current version is optimized for cross-regional travel paths and excludes local intra-city "Lada" or minibus transit services.

1.6 Project Significance

The development of this system holds significant value for the modernization of the Ethiopian transport sector. By digitizing the manual ticket booking process, the project reduces the time and effort required for passengers to secure travel. It eliminates the need for physical lines, entirely cuts out human error in seat allocation, and provides an independent, transparent pricing model. For transport operators and drivers, it provides streamlined, paperless manifest routing. For the ETA, it offers a centralized data management tool that ensures financial accountability, enforces regulatory tariffs, and improves broad revenue tracking.

1.7 Project Beneficiaries

The primary beneficiaries of the "Transport Scheduling and Ticket Booking System" include:

- Passengers: Gain the convenience of remote booking, real-time schedule transparency, and secure digital payment options while restricted to a fair single-ticket hold rule.
- Drivers: Experience clear, structured schedule planning and receive real-time digital passenger manifests sent straight to their application viewports before departing.
- Transport Associations & Operators: Benefit from automated vehicle management, automated revenue split summaries, reduced paperwork, and more efficient vehicle queue line tracking.
- The Ethiopian Transport Authority (ETA): Receives accurate data metrics for regulatory oversight, tax assessment, base tariff enforcement, and nationwide transport planning.
- Terminal Clerks & Dispatchers: Experience a vastly reduced workload through automated passenger manifests and rapid handheld digital ticket verification tools.

1.8 Feasibility Study of the Project

1.8.1 Technical Feasibility

The project is technically feasible as the required development tools (Next.js, Spring Boot, Flutter, and PostgreSQL) are industry-standard, secure, and highly supported. The team possesses the necessary skills in software engineering and web/mobile development. Modern cloud hosting services and local payment APIs (Telebirr/CBE Birr) provide the infrastructure needed to deploy and maintain the database row locks and role permissions securely.

1.8.2 Economic Feasibility

The economic benefits outweigh the initial development costs. By reducing manual entry errors, preventing ticket duplicate scams, and eliminating the overhead of physical paper-based record-keeping, the system creates long-term savings. The cost of cloud hosting and database maintenance is low compared to the increased revenue tracking efficiency achieved for transport associations via automated commission splits.

1.8.3 Operational Feasibility

The system is designed to integrate into the existing transport workflow seamlessly. By providing a simple interface for terminal clerks and drivers (QR scanning passes) and a centralized dashboard for association admins, it does not require an aggressive overhaul of terminal infrastructure, but rather a digital upgrade. The high rate of smartphone adoption in Ethiopia ensures that passengers and drivers will be able to utilize the mobile features easily.

1.8.4 Schedule Feasibility

The project is scheduled to be completed within the academic timeline. The timeline follows a structured approach: Requirements Gathering (Phase 1), System Design (Phase 2), Implementation (Phase 3), and Testing/Deployment (Phase 4), ensuring all system modules (including the 80% manifest capacity trigger) are fully functional before the submission deadline of June 2026.

1.9 Methodology of the Project

1.9.1 Data Gathering Methodologies

To ensure the system maps accurately to real-world transit habits, the following methods were used:

- Interviews: Engaging directly with terminal dispatchers, drivers, and transport association managers to understand the current vehicle queue tracking loops and manual manifests.
- Observation: Visiting bus terminals to witness the physical ticket-buying process, analyze lane constraints, and identify customer service bottlenecks.
- Document Review: Analyzing existing manual ticket stubs, carbon receipts, and paper passenger manifests to explicitly define required database fields.

1.9.2 System Analysis and Design Methodology

The project follows the Object-Oriented System Analysis and Design (OOSAD) approach. This methodology allows the team to model the system using real-world entities (such as "Vehicle," "Driver," "Passenger," and "Ticket") through Unified Modeling Language (UML) structural and behavioral diagrams, ensuring a clean, modern, and modular architecture.

1.9.3 System Development Model

The Incremental Development Model is utilized. This allows the team to build and test the core foundational modules (like User Auth, Role-Based Permission Guards, and Vehicle Management) first, and iteratively append more complex transactional features (like the 10-minute pessimistic seat hold, gateway checkouts, and digital QR scanning passes) based on testing and feedback.

1.10 Development Tools

1.10.1 Software Tools

- Frontend: Next.js (Web Admin), Flutter (Mobile App).
- Backend: Java (Spring Boot) or PHP (Laravel optional) for RESTful API development.
- Database: PostgreSQL or MySQL for relational data storage.
- Version Control: Git/GitHub for team collaboration.
- Design: Figma for UI/UX mock-ups.

1.10.2 Hardware Tools

- Development Machines: Laptops with minimum 8GB RAM for coding and testing.
- Test Devices: Android and iOS smartphones to test the mobile application.
- Servers: Cloud-based hosting (e.g., AWS or DigitalOcean) for deployment.

1.11 Team Roles and Responsibilities

To ensure accountability throughout the project lifecycle, the following matrix outlines the involvement of each member, noting that the entire team participates in every phase of the Software Development Life Cycle (SDLC):

- Requirement Gathering & Analysis
    - ✓ Lead: Belete Siyum
    - ✓ Support: Abirham Temesgen, Natnael Argaw, Sisay Bizualew

- System Analysis and Design (OOSAD)
    - ✓ all member
- Implementation (Frontend - Web & Mobile)
    - ✓ all member
- Implementation (Backend & API)
    - ✓ Lead: Belete Siyum, Abirham Temesgen
    - ✓ Support: Natnael Argaw, Sisay Bizualew
- Testing and Quality Assurance
    - ✓ Lead: Belete Siyum, Abirham Temesgen
    - ✓ Support: Entire Team
- Deployment and Installation
    - ✓ Lead: Abirham Temesgen
    - ✓ Support: Belete Siyum

CHAPTER 2: ANALYSIS OF THE CURRENT SYSTEM

2.1 Description of the Current System

The current system for transport scheduling and ticket booking at Woldia and surrounding regional terminals is a purely manual, paper-based process. It relies completely on the physical presence of passengers at the bus station and the hand-written coordination of schedules by terminal dispatchers.There is no centralized digital database or communication network linking different transport associations. This decentralized setup means each association must manage its own fleet, financial tracking, and seating allocations entirely through physical logbooks and paper ticket stubs.

2.2 Overview of the Current System (Manual Workflow)

The manual workflow governing daily terminal activities is divided into three primary operational areas:

2.2.1 Queue and Schedule Management

The scheduling of transport vehicles is performed manually by dispatchers using a physical blackboard or a ledger book.

- As buses arrive at the terminal, they are registered into an operational queue based strictly on their arrival timestamp and vehicle classification level (Level 1, Level 2, or Level 3).
- Departure times are not fixed or predetermined. Instead, a vehicle's departure depends entirely on when it reaches its maximum passenger capacity. This lack of fixed scheduling creates major uncertainty for passengers, who cannot plan their trips in advance.

2.2.2 Manual Ticketing and Seat Allocation

When a passenger arrives at the terminal, they must visit the specific ticket window designated for the transport association operating their intended route.

- The ticketing clerk checks a hand-drawn seat map or a physical list of numbers to verify which seats remain vacant.
- Seats are allocated on a strict first-come, first-served basis; ticket numbers are used for sales tracking and do not correspond to fixed seat positions on the vehicle.
- Upon receiving a cash payment, the clerk manually transcribes the passenger's name, travel date, and seat number onto a pre-printed paper receipt book. One physical copy is torn out and given to the passenger, while the carbon copy remains bound inside the ticket book as the sole transactional record.

2.2.3 Ticket Collection and Dispatching

The current system does not utilize a centralized pre-departure passenger list or manifest board before boarding. Instead, validation and passenger dispatching rely completely on physical ticket handovers and direct coordination between terminal administration staff and transport operators:

- Ticket Collection: Rather than verifying names against a master board, passengers board the bus and directly hand over their physical paper tickets to the driver or vehicle owner as proof of fare payment.
- Revenue Splitting: The total revenue generated from the route's ticket sales is split based on a fixed percentage. The terminal administration retains a small percentage (typically around 5%) as a service and administrative fee to cover terminal maintenance and operational costs. The remaining majority (approximately 95%) belongs entirely to the vehicle owner or driver.
- Payout Timing: The accumulated fare cash is manually handed over to the driver or owner by the terminal administration staff. Depending on the association's protocol and route type, this payout occurs either directly during the boarding process as the vehicle prepares to leave, or after the bus completes its journey and returns to its home station (e.g., returning to Woldia after traveling to Bahir Dar).

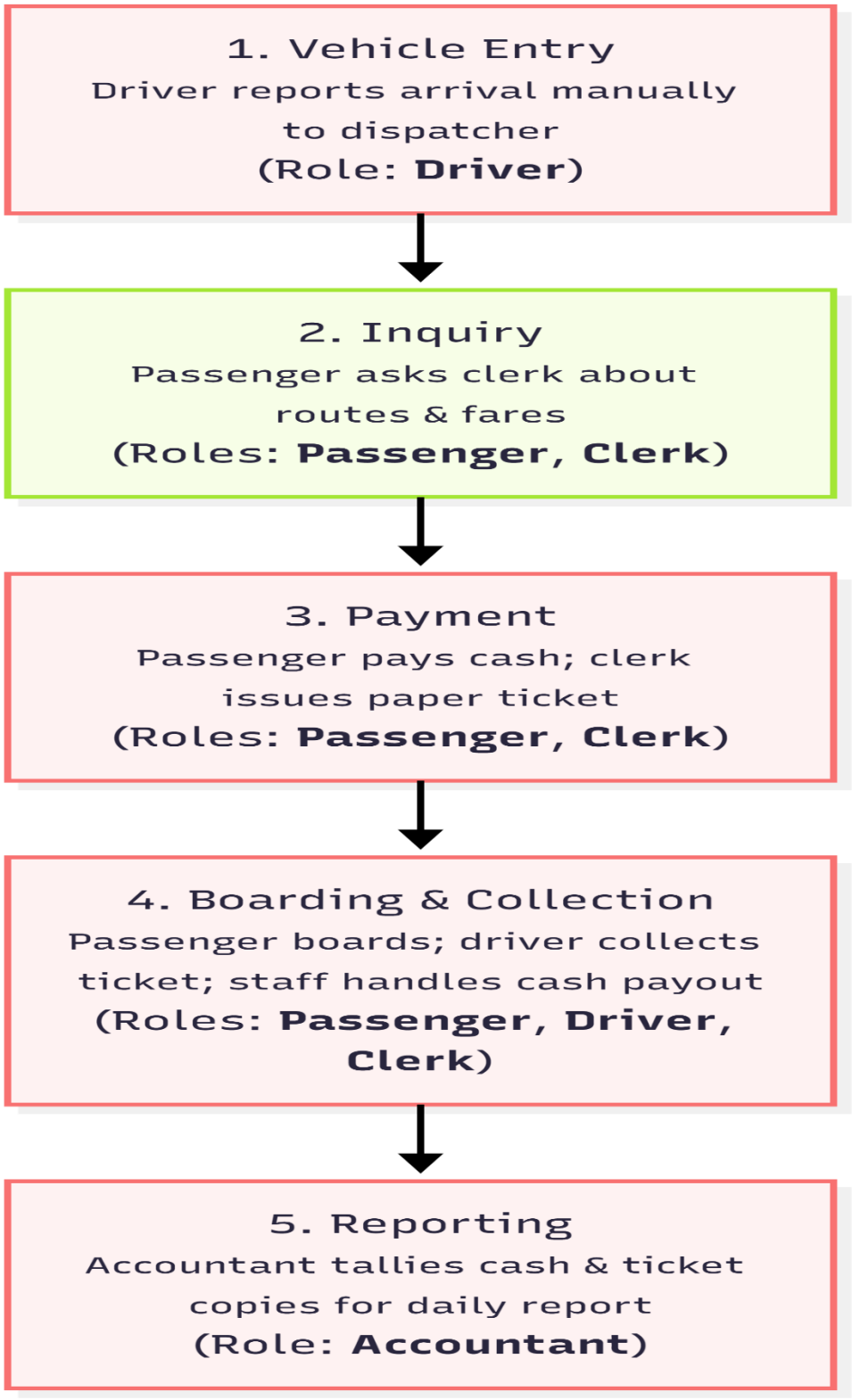
2.3 Structure of the Existing System (Process Flow)

The structural architecture of the existing manual system is defined by a linear, physical flow of information and currency:

[Vehicle Entry] ➔ [Inquiry] ➔ [Payment] ➔ [Boarding & Collection] ➔ [Reporting]

1. Vehicle Entry: The driver arrives at the terminal and manually reports their arrival to the dispatcher to secure a slot in the departure queue ledger.
2. Inquiry: The passenger physically walks to the terminal ticket windows to ask clerks about route availability, vehicle levels, and fare prices.
3. Payment: The passenger pays the full fare in cash, and the clerk issues a physical paper ticket stub.
4. Boarding & Collection: The passenger boards the vehicle, handing their ticket to the driver or owner. The terminal staff calculates the revenue split and dispenses the cash payout to the driver either at departure or upon their return trip.
5. Reporting: At the end of the operating day, a terminal accountant manually sums up the collected cash balances and counts the physical carbon copies of sold tickets to calculate total daily revenue and terminal service fees.

The structure of existing system process work flow is like below diagram.


1. Vehicle Entry
Driver reports arrival manually to dispatcher
(Role: Driver)
2. Inquiry
Passenger asks clerk about routes & fares
(Roles: Passenger, Clerk)
3. Payment
Passenger pays cash; clerk issues paper ticket
(Roles: Passenger, Clerk)
4. Boarding & Collection
Passenger boards; driver collects ticket; staff handles cash payout
(Roles: Passenger, Driver, Clerk)
5. Reporting
Accountant tallies cash & ticket copies for daily report
(Role: Accountant)

2.4 Key Limitations of the Current System

- Time Inefficiency: Because there is no remote access, passengers are forced to travel to the terminal twice—once to physically secure a ticket and another time to actually travel.
- Lack of Transparency: Ticket pricing data is confined to physical booths. Fare prices can occasionally fluctuate unexpectedly, leaving passengers with no independent way to verify if they are being charged the official rate set by the Ethiopian Transport Authority (ETA).
- High Data Vulnerability: Financial logs, queue ledgers, and ticket carbon copies are highly vulnerable to physical hazards. They can be easily misplaced, destroyed by weather conditions (such as rain or moisture), or altered without an audit trail, leading to a loss of financial accountability.
- Double-Booking Risks: In high-traffic terminal environments, poor communication between multiple booking clerks operating from separate physical ticket books can result in duplicate seat assignments for the same vehicle.
- Limited Regional Reach: Commuters living in rural areas or far from the city terminal have no way of knowing vehicle availability, queue lengths, or schedule statuses without incurring the time and cost of traveling to Woldia directly.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION

3.1. Proposed System Overview

The proposed Ethiopian Transport Scheduling and Ticket Booking System is a centralized, multi-platform digital solution engineered to completely replace the legacy manual, paper-based ticketing and dispatching processes at national transport terminals. The system architecture is built on a modular Service-Oriented Architecture (SOA), consisting of a robust backend REST API, a web administration dashboard for regulators and transport associations, and a mobile application for travelers. To ensure total consistency across the entire system specification, the system strictly models all transport entities uniformly as **Vehicles**, fully accommodating diverse public transit form factors—such as interstate buses, standard minibuses, and regional vans (Abadula vehicles)—within a unified system ledger.

At its core, the system transforms the traditional terminal-bound booking process into a dynamic, real-time digital experience through the following workflows:

- **Traveler Experience:** Using the mobile application, travelers can seamlessly browse available routes using multi-filter searches and filter options by official vehicle classifications (Level 1, Level 2, or Level 3). The system enforces a strict single-selection rule, preventing a user from selecting more than one seat or checking out multiple tickets within a single transaction session. The interface maps the account owner's registration details directly to the single ticket record to guarantee complete data integrity.
- **Concurrency and Data Integrity:** To eliminate the real-world double-booking vulnerabilities of the manual system, the backend implements a strict 10-minute transactional seat-locking mechanism that prevents race conditions during user checkout. If the transaction is not completed before the 10-minute countdown expires, the system automatically releases the seat back to open inventory.
- **Automated Financial Routing:** Transactions are secured through integrated local mobile payment gateways, including Telebirr, CBE Birr, and Chapa. Upon real-time backend verification of funds transfer, the system automatically routes the terminal's 5% administrative fee to the operating authority's ledger and updates the remaining 95% revenue split balance for the respective vehicle owner. This automated balance tracking ensures full transparency, regardless of whether actual payouts are disbursed at the time of boarding or upon destination return. Automated SMS alerts are dispatched immediately upon successful ticket validation.

For administrative, operational, and regulatory stakeholders, the web dashboard provides a comprehensive management suite:

- **Associate Managers:** Can digitalize and manage active fleet profiles, maintain a secure driver database, structure fair rotation-based queues using a strict First-In, First-Out (FIFO) rule based on terminal check-in logs, and coordinate owner payouts.
- **The Ethiopian Transport Authority (ETA):** Gains centralized oversight tools to define global interstate routes, monitor structural fleet records, and manage a dynamic tariff engine for automated, uniform price updates across the country based on distance baseline rates.

To bridge the digital divide and ensure complete inclusivity for all travelers, the system incorporates a Walk-in Booking Interface within the web platform. This allows terminal ticketing clerks to manually select an open seat, input a traveler's details, collect physical cash payments, and book a single ticket for walk-in travelers who do not use the mobile application.

Furthermore, terminal operational efficiency is modernized through real-time trip status updates and mobile utility tools. The system processes ticket data inputs captured via the Terminal Clerk's device camera to **scan the with scaner**. The system cross-checks the ticket records, verifies that the entry is valid and fully paid, and instantly updates the status to "Checked-In" inside the master system log, effectively replacing the manual physical ticket collection process. To ensure operational tracking without adding terminal bottlenecks, the system grants certified operators, owners, and drivers restricted access to view their vehicle's active Passenger Manifest starting exactly two hours prior to the scheduled departure time, blocking visibility prior to this threshold.

Finally, the system introduces an automated, rule-based ticket cancellation and refund mechanism to handle traveler itinerary changes. Travelers can request a cancellation directly through the system up to exactly 30 minutes before the vehicle’s scheduled departure time. If requested within the valid threshold, the system automatically returns the seat back to the open inventory pool and triggers an automated refund process. The system automatically rejects any cancellation requests submitted within the final 30 minutes before departure.

3.2 Functional Requirements

3.2.1 FR-URM: User Management & Authentication

I. FR-URM-01: Role-Based Access Control (RBAC)

The system automatically identifies who logs in and changes the features they can see based on their specific role:

- TRAVELER: Can check schedule availability, click a single open seat on a visual map, make a digital payment, view a list of their past trips, download a verification ticket code, and cancel a trip if they change their mind.

- TERMINAL CLERK: Can sell tickets to people walking up to the station window, scan travelers' ticket codes with a device camera to check them in, update boarding logs, and print out passenger lists.
- OPERATOR / DRIVER: Can log in to see their specific vehicle's official passenger list, track who has boarded, and monitor the money they have earned from their 95% share of ticket sales.
- ASSOCIATE MANAGER: Can register new vehicles into the system, manage profiles for drivers, place vehicles into a daily departure schedule queue, and look at revenue performance reports.
- SYSTEM ADMIN / ETA REGULATOR: Has total control to change user permissions, draw new routes on the map, change pricing rules, add new transport associations to the system, and configure global layouts.

I. FR-URM-02: User Registration & Profile Management

- Traveler Account Creation: Allows new travelers to set up an account by entering their Full Name, a password, and their Phone Number (which serves as their unique username identifier across the system).
- Single-Ticket Form Constraints: When a traveler goes to buy a ticket, the system blocks them from picking more than one seat. The system instantly links the person's account name and phone number straight to that single seat booking.
- Profile Access: Gives every user a private homepage where they can edit their information, view active digital tickets, and view an archive of their old trips.

II. FR-URM-03: Access Authentication

- Identity Verification: When a user logs in, the system double-checks that their password matches their phone number and keeps their account securely connected while they use the app.
- Access Control: The system watches every click a user makes. If an ordinary traveler tries to view a driver's financial page or an admin settings page, the system automatically blocks the action.

3.2.2 FR-FLM: Vehicle Fleet & Association Management

I. FR-FLM-01: Association Onboarding

- Registration: Allows the high-level System Admin to add a new transport association to the platform by inputting its business name, licensing paperwork, and the physical terminal hub it operates from.

II. FR-FLM-02: Vehicle, Driver, and Capacity Registration

- System-Wide Vehicle Generalization: The system standardizes all transport machinery under the single word "Vehicle" instead of calling them buses or minibuses, making sure everything is counted the same way.

- Fleet Profiling: Allows Associate Managers to input new fleet inventory into the system by typing in the vehicle's unique License Plate Number and side operational tracking number.
- Grade Categorization: The system tags each vehicle as Level 1, Level 2, or Level 3. The system uses these level tags to automatically adjust and calculate the ticket price later.
- Capacity Definitions: When a vehicle is saved, the system automatically generates its seat map configuration based on its level (for example: loading a 45-seat map layout for a Level 1 vehicle or a 60-seat layout for Level 2/3).
- Driver Records: Keeps a digital file of all active drivers and links each driver directly to the association they work for.

III. FR-FLM-03: Route and Rotation Schedule Management

- Route Definition: Allows the System Admin to program travel paths by setting the exact departure station, arrival destination, specific intermediate rest stops, and total distance.
- Rotation Queuing: Allows Associate Managers or dispatchers to drop a vehicle and driver into a departure schedule. The system automatically organizes the line using a strict First-In, First-Out (FIFO) rule—meaning the vehicle that checks into the terminal first gets placed at the front of the line to leave first.

IV. FR-FLM-04: Dynamic Tariff Configuration

- Tariff Processing: Allows the ETA Admin to change the base pricing rate per kilometer or per vehicle level. The moment the admin changes this rate, the system automatically runs a calculation and updates ticket prices for all active trips across the country.

3.2.3 FR-TKM: Search, Selection & Booking Management

I. FR-TKM-01: Multi-Filter Availability Search

- Search Execution: Allows a traveler to type in where they are starting, where they want to go, their travel date, and what vehicle level they want. The system immediately outputs a real-time list of matching vehicles that have open seats.

II. FR-TKM-02: Graphical Seat Selection

- Layout Matrix Display: Shows the traveler an interactive map of the inside of the vehicle. It uses clear colors so the user can easily see which seats are already taken, which are open, and which one they have clicked.
- Single Booking Constraint: The system stops the user from clicking more than one seat on the map, enforcing the rule that a traveler can only purchase a single ticket per transaction.

III. FR-TKM-03: Transactional Seat Locking

- Seat Lock Mechanism: The exact second a traveler selects an open seat, the system changes that seat's status to "Locked" and holds it for exactly 10 minutes so nobody else can buy it while the traveler types in their payment details.

- Automated Release: The system runs a countdown timer. If 10 minutes pass and the traveler has not completed their payment, the system unlocks the seat and changes its status back to "Available" for other customers.

IV. FR-TKM-04: Walk-in Booking Interface

- Clerk Sales Portal: Provides a layout for terminal clerks to book tickets for travelers who do not have smartphones. The clerk can manually choose a seat on the screen, type in the traveler's name, collect cash, and print out a physical ticket receipt.

3.2.4 FR-PYM: Payment, Ticketing & Notifications

I. FR-PYM-01: Payment Gateway Processing

- Transaction Verification: Connects to local mobile money networks (Telebirr, CBE Birr, and Chapa). The system waits to receive an official digital confirmation that the traveler's money has been transferred before finalizing the ticket sale.

II. FR-PYM-02: Automated Revenue Splitting

- Financial Splitting: The exact moment payment is confirmed, the system splits the fare automatically: it takes exactly 5% and sends it to the terminal authority's account for administrative fees, and places the remaining 95% directly into the vehicle operator's digital balance.

III. FR-PYM-03: Digital Ticket & QR Generation

- Ticket Issuance: Once the payment is successful, the system generates an official digital ticket that contains the traveler's name, vehicle license plate, date of travel, seat number, and an encrypted scannable verification code.

IV. FR-PYM-04: Automated Notifications

- SMS Dispatch: The system instantly sends a text message to the traveler's phone containing their ticket confirmation, seat number, and vehicle information.
- Offline Access: The system saves the digital ticket directly into the traveler's mobile app storage, allowing them to open and display their verification code to ticket clerks even if they have no mobile data or internet at the station gate.

V. FR-PYM-05: Time-Bounded Cancellation & Refunds

- Cancellation Processing: Allows a traveler to hit a "Cancel Ticket" button on their app up to exactly 30 minutes before the vehicle is scheduled to leave.
- Inventory & Refund Automation: If the traveler cancels before the 30-minute deadline, the system automatically marks their seat as "Available" again and triggers a digital refund

back to their mobile wallet. If they try to cancel within the final 30 minutes before departure, the system automatically rejects the request.

3.2.5 FR-OPS: Dispatch & Terminal Operations

I. FR-OPS-01: Scan the with Scaner

- Ticket Verification Function: The system shall process ticket data inputs captured via the Terminal Clerk's device camera scanning a ticket code. The system shall cross-check the ticket records, verify that the entry is valid and fully paid, and instantly update the status to "Checked-In" inside the master system log.

II. FR-OPS-02: Restricted Manifest Visibility

- Manifest Access Control: The system hides the digital Passenger Manifest from drivers and vehicle owners until exactly two hours before the scheduled departure time. The system completely blocks them from opening or looking at the passenger list prior to this two-hour mark.

III. FR-OPS-03: Real-Time Trip Status Synchronization

- Status Updates: Allows dispatchers to click a button to change a trip's status (such as moving it from "Boarding" to "Delayed" or "Departed"). The system immediately changes this status on the traveler’s phone app and on the terminal's overhead display screens.

3.2.6 FR-REP: Reporting & Analytics

I. FR-REP-01: Financial Reconciliation Reporting

- Statement Generation: Automatically generates accounting spreadsheets that track total money collected. Users can filter these reports by day, week, or month to see exactly how much money was kept as the 5% terminal fee and how much was paid out as the 95% operator share.

II. FR-REP-02: Traffic and Capacity Optimization Analytics

- Throughput Metrics: Evaluates historical travel data to pinpoint exactly which hours of the day are busiest and which routes have the highest traveler traffic, helping transport planners balance the number of vehicles allocated to different routes.

3.2.7 Functional Requirement Traceability Matrix

ID	Requirement Name	Priority	Actor	Description
FR-URM-01	Role-Based Access Control (RBAC)	Critical	All Users	Automatically alters system menus and functional access permissions depending on the logged-in role.
FR-URM-02	Registration & Profile Management	High	Traveler	Manages traveler accounts and enforces the strict single-ticket selection booking limit per session.
FR-URM-03	Access Authentication	Critical	All Users	Manages secure identity verification during login and dynamically blocks unauthorized system actions.
FR-FLM-01	Association Onboarding	Medium	System Admin (ETA)	Records certified transport association profile entities into the centralized platform directory.
FR-FLM-02	Vehicle Driver & Capacity Registration	High	Associate Manager	Uniformly profiles transport machinery under the category of "Vehicles" along with level classifications.
FR-FLM-03	Route & Schedule Management	High	Admin / Dispatcher	Records travel paths and organizes scheduling departures using an automated First-In First-Out (FIFO) queue.
FR-FLM-04	Dynamic Tariff Configuration	High	System Admin (ETA)	Automatically recomputes real-time ticket fare values from distance-based baseline tariff changes.
FR-TKM-01	Multi-Filter Real-Time Search	High	Traveler	Leverages multi-filter parameters to output real-time available vehicle paths and open seat counts.
FR-TKM-02	Single-Seat Matrix Selection	High	Traveler	Renders interactive vehicle maps while programmatically limiting selections to one seat per session.
FR-TKM-03	Transactional Seat Locking (10 min)	Critical	System Engine	Temporarily isolates a selected seat inventory by applying a strict 10-minute automated countdown lock.

FR-TKM-04	Walk-in Booking Interface	High	Terminal Clerk	Accepts manual traveler data inputs to generate a single valid ticket stub for offline customers.
FR-PYM-01	Payment Gateway Processing	High	Traveler / System	Processes financial verification requests with local mobile money providers before finalizing sales.
FR-PYM-02	Automated Revenue Splitting	High	System Engine	Programmatically parses gross fares into the 5% terminal operating fee and the 95% operator ledger share.
FR-PYM-03	Digital Ticket & Code Generation	High	System Engine	Compiles single-ticket data parameters into a secure digital receipt with an encrypted verification code.
FR-PYM-04	Automated Notifications & Offline Access	Medium	System Engine	Automatically dispatches confirmation SMS alerts and manages local storage files for offline gate view.
FR-PYM-05	Time-Bounded Cancellation & Refunds	Medium	Traveler / System	Executes automated wallet refunds and seat release up to exactly 30 minutes prior to scheduled departure.
FR-OPS-01	Scan the with Scaner	High	Terminal Clerk	Uses device camera code capture to verify payment authenticity and update travel status to "Checked-In".
FR-OPS-02	Restricted Manifest Visibility	High	Driver / Operator	Dynamically restricts and opens visibility of the digital Passenger Manifest exactly two hours before departure.
FR-OPS-03	Real-Time Trip Status Updates	Medium	Dispatcher / Traveler	Synchronizes operational journey state shifts across traveler mobile interfaces and terminal display monitors.
FR-REP-01	Financial Reconciliation Reporting	Medium	Admin / Associate Manager	Automatically compiles downloadable revenue spreadsheets filtered by dates routes or association cuts.
FR-REP-02	Traffic & Operational Analytics	Low	System Admin (ETA)	Evaluates historical traveler throughput data trends to optimize resource allocation across busy corridors.

3.3. Non-Functional Requirements

3.3.1 Performance Requirements

- Response Time: The system shall process single-seat selections, transactional seat-locking requests, and payment confirmations within a maximum of 3 seconds under normal operating loads.
- Throughput: The system shall maintain concurrent, steady operational states for at least 200 active traveler, clerk, and manager sessions simultaneously without experiencing speed degradation or transaction dropouts.
- Latency: The system shall retrieve and refresh vehicle schedule availability arrays and visual seat layout matrices in less than 1 second following an execution request.

3.3.2 Security Requirements

- Authentication: The system shall protect all administrative, association management, and terminal clerk interfaces using secure identity checks and robust credential complexity rules.
- Data Encryption: The system shall secure traveler profiles, payment transactions, and identity records by applying industry-standard data masking and encryption both while moving across networks and while stored within the system repository.
- Authorization (RBAC): The system shall strictly isolate user actions based on roles, ensuring an ordinary traveler cannot view terminal financial ledgers, and a Terminal Clerk cannot modify the national baseline tariff rules.

3.3.3 Usability Requirements

- Learnability: The terminal clerk user interface shall be simple enough that a new station worker can execute basic tasks—such as walk-in ticketing or using the device camera to scan the with scaner—within 1 hour of orientation.
- Responsiveness: The system views shall automatically scale, adapt, and remain fully legible across diverse screen formats, including traveler mobile applications, clerk utility tools, and manager desktop layout portals.
- System Feedback: The system shall instantly provide clear visual success badges for confirmed single-ticket bookings and display explicit, understandable text notices when an error or payment failure occurs.

3.3.4 Reliability Requirements

- Transaction Rollback: The system shall enforce total transactional completion rules; if a network outage occurs midway through a digital booking checkout, the system shall completely cancel the session and release the seat to prevent broken inventory allocations.
- Fault Recovery: The system shall recover gracefully from sudden database connection drops or session interruptions without corrupting existing traveler itineraries, manifest histories, or active 10-minute seat locks.

3.3.5 Availability Requirements

- Operational Uptime: The system shall maintain a continuous operational availability target of 99.9% during primary terminal operating windows, specifically from 6:00 AM to 10:00 PM local time daily.
- Maintenance Scheduling: The system shall restrict planned administrative updates and offline configurations strictly to off-peak night hours, requiring system alerts to be broadcasted to transport associations 24 hours in advance.

3.3.6 Scalability Requirements

- Data Growth Handling: The system data repository shall maintain optimal sorting and retrieval speeds while scaling up to accommodate a baseline archive of 100,000 active traveler accounts and 5 years of historical trip records.
- Demand Surge Management: The system framework shall support seamless distribution across multiple processing nodes to absorb massive traveler search and booking volumes during high-congestion holiday travel seasons.

3.3.7 Maintainability Requirements

- Functional Modularity: The system shall separate its operational business modules cleanly so that updating or modifying the local payment gateway integrations will not affect or break the core vehicle scheduling and rotation engines.
- Operational Logging: The system shall continuously populate organized internal logs detailing system exceptions, tariff adjustments, and booking errors to allow for rapid troubleshooting and maintenance.

3.3.8 Compatibility Requirements

- Web Runtime Compatibility: The web-based admin dashboards for Associate Managers and ETA Regulators shall load and function uniformly across all major modern internet web browsers.
- Data Export Compatibility: The system reporting engines shall provide clean data exports into industry-standard, document-ready structures (such as CSV and PDF) for seamless regulatory reporting to the Ethiopian Transport Authority (ETA).

3.3.9 Portability Requirements

- Environment Independence: The system backend services and data structures shall be capable of operating identically across both standard Linux and Windows operating environments without modification to business logic.

3.3.10 Data Integrity & Accuracy Requirements

- Financial Precision: The system calculation engine shall compute all tariff distances, individual ticket costs, terminal authority operating cuts (the 5% allocation), and vehicle operator disbursements (the 95% allocation) with accurate precision to exactly two decimal places.
- Capacity and Seat Validation: The system shall strictly block the generation of duplicate seat assignments and guarantee that total tickets issued for any single trip can never exceed the pre-configured seat matrix limit of the assigned vehicle.

3.3.11 Legal & Compliance Requirements

- Regulatory Compliance: The system shall structure its digital manifest grids and downloadable financial reconciliation summaries to match the formal documentation rules enforced by the ETA.
- Data Privacy Protection: The system shall protect traveler identity records, permanently guarding traveler names, phone numbers, and transaction paths from unauthorized distribution.

3.3.12 Localization & Language Requirements

- Dual-Language Interface: The system shall provide comprehensive multi-language support, allowing any user to toggle all visible interfaces, forms, alerts, and instructions instantly between Amharic and English.
- Calendar Synchronization: The system shall seamlessly manage dual-calendar operations, displaying and processing scheduling matrices and traveler tickets in both the Gregorian and Ethiopian calendars.

3.3.13 Backup & Recovery Requirements

- Backup Frequency: The system shall execute automated, hands-free incremental backups daily to capture all active single-ticket transactions, financial cuts, and compiled traveler manifests.
- Disaster Recovery Target: The system shall be structurally restorable from a fatal system crash or host failure back to its last verified backup state within a maximum timeline of 4 hours.

3.3.14 Ethical & Transparency Requirements

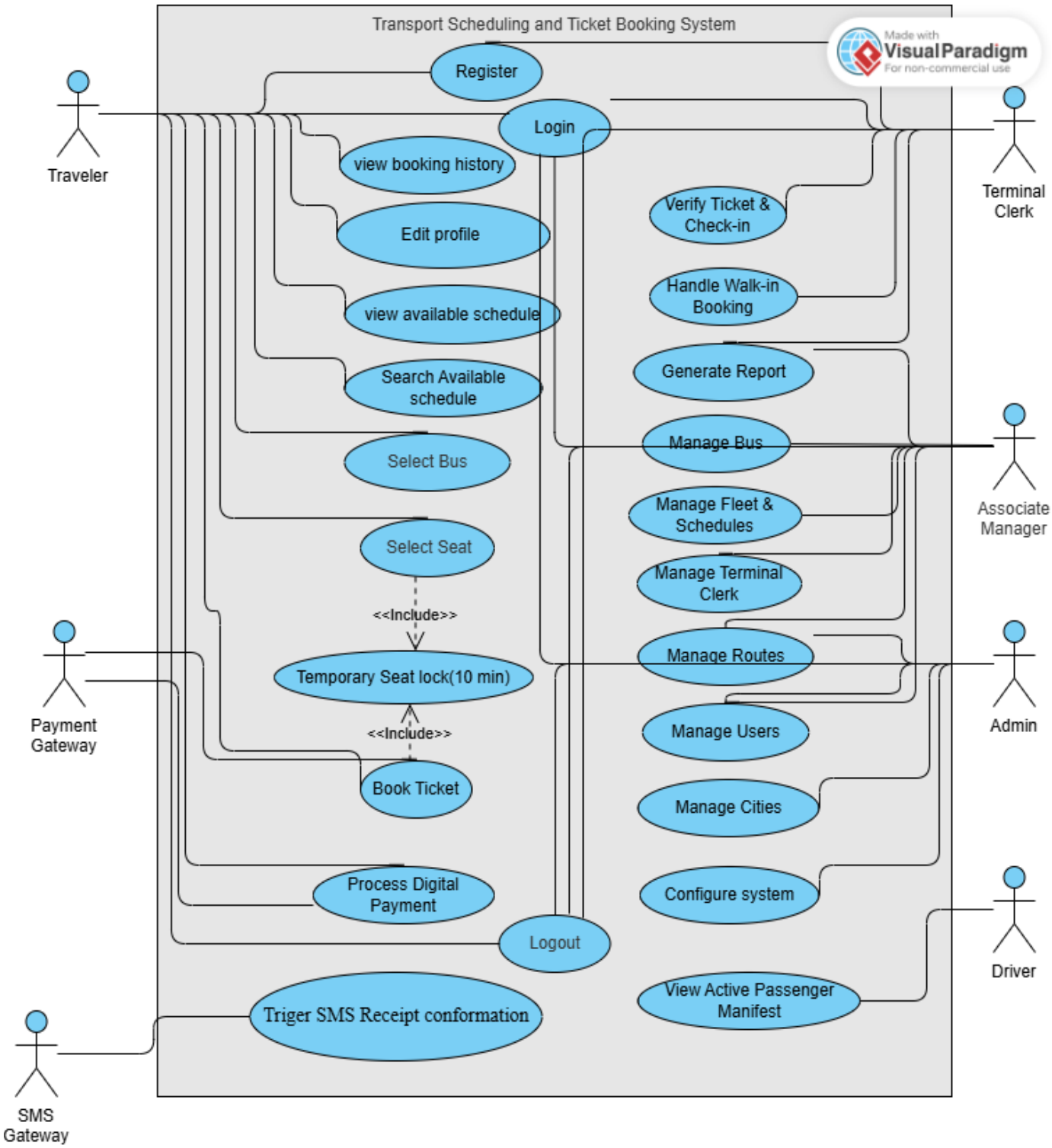
- Immutable Audit Logs: The system shall maintain an unalterable history ledger, stamping every route addition, tariff adjustment, ticket cancellation, and vehicle rotation shift with the exact chronological timestamp and user ID.
- Fare Breakdown Transparency: The system shall provide travelers with an explicit, itemized receipt view during checkout, showing the baseline route cost alongside the designated service fees before prompting payment confirmation.

3.3.15 Operational Constraints

- Network Connectivity Dependency: The system requires continuous, stable internet connectivity at verification checkpoints to run live payment validations, process live FIFO queues, and synchronize real-time seat states.
- Resource Optimization: The system services shall be highly optimized to run efficiently within standard administrative computing bounds, minimizing memory consumption to ensure rapid data transactions under typical regional host configurations.

3.4 Use Case Models

3.4.1 Use Case Diagram




Ethiopian Transport Scheduling and Ticket Booking System
Activate Transactional Seat Locking (10 min)
«include»
Select Single Seat
Trigger SMS Receipt Confirmation
smsSys
«extend»
Automated Revenue Splitting (5% Terminal / 95% Operator)
«include»
Process Digital Payment
«include»
Book Single Ticket
«include»
«include»
Process Walk-In Booking
paymentSys
Process Automated Wallet Refund
«extend»
Cancel Ticket Booking
Search Route Availability
Check Visibility Window (2-hour prior rule)
«include»
View Active Passenger Manifest
Profile Fleet Assets & Drivers
Apply Automated FIFO Queuing Rules
«include»
Assign Route Departure Rotation
Configure Network Routes
Cascading Price Recalculation
«include»
Manage Central Tariff Engine
Traveler
Terminal Clerk
Operator / Driver
Associate Manager
System Admin / ETA Regulator

3.4.2 Use Case Descriptions

The following tables provide a detailed breakdown of the high-priority use cases identified in the diagram above.

Use Case 1: Register

Field	Details
Use Case ID	UC-01
Use Case Name	Register Account
Actor(s)	Traveler, Terminal Clerk, Operator / Driver, Associate Manager
Description	Allows any new standard user role (except the System Admin / ETA Regulator, who is pre-configured dynamically at deployment) to create a personal account in the system to access their respective system dashboards and tools.
Preconditions	* The user must have accessed the application registration portal on their mobile device or workstation. * The user must not have an existing active profile registered under their current phone number.
Basic Flow	1. The user opens the registration page and selects their intended role category (Traveler, Terminal Clerk, Operator/Driver, or Associate Manager). 2. The system dynamically updates the input form fields based on the selected role. 3. The system prompts the user to input their Full Name, Phone Number, and a secure Password. 4. The user completes the required text forms and submits the registration data. 5. The system parses the submission inputs to ensure formatting rules are correct and complexity constraints are fulfilled. 6. The system checks the master repository to ensure the phone number identifier is completely unique across all roles. 7. The system hashes the password, saves the profile entity into the database log, and assigns the selected operational role metadata tag. 8. The system displays a registration success confirmation notice and redirects the newly registered user to the secure login screen.
Alternative Flows	* **Alt Flow 1: Phone Number Identification Conflict (Step 6)** 1. The system flags that the provided phone number is already actively assigned to an account block. 2. The system terminates the database save process. 3. The system generates a "Registration Failed: Identifier Already Registered" error message and prompts the user to use the recovery options. * **Alt Flow 2: Form Entry Validation Drop (Step 5)** 1. The system identifies that a mandatory fields is blank or structured incorrectly. 2. The system pauses submission tracking. 3. The system highlights the invalid field line with contextual helper text, allowing the user to re-input data.
Postconditions	* **Successful:** A new, verified user identity is committed to the central database under the specified role tier, granting clear clearance for subsequent authentication sessions. * **Unsuccessful:** No database rows are written, the operation is completely canceled, and the user interface clears fields or retains previous form state data for correction.

Use Case 2: Login

Field	Details
Use Case ID	UC-02
Use Case Name	Secure Access Authentication (Login)
Actor(s)	Traveler, Terminal Clerk, Operator / Driver, Associate Manager, System Admin / ETA Regulator
Description	Allows an existing registered user to authenticate their identity using their unique identifier (Phone Number) and password to access their role-specific dashboard and functions.
Preconditions	* The user must have a verified account already registered within the system database ledger. * The user must display the login interface on their mobile device or administrative web workstation.
Basic Flow	1. The user opens the application login interface. 2. The system prompts the user to input their registered Phone Number and Password. 3. The user inputs their unique credentials into the provided form fields and executes the submission request. 4. The system securely transfers the login data parameters to the backend processing layer. 5. The system cross-checks the credentials against the user records inside the master database repository. 6. The system verifies the matching record and retrieves the specific security authorization role metadata tag (Traveler, Terminal Clerk, Operator/Driver, Associate Manager, or Admin) linked to that unique account. 7. The system establishes a secure active application session for the user. 8. The system displays a brief welcome success message and dynamically redirects the user to their designated role-specific system interface dashboard.
Alternative Flows	* Alt Flow 1: Invalid Credentials Configuration (Step 5) 1. The system flags that the phone number does not exist or the entered password does not match the stored secure database record. 2. The system blocks dashboard entry processing. 3. The system generates an "Authentication Failed: Invalid Phone Number or Password" error alert and resets the password form field for a retry. * Alt Flow 2: Form Entry Omission (Step 3) 1. The system detects that either the phone input line or password field is completely blank during the submission request. 2. The system pauses submission validation tracking. 3. The system highlights the omitted field with a "This field is mandatory" inline prompt, preventing unnecessary database request traffic.
Postconditions	* Successful: An active, encrypted role-based authorization token/session is assigned to the interface, modifying the menus and visible operational features completely based on the user's role. * Unsuccessful: Access to restricted system features remains blocked, no session footprint is created, and the user is kept at the login interface with validation error prompts.

Use Case 3: Edit Profile

Field	Details
Use Case ID	UC-03
Use Case Name	Edit Profile
Actor(s)	Traveler, Terminal Clerk, Operator / Driver, Associate Manager
Description	Allows an authenticated user to update their personal registry information, modify account details, and manage their system account parameters.
Preconditions	* The user must have a secure, active login session established within the application environment. * The user must navigate to their account homepage or personal settings interface block.
Basic Flow	1. The user clicks on the "Profile Settings" or "Account Management" option on their dashboard view. 2. The system queries the database layer, retrieves the current profile fields, and displays them within an editable text layout grid. 3. The user alters their personal details (such as editing their Full Name, entering a new Password, or modifying secondary notification options). 4. The user verifies their modifications and clicks the "Save Changes" execution button. 5. The system captures the modified form inputs and reviews data formats against standard type parameters. 6. The system checks if the primary Phone Number is being updated; if so, it validates that the new phone number remains completely unique across the registry log. 7. The system commits the fresh string parameters to the user profile row in the master database ledger. 8. The system pops up a "Profile Updated Successfully" notification message and refreshes the dashboard view with the new data records.
Alternative Flows	* **Alt Flow 1: Phone Identifier Collision (Step 6)** 1. The system determines that the newly inputted primary phone number is already assigned to a different active profile item. 2. The system aborts the database write operation. 3. The system generates an inline "Update Failed: Phone Number Already in Use" error prompt and retains the unsaved form values for adjustment. * **Alt Flow 2: Security Verification Dropout (Step 5)** 1. The user attempts to set a new password, but the complexity validation marks it as weak, or the mandatory "Current Password" confirmation field fails. 2. The system suspends database tracking updates. 3. The system highlights the invalid field line with contextual warning labels, prompting a secure re-input.
Postconditions	* **Successful:** The updated profile parameters are permanently overwritten into the central user ledger, updating all dynamic interfaces across the platform immediately. * **Unsuccessful:** No changes are written to the database repository, previous entity values are preserved intact, and the user interface loops back to display validation warnings on the active form.

Use Case 4: View Booking History

Field	Details
Use Case ID	UC-04
Use Case Name	View Booking History
Actor(s)	Traveler
Description	Allows an authenticated traveler to access a comprehensive archive of their completed, canceled, and currently active single-ticket trip reservations.
Preconditions	* The traveler must have a secure, active login session established within the mobile or web application interface. * The traveler must navigate to their personal itinerary dashboard or profile archive section.
Basic Flow	1. The Traveler clicks on the "My Trips", "Booking History", or "Itinerary Archive" option on their navigation panel. 2. The system tracks the unique Traveler ID associated with the current encrypted session. 3. The system queries the centralized transaction ledger to pull all reservation rows matching that specific Traveler ID. 4. The system sorts the retrieved records chronologically, separating them into two clear tabs: "Active/Upcoming Journeys" and "Past/Canceled Trips". 5. The system dynamically renders the historical grid layout on screen, displaying the trip dates, route paths (departure and arrival terminals), assigned vehicle license plate numbers, seat coordinates, fare costs, and transaction status values (e.g., Paid, Checked-In, Canceled). 6. The traveler views the summarized history logs or clicks on an upcoming active entry to view and pull up their encrypted scannable verification code.
Alternative Flows	* **Alt Flow 1: Empty Registry State (Step 4)** 1. The system database query returns zero matching reservation rows for the specific Traveler ID. 2. The system stops processing layout data filters. 3. The system displays a clean, user-friendly "No Bookings Found" placeholder screen along with a quick-link button directing the traveler back to the route availability search tool.
Postconditions	* **Successful:** The traveler successfully receives and views an accurate, real-time localized display of their entire system booking history without modifying any system values. * **Unsuccessful:** If a network interruption occurs, the data retrieval fails, and the system prompts an offline state warning or a "Failed to sync history. Retry" error notification.

Use Case 5: Manage Users

Field	Details
Use Case ID	UC-05
Use Case Name	Manage Users
Actor(s)	System Admin / ETA Regulator
Description	Allows top-level system administrators to view, audit, modify role permissions, or suspend user profile accounts across the entire platform directory to maintain system security and oversight.
Preconditions	* The administrator must be securely authenticated with administrative role clearances. * The administrator must navigate to the centralized "User Management" workspace panel.
Basic Flow	1. The System Admin clicks on the "Manage Users" option on the primary administration command board. 2. The system queries the entire global user directory and displays an organized layout grid listing users by Name, Unique Phone Number, Role, Account Status (Active/Suspended), and Registration Date. 3. The admin inputs criteria into a search bar (such as filtering by a specific phone identifier or grouping profiles by a role like "Terminal Clerk"). 4. The system updates the view to display matching accounts instantly. 5. The admin selects a specific user account entry to open its detailed management window. 6. The admin updates configuration states, such as manually altering an operational role tag or switching an account state toggle from "Active" to "Suspended". 7. The admin confirms the structural adjustments and executes the save command. 8. The system updates the account database record, pushes changes to the central ledger log, and flashes a "User Permissions Modified Successfully" success text.
Alternative Flows	* **Alt Flow 1: Self-Demotion Block Mechanism (Step 6)** 1. The System Admin attempts to strip administrative permissions or suspend their own currently logged-in account profile. 2. The system intercept engine recognizes the identity match and blocks the data submission. 3. The system raises a critical "Operation Denied: Administrative Self-Modification Prohibited" warning message to preserve system access integrity. * **Alt Flow 2: Target Record Deletion Conflict (Step 6)** 1. The admin attempts to purge an account that has open transactional dependencies (e.g., an Associate Manager with a vehicle currently placed in a live FIFO queue). 2. The system stops the deletion to preserve database constraints. 3. The system prompts the admin to reassign or close active assets before modifying or archiving the user account.
Postconditions	* **Successful:** The target user's metadata role tags or security visibility states are updated in the repository, immediately altering their interface access parameters upon their next network action.

	* **Unsuccessful:** No database records are written or modified, the user account state remains in its original configuration, and the dashboard returns an explicit administrative error message.

Use Case 6: Logout

Field	Details
Use Case ID	UC-06
Use Case Name	Logout
Actor(s)	All users (Traveler, Terminal Clerk, Operator / Driver, Associate Manager, System Admin / ETA Regulator)
Description	Safely terminates an active authenticated user session, clears local credentials memory from the interface interface client, and guarantees secure system closure.
Preconditions	* The user must be actively logged into the platform with an authenticated, operational session token.
Basic Flow	1. The user clicks on the "Logout" button, located on the profile page or the main navigation menu sidebar panel. 2. The application intercepts the request and prompts the user with a confirmation modal asking, "Are you sure you want to log out?". 3. The user confirms the action by clicking "Yes". 4. The system communicates with the server backend to immediately invalidate the current session token or session record. 5. The client application flushes all temporary session memory storage parameters and local role authorizations from the device. 6. The system gracefully redirects the user interface view back to the public, unauthenticated landing or login page panel. 7. The system displays a brief "Logged Out Safely" text pop-up.
Alternative Flows	* **Alt Flow 1: User Cancels Logout Action (Step 3)** 1. The user clicks "No" or clicks away from the logout confirmation dialog box. 2. The system dismisses the confirmation window interface layer. 3. The system returns the user directly back to their active dashboard layout block without interrupting the current session state.
Postconditions	* **Successful:** The secure interaction session is entirely severed. The user profile is returned to an unauthenticated status state, preventing unauthorized browser back-button access or menu tampering. * **Unsuccessful:** The confirmation window drops without clearing authorizations, keeping the encrypted user session active and fully functional.

Use Case 7: Profile Fleet Assets & Drivers

Field	Details
Use Case ID	UC-07
Use Case Name	Profile Fleet Assets & Drivers
Actor(s)	Associate Manager
Description	Allows an Associate Manager to input and register new transportation hardware under the standardized "Vehicle" classification, specify its grade level, automatically map its seat capacity layout, and link certified drivers directly to their association row.
Preconditions	* The Associate Manager must be securely logged in with verified association administrative permissions. * The manager must navigate to the "Fleet Management & Inventory" workspace view.
Basic Flow	1. The Associate Manager clicks on the "Add New Vehicle Profile" action button. 2. The system opens a data configuration layout prompting for specific hardware markers. 3. The manager types in the vehicle's unique License Plate Number and internal side operational tracking number. 4. The manager selects the vehicle's certified quality classification status tier: Level 1, Level 2, or Level 3. 5. The system automatically intercepts the level selection to apply standard capacity definitions (e.g., mapping a 45-seat blueprint layout for Level 1, or a 60-seat layout for Level 2 or Level 3). 6. The manager searches for an unassigned active driver name via the association's personnel registry menu. 7. The manager links the selected driver profile directly to this vehicle row and submits the form. 8. The system cross-verifies data inputs, saves the asset record into the database log, and displays a "Vehicle Asset Profiled Successfully" success text.
Alternative Flows	* **Alt Flow 1: Asset Identifier Collision (Step 8)** 1. The system scans the network and identifies that the inputted License Plate Number is already registered under an active deployment block. 2. The system terminates the database save routine. 3. The system highlights the entry row, displays a "Registration Refused: License Plate Already Exists" error badge, and prompts for correction. * **Alt Flow 2: Driver Allocation Conflict (Step 7)** 1. The system notes that the selected driver is already tied to another active operating vehicle inside the live fleet data log. 2. The system pauses submission validation tracking. 3. The system renders an alert text warning stating "Driver Currently Assigned to Another Vehicle" and prompts the manager to reassign or clear the previous link.
Postconditions	* **Successful:** The vehicle is added to the system directory with its standardized seat layout, making it available to participate in terminal line-up rotation queues. * **Unsuccessful:** No database rows are written, previous fleet states remain unaltered, and the manager is retained on the entry form panel with clear error markers.

Use Case 8: Manage Bus/Vehicle Records

Field	Details
Use Case ID	UC-08
Use Case Name	Manage Bus/Vehicle Records
Actor(s)	Associate Manager
Description	Allows an Associate Manager to view, edit, or update operational status details for existing vehicle profiles registered within their specific transport association.
Preconditions	* The Associate Manager must be securely authenticated and authorized for their association branch. * The manager must navigate to the active "Fleet Inventory List" workspace block.
Basic Flow	1. The Associate Manager clicks on the "Manage Fleet Records" option on their administration panel. 2. The system queries the database and displays an organized table tracking all vehicles assigned to that manager's association by License Plate, Side Number, Classification Level, and current Status (e.g., Active, Maintenance, In-Queue). 3. The manager selects a specific vehicle profile entity from the dashboard view. 4. The system populates the current details of the vehicle into an editable modification form. 5. The manager modifies the necessary tracking metrics, such as reassigning a new Driver profile or toggling the vehicle state to "Maintenance" due to a mechanical issue. 6. The manager clicks on the "Update Record" button. 7. The system cross-verifies that any updated data fields remain compliant with standard system constraints. 8. The system commits the updated configuration parameters to the fleet database ledger and highlights the line entry with a "Vehicle Record Updated Successfully" notification banner.
Alternative Flows	* **Alt Flow 1: Active Rotation Queue Lockout (Step 5)** 1. The manager attempts to change a vehicle's status to "Maintenance" or alter its level configuration while that vehicle is actively placed in a live departure line-up sequence. 2. The system scheduling engine intercepts the modification request and blocks it. 3. The system generates a "Modification Denied: Vehicle is currently locked in an active FIFO queue" error prompt, requiring the manager to manually remove the vehicle from the rotation line before editing its record.
Postconditions	* **Successful:** The modified vehicle entity parameters are updated within the database directory, immediately altering the asset's operational availability state across the dispatch modules. * **Unsuccessful:** No changes are saved to the database repository, previous structural vehicle logs remain completely unaltered, and the user interface returns a contextual layout error flag.

Use Case 9: Onboard Transport Association

Field	Details
Use Case ID	UC-09
Use Case Name	Onboard Transport Association
Actor(s)	System Admin / ETA Regulator
Description	Allows top-level administrative regulators to legally register and provision a brand new corporate transit association entity into the centralized platform directory.
Preconditions	* The administrator must be securely authenticated with peak System Admin role clearance credentials. * The administrator must navigate to the centralized "Association Directory" administration portal.
Basic Flow	1. The System Admin clicks on the "Onboard New Association" registration button. 2. The system opens a fresh corporate intake form, prompting for organizational parameters. 3. The admin inputs the Association Official Name, Business License Reference ID, Regional Office Hub, and Primary Liaison Contact Phone Number. 4. The admin creates the first authorized "Associate Manager" account profile linked directly to this corporate profile block. 5. The admin submits the completed onboarding form framework to the backend database layer. 6. The system runs integrity checks to verify that the Business License ID and Liaison Contact Phone Number do not exist anywhere in the active directory records. 7. The system creates a new corporate entry row, allocates a dedicated association namespace data block, and saves the entity details. 8. The system displays an "Association Onboarded and Provisioned Successfully" success message and automatically dispatches login setup codes to the newly designated Associate Manager's phone.
Alternative Flows	* **Alt Flow 1: Legal Registration Conflict (Step 6)** 1. The system identifies that the entered Business License Reference ID or Association Name matches an already onboarded operational transit entity. 2. The system halts the registration wizard process. 3. The system returns an administrative alert text reading "Onboarding Blocked: Dual Registration Detected for this License ID" and locks the submission button until the conflicting entries are resolved.
Postconditions	* **Successful:** A new transport association partition is legally recorded within the master ledger, creating a valid corporate parent category under which managers can begin profiling fleet assets. * **Unsuccessful:** The intake wizard is completely aborted, no new association partitions or manager profiles are built, and the admin dashboard returns a red validation exception alert.

Use Case 10: Assign Route Departure Rotation

Field	Details
Use Case ID	UC-10
Use Case Name	Assign Route Departure Rotation
Actor(s)	Associate Manager / Dispatcher
Description	Allows an Associate Manager or Terminal Dispatcher to queue an available association vehicle and its assigned driver into a specific, scheduled route departure slot following strict terminal arrival rules.
Preconditions	* The manager or dispatcher must be securely logged into their terminal operations panel. * The targeted vehicle and driver must be registered, active, and physically present in the terminal holding area. * The destination route parameters must be actively configured in the system network.
Basic Flow	1. The user navigates to the "Terminal Dispatch & Rotations" panel on their workspace interface. 2. The user selects an upcoming open scheduled travel block (e.g., Dessie to Addis Ababa at 6:30 AM). 3. The user requests to assign a vehicle to this slot, which prompts the system to invoke the **«include»** *Apply Automated FIFO Queuing Rules* system action. 4. The system scans the terminal arrival timestamp logs to automatically check which eligible association vehicle has been waiting the longest. 5. The system populates the top-ranked vehicle's License Plate, Side Number, and linked Driver details into the assignment view. 6. The user reviews the automated placement data and clicks the "Confirm Rotation Assignment" button. 7. The system locks the vehicle asset into that departure block and removes it from the open terminal waiting queue pool. 8. The system automatically publishes the trip instance online to make it searchable for travelers, updates the physical terminal departure monitors, and displays an "Assignment Successful" message.
Alternative Flows	*** Alt Flow 1: Out-of-Sequence Queue Override (Step 5)** 1. The dispatcher attempts to bypass the top FIFO vehicle to assign a different vehicle due to an emergency or mechanical breakdown detected right at the platform. 2. The system flags the deviation from the strict FIFO sequence and prompts for a mandatory "Override Reason" text input. 3. The user inputs the valid operational justification code and submits. 4. The system logs the bypass event with an administrative warning stamp and permits the manual vehicle rotation assignment. *** Alt Flow 2: Missing Vehicle Check-In Log (Step 4)** 1. The system checks the arrival logs and finds no vehicles from that specific association checked into the terminal lot. 2. The system halts the rotation workflow and displays an "Assignment Failed: No available vehicles found in terminal queue" error text.

	3. The dispatcher must wait for an asset to complete its physical checkpoint check-in before retrying.
Postconditions	* **Successful:** The vehicle and driver are officially committed to the route schedule block, creating a live trip profile that allows travelers to view seat availability and make reservations. * **Unsuccessful:** No schedule mappings are updated, the vehicle remains unassigned in the virtual waiting pool, and the interface returns a queue processing error.

Use Case 11: Manage Central Tariff Engine

Field	Details
Use Case ID	UC-11
Use Case Name	Manage Central Tariff Engine
Actor(s)	System Admin / ETA Regulator
Description	Allows top-level administrative regulators to modify global per-kilometer baseline tariff rates and vehicle level weights, dynamically recalculating transit pricing across the entire transport network.
Preconditions	* The administrator must be securely logged in with highest-level regulatory administrative credentials. * The administrator must navigate to the "Global Pricing & Tariff Configuration" module workspace.
Basic Flow	1. The System Admin clicks on the "Modify Core Tariff Pricing" action button. 2. The system pulls and displays the current active pricing rules matrix, highlighting baseline cost-per-kilometer entries and multipliers for Vehicle Levels 1, 2, and 3. 3. The admin inputs the modified monetary values into the target parameters to reflect updated Ethiopian Transport Authority (ETA) pricing regulations. 4. The admin confirms the adjustments and clicks the "Publish Tariff Adjustments" execution button. 5. The system captures the inputs and automatically invokes the **«include»** *Cascading Price Recalculation* underlying system action. 6. The background calculation engine sweeps all future scheduled trips currently registered in the database ledger. 7. The system re-evaluates the distance parameters and vehicle levels for each trip, recomputes the new final customer ticket fares, and saves them back to the live database matrix. 8. The system updates the live marketplace portals, logs the regulatory pricing change event, and displays a "Global Tariff Rules Applied and Fares Synchronized" success notification text.
Alternative Flows	* **Alt Flow 1: Out-of-Bounds Pricing Entry (Step 3)** 1. The admin inputs an invalid negative numerical value, symbols, or a rate that violates maximum authorized inflation caps set within system bounds. 2. The system tariff validator catches the value violation and halts submission processing. 3. The system highlights the invalid input field, outputs a "Tariff Out of Acceptable Regulatory Bounds" error prompt, and keeps the configuration form active for data entry correction.

Postconditions	* **Successful:** The global pricing rule parameters are updated in the database, and the system automatically rolls out recalculations for all future routes, immediately updating the booking ticket costs visible to travelers. * **Unsuccessful:** No macro configuration changes are saved, existing baseline ticket fares remain completely locked at old rates, and the admin interface displays a validation error notification.

Use Case 12: Cascading Price Recalculation

Field	Details
Use Case ID	UC-12
Use Case Name	Cascading Price Recalculation
Actor(s)	System Engine (Automated)
Description	Automatically intercepts updates to macro pricing structures, sweeps all upcoming journey entities, and updates their active final ticket fares to match newly published regulatory tariffs.
Preconditions	* The *Manage Central Tariff Engine* use case (UC-11) must complete successfully, committing new per-kilometer base rates or vehicle level weights to the system database configuration log.
Basic Flow	1. The system background processing layer intercepts the database commit trigger signaling a successful change in macro tariff rules. 2. The calculation engine initiates an asynchronous system block query, extracting all future scheduled trip rows whose departure times are greater than or equal to the current operational terminal clock time. 3. The system targets the first matching trip record in the database array loop. 4. The engine reads the trip's pre-configured distance matrix value (total kilometers between departure and arrival terminals) and the assigned vehicle's classification tier (Level 1, Level 2, or Level 3). 5. The system computes the updated traveler base ticket price by applying the formula mapped to those parameters. 6. The engine automatically overlays the required processing cuts, computing the 5% terminal operational authority fee and the remaining 95% vehicle operator share to exactly two decimal places. 7. The system overwrites the old ticket fare fields with the fresh calculated values in that specific trip row layout. 8. The system checks if secondary rows exist in the array loop and repeats steps 4 through 7 for every single upcoming trip until the entire active database partition is fully synchronized. 9. The engine commits the changes to the live marketplace views and terminates the process thread.
Alternative Flows	* **Alt Flow 1: Interrupted Transaction Rollback (Step 8)** 1. A system memory crash or network database connection timeout occurs midway through the massive cascading loop operation. 2. The background engine completely aborts the uncommitted pricing adjustments.

	3. The system executes a rollback command to revert all partially modified trip rows back to their previous verified ticket pricing states, logging a high-priority system exception warning for the administrator to prompt a recalculation retry.
Postconditions	* **Successful:** All upcoming scheduled trips nationwide are updated with perfectly accurate, legally compliant ticket fares, ensuring travelers see real-time updated checkout totals instantly. * **Unsuccessful:** No trip rows are modified, previous ticket prices remain completely unchanged across the marketplace, and a detailed system failure log is generated to safeguard transactional integrity.

Use Case 13: Search Route Availability

Field	Details
Use Case ID	UC-13
Use Case Name	Search Route Availability
Actor(s)	Traveler, Terminal Clerk
Description	Allows a traveler or a terminal counter clerk to query the active transport inventory database using specific parameters to view upcoming vehicle schedules, open seats, and dynamic ticket fares.
Preconditions	* The user must access the searching interface on either the traveler mobile application or the station terminal clerk console. * The system application must have an active connection to the central network database services.
Basic Flow	1. The user navigates to the primary trip search panel. 2. The system displays entry fields, prompting for travel criteria. 3. The user selects or inputs: Source Terminal (Origin), Destination Terminal, preferred Travel Date, and desired Vehicle Quality Classification (Level 1, Level 2, or Level 3). 4. The user clicks the "Search Trips" execution button. 5. The system captures the parameters and queries the master operational database partition. 6. The system isolates all active trip rows that match the user's route locations, departure dates, and vehicle specifications. 7. The system dynamically reads the live open inventory tallies for each matching trip instance. 8. The system renders a chronological schedule results list displaying Departure Time, Vehicle License Plate and Level, Ticket Fare, and Total Available Seats left.
Alternative Flows	* **Alt Flow 1: No Available Trips Found (Step 6)** 1. The system scans the database and finds zero active trip rows scheduled for that specific origin-destination pair or chosen date. 2. The system stops processing the inventory layout grid. 3. The system displays a "No Trips Scheduled for this Date" notice and suggests alternative schedules for adjacent dates or a different vehicle level category. * **Alt Flow 2: Incomplete Parameters (Step 4)** 1. The user triggers the search execution button while leaving the Source Terminal or Destination Terminal fields blank.

	2. The system pauses query execution traffic. 3. The system displays an inline "Please select both an origin and destination terminal" warning alert to enforce standard validation.
Postconditions	* **Successful:** The user is provided an accurate, real-time filtered lookup view of available vehicle schedules and open seating inventory, opening the path to select a trip block and begin a booking transaction. * **Unsuccessful:** No database search is processed, the system displays clear validation parameters or empty state prompts, and the user interface remains on the search entry panel.

Use Case 14: Book Ticket

Field	Details
Use Case ID	UC-14
Use Case Name	Book Ticket
Actor(s)	Traveler
Description	Allows an authenticated traveler to select a specific trip schedule, pick a seat coordinate from an interactive vehicle layout map, and initiate a checkout transaction.
Preconditions	* The traveler must be securely logged into their application session. * The traveler must have executed a search query and selected a specific, un-departed trip schedule block.
Basic Flow	1. The Traveler selects a target trip instance from the route search availability results view. 2. The system captures the selection and invokes the **«include»** *Select Single Seat* (UC-15) system action. 3. The system renders the interactive vehicle seating matrix graphic on screen, showing open vs. occupied seats mapped according to the vehicle's capacity grade. 4. The traveler selects a single open seat slot, which automatically triggers the **«include»** *Activate Transactional Seat Locking (10 min)* (UC-16) background action. 5. The system locks the chosen seat in the repository and starts a 10-minute payment countdown clock overlay on the user interface. 6. The traveler reviews the final calculated fare breakdown and clicks the "Proceed to Checkout" action button. 7. The system launches the **«include»** *Process Digital Payment* (UC-18) use case workflow. 8. Upon receiving a success confirmation from the payment gateway, the system updates the permanent seat status to "Reserved", links the ticket row to the traveler's ID, and flashes a "Ticket Booked Successfully" confirmation with a scannable code stub.
Alternative Flows	* **Alt Flow 1: Transactional Lock Timeout (Step 5)** 1. The traveler delays completing checkout, and the 10-minute session countdown timer drops to zero. 2. The background engine terminates the checkout session block. 3. The system clears the temporary lock, restores the seat state to "Available" in the public directory, and redirects the traveler back to the schedule page with a "Booking Window Expired" message.

	* **Alt Flow 2: Multi-Seat Form Violation Check (Step 4)** 1. The traveler attempts to click and select multiple seat icons simultaneously on the layout grid. 2. The selection validation filter intercepts the input and refuses multi-slot allocation. 3. The system updates the layout to hold only the most recently selected seat coordinate, keeping strict alignment with the single-ticket operational form constraint.
Postconditions	* **Successful:** A single seat coordinate is subtracted from public inventory, a unique ticket ledger instance is written under the traveler's account, and a digital ticket verification stub is generated. * **Unsuccessful:** No database rows are modified, any temporary coordinate locks are completely rolled back, and the ticket inventory remains available to the public marketplace.

Use Case 15: Select Seat

Field	Details
Use Case ID	UC-15
Use Case Name	Select Seat
Actor(s)	System Engine (Automated)
Description	Automatically retrieves the vehicle configuration model linked to a trip, reads the active seat availability map, and renders an interactive single-choice grid layout for the interface user.
Preconditions	* This use case must be explicitly invoked and included (**«include»**) by either *Book Single Ticket* (UC-14) or *Process Walk-In Booking* (UC-17). * A valid scheduled trip instance must be actively passed down as a parameter.
Basic Flow	1. The selection engine intercepts the incoming trip identifier passed from the parent booking workflow. 2. The system queries the vehicle fleet record database to determine the specific vehicle classification level (Level 1, Level 2, or Level 3) assigned to that trip. 3. The system pulls the corresponding standardized blueprint matrix template row (e.g., a 45-seat map grid for Level 1, or a 60-seat map grid for Level 2/3). 4. The system sweeps the live real-time booking ledger to cross-reference which seat coordinates on that specific vehicle are already flagged as "Reserved", "Checked-In", or temporarily "Locked". 5. The system dynamically renders the structural seating layout on the user's interface screen, visibly distinguishing available grid slots from occupied/disabled blocks using distinct color states. 6. The system tracks interactive screen inputs, allowing the client to click a single open coordinate cell. 7. The system updates the screen UI to highlight the chosen seat index and passes the selected coordinate index back to the invoking parent booking flow.
Alternative Flows	* **Alt Flow 1: Stale Inventory Detection (Step 6)** 1. The user clicks a seat coordinate that appears open on their screen, but another traveler completed a booking on that exact seat milliseconds prior before the client interface could refresh. 2. The system database layer catches the state collision.

	3. The system flashes a "Seat No Longer Available" inline warning badge, pulls the latest row matrix data, and dynamically refreshes the interactive layout view for a new selection.
Postconditions	* **Successful:** A single valid seat coordinate index is captured and returned to the parent routine, paving the way for the reservation engine to initiate secure locking procedures. * **Unsuccessful:** No seat index parameter is returned, the layout fails to render or throws a matrix mapping exception, and the parent booking flow is paused.

Use Case 16: Activate Transactional Seat Locking (10 min)

Field	Details
Use Case ID	UC-16
Use Case Name	Activate Transactional Seat Locking (10 min)
Actor(s)	System Engine (Automated)
Description	Temporarily isolates a selected seat coordinate under a strict 10-minute countdown clock to prevent other system transactions from reserving the same seat while checkout processing takes place.
Preconditions	* This system action must be automatically invoked and included (**«include»**) by a live checkout sequence in *Book Single Ticket* (UC-14) or *Process Walk-In Booking* (UC-17). * A specific seat coordinate index must be chosen and passed to the engine.
Basic Flow	1. The background transactional engine intercepts the seat selection parameter along with the corresponding Trip ID. 2. The system initiates an atomic database write lock operation on that specific seat index row. 3. The system switches the seat state field from "Available" to "Locked" and appends an active system timestamp token. 4. The system launches an asynchronous 10-minute (600-second) countdown timer thread linked to the active session block. 5. The system pushes the state change to the public cache layer, causing the targeted seat index to show up as grayed-out and unavailable on all other matching traveler search layouts network-wide. 6. The engine keeps monitoring transaction progress flags, waiting for an explicit "Payment Success" or "Booking Complete" status update signal from the parent process flow before the clock runs out.
Alternative Flows	* **Alt Flow 1: Expiration of the 10-Minute Checkout Window (Step 6)** 1. The 10-minute countdown timer runs down to zero without receiving a verified payment or manual completion flag. 2. The background engine interrupts the active reservation session block. 3. The system drops the write lock, changes the seat status field back to "Available", and deletes the temporary session token. 4. The system refreshes the public database index, allowing the seat coordinate to become searchable and select-ready for other clients. * **Alt Flow 2: Manual Booking Session Abort (Step 6)** 1. The traveler clicks the "Cancel Booking" or "Back" navigation buttons on their checkout interface screen before paying.

	2. The client application dispatches an explicit session teardown request to the server layer. 3. The background engine skips the remaining countdown time, unlocks the seat index row immediately, and sets its state back to "Available".
Postconditions	* **Successful:** The selected seat is securely isolated and blocked from double-booking conflicts for up to 10 minutes, providing a protected window to process payments safely. * **Unsuccessful:** The database lock fails to engage due to high-concurrency race conditions, the seat remains open to the public marketplace, and the parent checkout flow is safely aborted.

Use Case 17: Process Walk-In Booking

Field	Details
Use Case ID	UC-17
Use Case Name	Process Walk-In Booking
Actor(s)	Terminal Clerk
Description	Enables a station terminal counter clerk to manually reserve a trip seat, collect physical cash currency, and issue a print-ready verification ticket stub on behalf of an offline walk-in passenger.
Preconditions	* The Terminal Clerk must be securely authenticated and active on a designated station counter workstation console. * The target journey must have an active, published status state and open seating inventory before departure.
Basic Flow	1. An offline passenger approaches the physical ticket counter and requests a journey, specifying their target destination terminal, travel day, and preferred vehicle tier. 2. The Terminal Clerk inputs these metrics into the station console to invoke the **«include»** *Search Route Availability* (UC-13) operational flow. 3. The clerk reviews open trip matches on screen, selects the passenger's choice, and triggers the booking workspace, causing the system to pull the **«include»** *Select Single Seat* (UC-15) action routine. 4. The clerk highlights an available seat index block directly from the interactive matrix layout view. 5. The system locks that selected coordinate index row instantly by calling the **«include»** *Activate Transactional Seat Locking (10 min)* (UC-16) system script. 6. The clerk types the passenger's Full Name and unique Phone Number into the customer registration fields. 7. The clerk collects the exact cash payment amount corresponding to the calculated tariff cost and presses the "Complete Cash Transaction" action button. 8. The system bypasses external electronic payment rails, permanently registers the seat to the customer's phone ledger, and generates a print-ready digital ticket stub layout featuring an authorized tracking code.
Alternative Flows	*** Alt Flow 1: Passenger Refusal / Missing Cash (Step 7)** 1. The walk-in passenger rejects the ticket price or lacks the physical cash currency required to finish the transaction.

	2. The Terminal Clerk presses an "Abort Checkout" function key on their terminal screen. 3. The system drops the active countdown session, resets the seat status back to "Available", and clears the customer intake fields.
Postconditions	* **Successful:** A single seat allocation is deducted from the master vehicle manifest grid, the transaction is logged as an internal station cash entry, and a physical ticket stub is ready for printing. * **Unsuccessful:** No seat mappings are modified, the cash ledger remains unaltered, and the station counter application resets back to an open, uncommitted data entry state.

Use Case 18: Process Digital Payment

Field	Details
Use Case ID	UC-18
Use Case Name	Process Digital Payment
Actor(s)	System Engine (Automated), External Payment Gateway (Telebirr / CBE Birr)
Description	Handles the secure handshaking, routing, and processing of electronic monetary transactions via local mobile money API gateways to finalize ticket bookings.
Preconditions	* This use case must be implicitly invoked and included (**«include»**) by a live transaction in *Book Single Ticket* (UC-14). * A temporary transactional lock (UC-16) must be active on the chosen seat asset. * The traveler must have selected a mobile payment method option.
Basic Flow	1. The payment engine captures the dynamic ticket fare cost, traveler phone identifier, and unique transaction reference tracking ID from the parent booking layer. 2. The system establishes a secure HTTPS connection and dispatches an API request block to the chosen payment provider gateway (Telebirr or CBE Birr). 3. The system generates a specialized checkout deep-link redirect or push notification challenge directly to the traveler's mobile device application. 4. The traveler authorizes the payment by entering their personal security PIN inside the dedicated mobile payment interface layer. 5. The External Payment Gateway handles the account verification, deducts the specified monetary amount, and generates an encrypted payment status response token. 6. The system engine receives the asynchronous webhook response callback notification from the external gateway. 7. The system decrypts the payment response status code and confirms that the funds match the exact target ticket cost payload. 8. The system updates the internal transaction state database row from "Pending" to "Paid", registers the provider transaction hash index, and yields a confirmation signal back to the parent booking routine.
Alternative Flows	* **Alt Flow 1: External Gateway Transaction Rejection (Step 6)** 1. The External Payment Gateway returns a flat rejection callback code (e.g., due to insufficient customer wallet funds, bad authentication PIN, or transaction timeout). 2. The system engine records the payment failure log trace in the ledger files.

	3. The system raises an "Electronic Payment Failed: Please Check Balance or Retry" error prompt on the user dashboard interface screen, maintaining the temporary seat reservation lock for any remaining minutes. * **Alt Flow 2: Network Link Handshake Breakdown (Step 2)** 1. The local carrier network experiences an interruption, causing the system API request to timeout before reaching the gateway rails. 2. The system triggers an automatic internal 3-pass retry loop mechanism over a 15-second block. 3. If connection failures remain, the engine terminates the payment thread and notifies the parent booking flow to execute a safe transactional rollback.
Postconditions	* **Successful:** Financial funds are securely captured and logged, the system transaction state updates permanently to a completed settlement status, and the ticket confirmation flow is approved. * **Unsuccessful:** No money is captured from the traveler's electronic wallet balance, the system transaction log row marks the attempt as failed, and the booking app returns a detailed payment error explanation badge.

Use Case 19: Check-In & Boarding Verification

Field	Details
Use Case ID	UC-19
Use Case Name	Check-In & Boarding Verification
Actor(s)	Terminal Clerk / Conductor
Description	Allows a station terminal clerk or vehicle conductor to scan or manually verify a traveler's digital or physical ticket stub identifier, cross-referencing the master vehicle manifest grid to authorize passenger boarding.
Preconditions	* The Terminal Clerk or Conductor must be securely logged into their mobile verification terminal or station console application. * The target transit vehicle must be physically stationed at its designated terminal departure bay.
Basic Flow	1. The traveler arrives at the vehicle boarding gate and presents their digital ticket verification stub screen or printed ticket slip showing the encrypted tracking code. 2. The Terminal Clerk activates the camera hardware peripheral scanner on their operational interface tool. 3. The clerk scans the passenger's scannable verification code stub layout. 4. The system decodes the tracking string token and automatically queries the real-time reservation ledger rows. 5. The system pulls up the matching ticket entity, checking that the trip status is active, the departure date matches today, and the payment status equals "Paid". 6. The system updates the individual traveler's entry status from "Reserved" to "Checked-In" in the vehicle manifest table layout. 7. The application returns a green "Access Approved: Clear to Board" success layout block on the clerk's device interface, detailing the exact seat coordinate index assigned to that passenger.
Alternative Flows	* **Alt Flow 1: Manual Tracking Code Validation Override (Step 3)**

	1. The traveler's smartphone display glass is broken, or a paper ticket layout is torn, preventing the optical scanner tool from reading the code asset. 2. The clerk bypasses the scanner function and manually types the alpha-numeric verification serial string directly into an on-screen text input row. 3. The system processes the typed string parameter and resumes the verification stream from step 4 forward. * **Alt Flow 2: Ticket Validation Integrity Breach (Step 5)** 1. The system determines that the ticket tracking code does not exist in the master database, belongs to an entirely different departure day, or is already flagged as "Checked-In". 2. The verification engine blocks the boarding clearance authorization tracker. 3. The device interface flashes a high-visibility red "Invalid Ticket / Entry Denied" security warning alert text, prompting the clerk to turn away the passenger.
Postconditions	* **Successful:** The passenger's status is permanently updated to checked-in inside the digital dispatch manifest array, confirming their physical presence on the vehicle for the transport association logs. * **Unsuccessful:** The ticket status remains unverified, boarding is strictly denied, and the central manifest records undergo no data state modifications.

Use Case 20: Generate Operational Reports

Field	Details
Use Case ID	UC-20
Use Case Name	Generate Operational Reports
Actor(s)	System Admin / ETA Regulator, Associate Manager
Description	Allows administrators, transport authority regulators, or association managers to extract structured visual summaries and data sheets detailing passenger volumes, revenue generation cuts, and fleet utilization metrics over specified date blocks.
Preconditions	* The user must be authenticated with appropriate managerial or regulatory system role privileges. * The user must navigate to the "Analytics, Audits, & Reports" workspace interface.
Basic Flow	1. The user clicks on the "Generate Reports" functional workspace option within their system dashboard. 2. The system loads the reporting configurations view, displaying parameter filters including Date Range, Target Association, Route Path, and Report Type (e.g., Financial Revenue Audits, FIFO Queue Efficiency, or Passenger Load Factors). 3. The user specifies their desired reporting scope parameters and selects the required file output layout configuration (e.g., PDF or Excel Sheet data grid). 4. The user clicks on the "Compile Report" execution button. 5. The reporting engine pulls data metrics across transactional logs, seating manifests, and vehicle rotation history tables matching the filters. 6. The system calculates macroeconomic performance summaries, isolating the standard 5% terminal regulatory share and the remaining 95% association payout slices rounded to exactly two decimal places. 7. The system formats the aggregated data blocks into a structured, human-readable analytical file sheet layout.

	8. The application displays an on-screen preview of the compiled data metrics and renders an active link label button to download the finalized document structure.
Alternative Flows	* **Alt Flow 1: Zero Record Analytics Result (Step 5)** 1. The system sweeps the database ledger and identifies no transactional tickets, fleet deployments, or queue events matching the user's specific filter combination or date range window. 2. The reporting engine stops formatting the analytical document matrix. 3. The system returns an inline "Report Compilation Cancelled: No operational logs exist for selected parameters" informational notification text, prompting the user to broaden their search criteria.
Postconditions	* **Successful:** A read-only performance report overview document is compiled and delivered to the user interface client, providing verifiable business intelligence without modifying any baseline repository values. * **Unsuccessful:** The system fails to pull database entries due to structural timeouts, no analytical layout is outputted, and the user interface screen returns an explicit report rendering exception alert.

Use Case 21: Process Ticket Cancellation & Refund

Field	Details
Use Case ID	UC-21
Use Case Name	Process Ticket Cancellation & Refund
Actor(s)	Traveler / Terminal Clerk
Description	Allows a traveler via the mobile app (or a terminal counter clerk on behalf of an offline walk-in passenger) to cancel a valid, paid ticket reservation before the departure cutoff threshold, releasing the seat inventory and issuing a partial automated refund.
Preconditions	* The traveler must be securely logged into their application session, or the clerk must be authenticated on an active station console. * The targeted ticket entry must have an active status of "Reserved" with a transaction state of "Paid". * The system clock must indicate that the departure time for the linked trip block is still outside the minimum cancellation cutoff window (e.g., more than 2 hours before departure).
Basic Flow	1. The user navigates to their active reservations dashboard and selects the specific ticket record they wish to cancel. 2. The system fetches the reservation row parameters and verifies that the current terminal time complies with the cancellation window rules. 3. The system calculates the cancellation penalty deductions and presents a breakdown showing the Original Fare, Policy Penalty Fee (e.g., 10%), and the final Net Refund Amount. 4. The user clicks the "Confirm Cancellation & Request Refund" execution button. 5. The system changes the ticket row status in the master directory from "Reserved" to "Cancelled". 6. The system background layer immediately releases the linked seat coordinate index on that vehicle manifest grid, updating its public state back to "Available".

	7. The system dispatches an asynchronous refund transaction command payload to the external payment gateway provider ledger (Telebirr or CBE Birr) matching the original transaction hash. 8. The system displays a "Ticket Cancelled Successfully: Refund Processed" confirmation message on the screen and sends a text summary to the passenger's registered phone number.
Alternative Flows	*** Alt Flow 1: Cutoff Window Violation Lockout (Step 2)** 1. The system checks the trip schedule and determines the vehicle is scheduled to depart within less than the minimum cutoff threshold (e.g., 1 hour until departure). 2. The cancellation engine intercepts the request and blocks the execution process. 3. The system displays a "Cancellation Denied: Ticket is locked because the vehicle is preparing for departure" error text and deactivates the cancellation action buttons. *** Alt Flow 2: Offline Cash Refund Processing (Step 7)** 1. The system identifies that the original reservation was processed as an offline station transaction (*UC-17: Process Walk-In Booking*) paid via physical cash currency. 2. The system bypasses electronic digital payment gateway API routing channels. 3. The system records an internal station cash payout balancing line entry, changes the ticket status to "Cancelled", and prompts the Terminal Clerk: "Please disburse Net Refund Amount in cash to the walk-in passenger from the station drawer."
Postconditions	*** Successful:** The ticket row is marked as cancelled, the seat coordinate index is instantly restored to the open public inventory, and a financial refund log trace is committed to the ledger directory. *** Unsuccessful:** No changes are written to the database, the passenger's seat reservation remains completely secure, and the interface yields a transactional exception warning flag.

Use Case 22: Handle Vehicle Breakdown & Fleet Substitution

Field	Details
Use Case ID	UC-22
Use Case Name	Handle Vehicle Breakdown & Fleet Substitution
Actor(s)	Associate Manager / Dispatcher
Description	Enables an Associate Manager or Terminal Dispatcher to quickly swap a broken-down or malfunctioning vehicle with an idle standby vehicle from the same association, automatically remapping all existing passenger seat reservations to the new vehicle manifest.
Preconditions	* The manager or dispatcher must be securely logged into their terminal operations dashboard. * An active scheduled trip block must have a status of "Published" or "Boarding" with existing passenger ticket reservations. * The replacement standby vehicle must be registered, active, physically present in the terminal holding area, and have the same or higher capacity classification tier.
Basic Flow	1. The Associate Manager navigates to the "Active Dispatched Journeys" dashboard panel and selects the specific trip block affected by a mechanical failure. 2. The manager clicks on the "Report Fleet Breakdown / Substitute Vehicle" emergency action button.

	3. The system pulls a filtered list displaying all idle standby vehicle profiles registered under that specific manager's transport association that match the passenger capacity profile. 4. The manager selects an available replacement vehicle from the list and clicks "Execute Asset Substitution". 5. The system calculation engine automatically matches the structural layout of the old vehicle grid to the new replacement vehicle grid, mapping every "Reserved" and "Checked-In" ticket row to identical seat coordinate indexes on the new asset. 6. The system updates the scheduled trip row, replacing the broken vehicle's License Plate and Side Number parameters with the replacement vehicle's technical identifiers. 7. The system sends an automated high-priority system signal to the central terminal gate monitors and dispatches updated SMS push notifications to all booked passengers' phone numbers. 8. The interface flashes an emergency "Fleet Substitution Completed: Manifest Synchronized" success notification banner.
Alternative Flows	* **Alt Flow 1: Capacity Downgrade Constraint Violation (Step 5)** 1. The system detects that the chosen replacement vehicle has a lower seating capacity grid than the original broken vehicle, meaning some passengers with paid tickets would be left without a seat coordinate index. 2. The substitution engine halts the database mapping thread. 3. The system displays a "Substitution Blocked: Standby vehicle capacity is insufficient to accommodate existing manifest" error prompt and forces the manager to select a compliant asset. * **Alt Flow 2: Association Asset Border Cross-Over (Step 3)** 1. The system identifies that the manager's own transport association has zero idle standby vehicles available at the terminal platform. 2. The system prompts the manager to request a "Cross-Association Emergency Lease". 3. Upon selecting a temporary vehicle asset from a partner association, the system logs an inter-corporate financial lease tracking flag and permits the substitution workflow to proceed to step 5.
Postconditions	* **Successful:** The broken vehicle is flagged out of service, the replacement vehicle is locked into the active route schedule block, and all existing passenger reservations are seamlessly transferred to the new manifest without transactional ticket losses. * **Unsuccessful:** No changes are written to the active journey database row, the passenger manifest remains bound to the broken asset, and the manager interface returns a critical operation failure alert.

Use Case 23: Manage Terminal Stations

Field	Details
Use Case ID	UC-24
Use Case Name	Manage Terminal Stations
Actor(s)	System Admin / ETA Regulator
Description	Allows administrative regulators to create, update, or disable physical terminal location nodes (e.g., Addis Ababa Lamberet Terminal, Dessie Terminal, Woldia Terminal) within the master database system network.
Preconditions	* The user must be authenticated with top-level System Administrator or ETA Regulator system role privileges. * The user must navigate to the "Infrastructure & Node Management" configuration panel.
Basic Flow	1. The Admin clicks on the "Manage Terminal Stations" action tab within the infrastructure panel. 2. The system fetches and renders a complete data grid list of all existing physical station records showing Terminal ID, Name, Regional Location, and Status (Active/Inactive). 3. The Admin clicks the "Add New Terminal" action button. 4. The system loads a clean data entry form prompting for parameters: Official Terminal Name, Geographic Coordinates, Total Vehicle Bay Capacity, and Responsible Regional Authority Branch. 5. The Admin inputs the required terminal profile details and clicks the "Save Station Node" execution button. 6. The system runs schema validation to ensure the terminal name does not create a duplication conflict in the directory. 7. The database layer writes a new terminal infrastructure row into the master location partition. 8. The interface updates the live data grid layout and flashes a "Terminal Station Node Registered Successfully" confirmation badge.
Alternative Flows	* **Alt Flow 1: Node Deactivation Integrity Check (Step 2)** 1. The Admin attempts to change the operational status of an existing terminal node from "Active" to "Inactive". 2. The infrastructure engine sweeps the live rotation tables and identifies that this terminal is currently listed as an active source or destination for un-departed trip blocks. 3. The system halts the status modification script. 4. The system returns a "Deactivation Blocked: Terminal is currently bound to active route schedules" error text, forcing the admin to reroute those schedules first.
Postconditions	* **Successful:** A new physical terminal station entity is securely committed to the system network directory, making it instantly available for route layout mapping. * **Unsuccessful:** No database modifications take place, the infrastructure list remains unaltered, and the workspace interface returns a data entry validation exception warning.

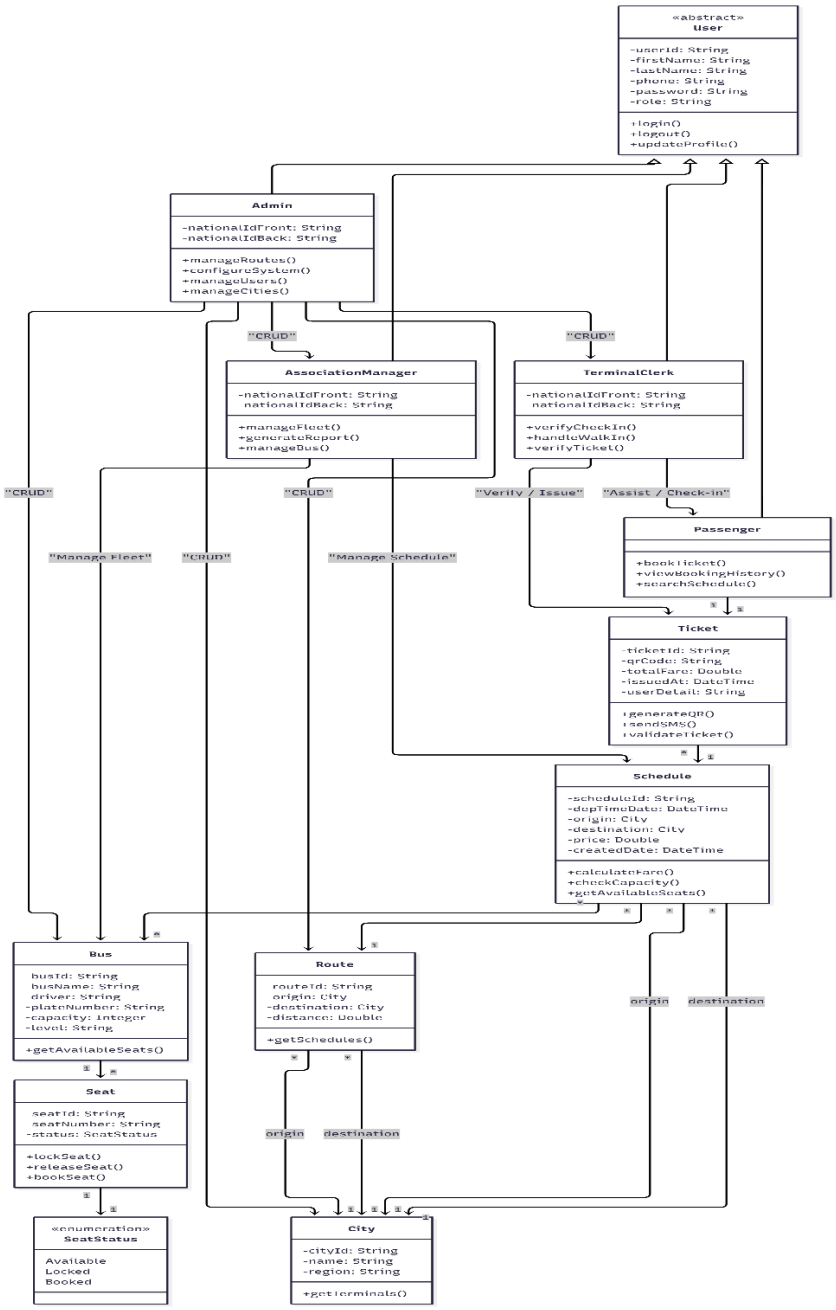
Use Case 24: Dispatch Automated Trip Notifications

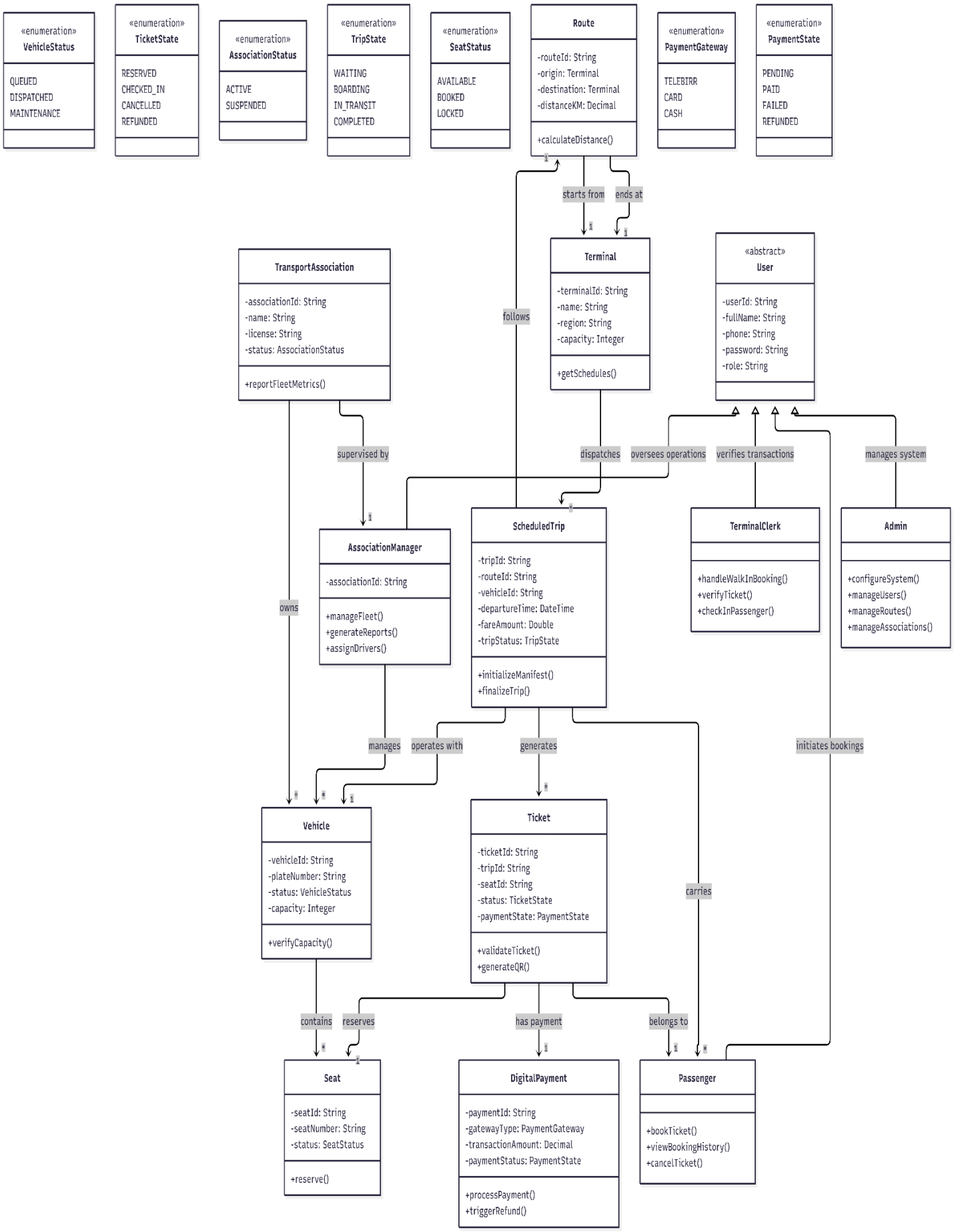
Field	Details
Use Case ID	UC-26
Use Case Name	Dispatch Automated Trip Notifications
Actor(s)	System Engine (Automated)
Description	Automatically triggers background SMS alerts, push notifications, or system terminal dashboard flashes to travelers, drivers, and clerks containing departure reminders, schedule updates, or high-priority transit alterations.
Preconditions	* This use case is a system-triggered daemon action invoked automatically by state changes in parent workflows (such as *UC-14: Book Single Ticket*, *UC-22: Handle Vehicle Breakdown*, or *UC-23: Cancel Scheduled Trip Block*). * The central notification gateway subsystem must be operational and linked with local telecom SMS routing configurations.
Basic Flow	1. The central messaging daemon monitors the system event bus for targeted transactional state changes or time-based triggers (e.g., exactly 2 hours before a scheduled trip departure). 2. The engine catches an event trigger and queries the relational passenger manifest and vehicle driver assignment rows linked to that targeted Trip ID. 3. The system compiles a target recipient array containing unique phone number string indicators and client application push session tokens. 4. The system reads the corresponding parameterized template string matching the event type (e.g., "Dear Passenger, your bus [Plate Number] departs from Bay [X] at [Time]. Please arrive for check-in."). 5. The system engine constructs the final localized data payload block, filling in the custom variables dynamically. 6. The system dispatches the formatted payload blocks to the integrated external SMS aggregator gateway or internal mobile push notification broadcast relay. 7. The system appends a read-only historical trace timestamp log to the message repository, linking it directly to the customer's account profile registry for auditing purposes.
Alternative Flows	* **Alt Flow 1: Client Device Push Token Invalidity (Step 6)** 1. The push relay returns a delivery error code indicating that a traveler's application session token has expired or the app was uninstalled. 2. The notification subsystem automatically intercepts the failure track. 3. The engine falls back to the secondary notification channel, formatting the text alert layout into a standard SMS carrier payload and dispatching it directly to the traveler's registered mobile number string to ensure delivery integrity.
Postconditions	* **Successful:** Customized operational alert records are compiled, successfully pushed down to carrier rails or device channels, and a delivery tracking block is safely written into the internal system log files. * **Unsuccessful:** The message execution thread throws an API connection exception, no external alerts are dispatched across the network, and a high-visibility message failure flag is written to the system administrator's error log dashboard.

3.5 Object Model

3.5.1 Class Diagram

The class diagram for the Transport Scheduling and Ticket Booking System defines the structure of the system by modeling its classes, their attributes, methods, and the relationships between them. It shows the core entities such as User, Passenger, Bus, Route, Schedule and Ticket.




«enumeration»
VehicleStatus
QUEUED
DISPATCHED
MAINTENANCE
«enumeration»
TicketState
RESERVED
CHECKED_IN
CANCELLED
REFUNDED
«enumeration»
AssociationStatus
ACTIVE
SUSPENDED
«enumeration»
TripState
WAITING
BOARDING
IN_TRANSIT
COMPLETED
«enumeration»
SeatStatus
AVAILABLE
BOOKED
LOCKED
Route
-routeId: String
-origin: Terminal
-destination: Terminal
-distanceKM: Decimal
+calculateDistance()
«enumeration»
PaymentGateway
TELEBIRR
CARD
CASH
«enumeration»
PaymentState
PENDING
PAID
FAILED
REFUNDED
starts from
ends at
Terminal
-terminalId: String
-name: String
-region: String
-capacity: Integer
+getSchedules()
«abstract»
User
-userId: String
-fullName: String
-phone: String
-password: String
-role: String
TransportAssociation
-associationId: String
-name: String
-license: String
-status: AssociationStatus
+reportFleetMetrics()
follows
supervised by
dispatches
oversees operations
verifies transactions
manages system
AssociationManager
-associationId: String
+manageFleet()
+generateReports()
+assignDrivers()
ScheduledTrip
-tripId: String
-routeId: String
-vehicleId: String
-departureTime: DateTime
-fareAmount: Double
-tripStatus: TripState
+initializeManifest()
+finalizeTrip()
TerminalClerk
+handleWalkInBooking()
+verifyTicket()
+checkInPassenger()
Admin
+configureSystem()
+manageUsers()
+manageRoutes()
+manageAssociations()
owns
manages
operates with
generates
initiates bookings
Vehicle
-vehicleId: String
-plateNumber: String
-status: VehicleStatus
-capacity: Integer
+verifyCapacity()
Ticket
-ticketId: String
-tripId: String
-seatId: String
-status: TicketState
-paymentState: PaymentState
+validateTicket()
+generateQR()
carries
contains
reserves
has payment
belongs to
Seat
-seatId: String
-seatNumber: String
-status: SeatStatus
+reserve()
DigitalPayment
-paymentId: String
-gatewayType: PaymentGateway
-transactionAmount: Decimal
-paymentStatus: PaymentState
+processPayment()
+triggerRefund()
Passenger
+bookTicket()
+viewBookingHistory()
+cancelTicket()

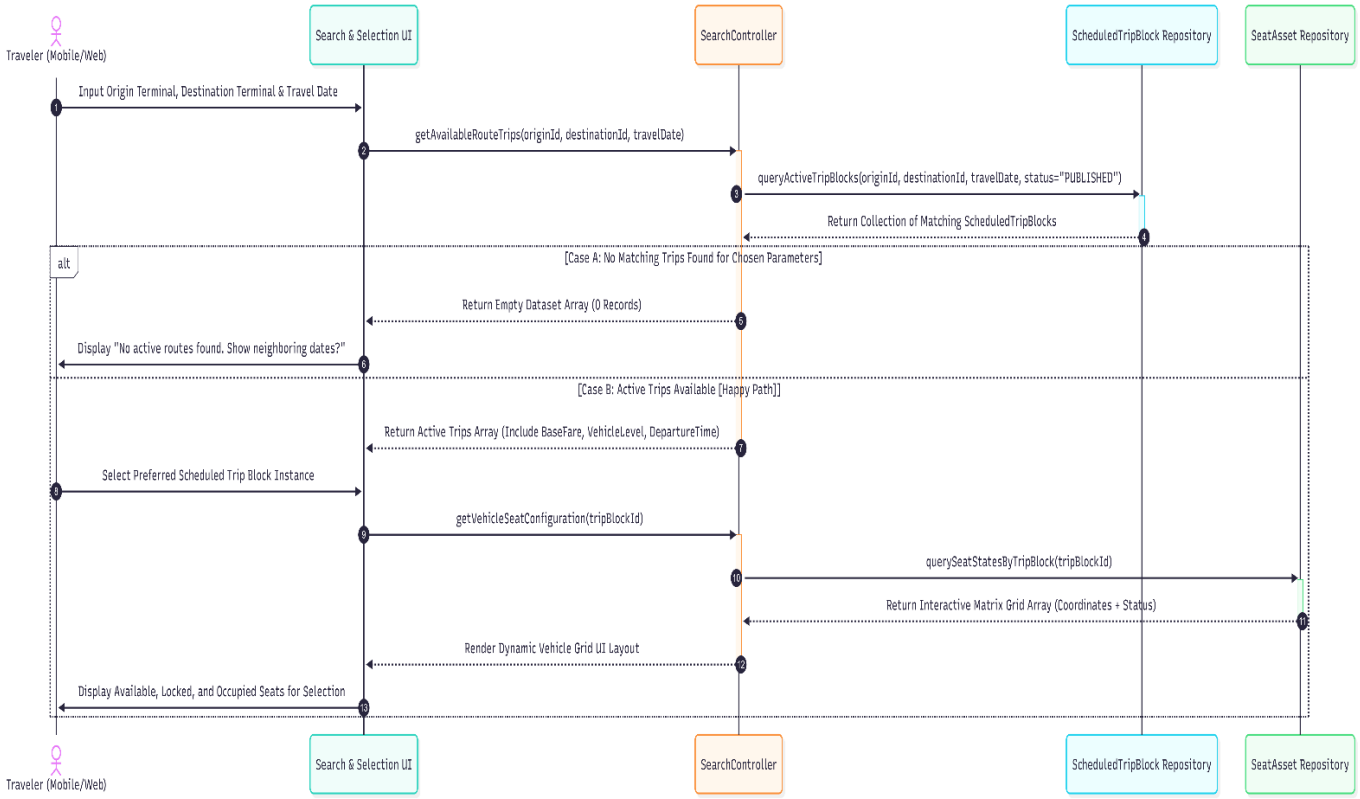
3.6 Dynamic Models

3.6.1 Sequence Diagram

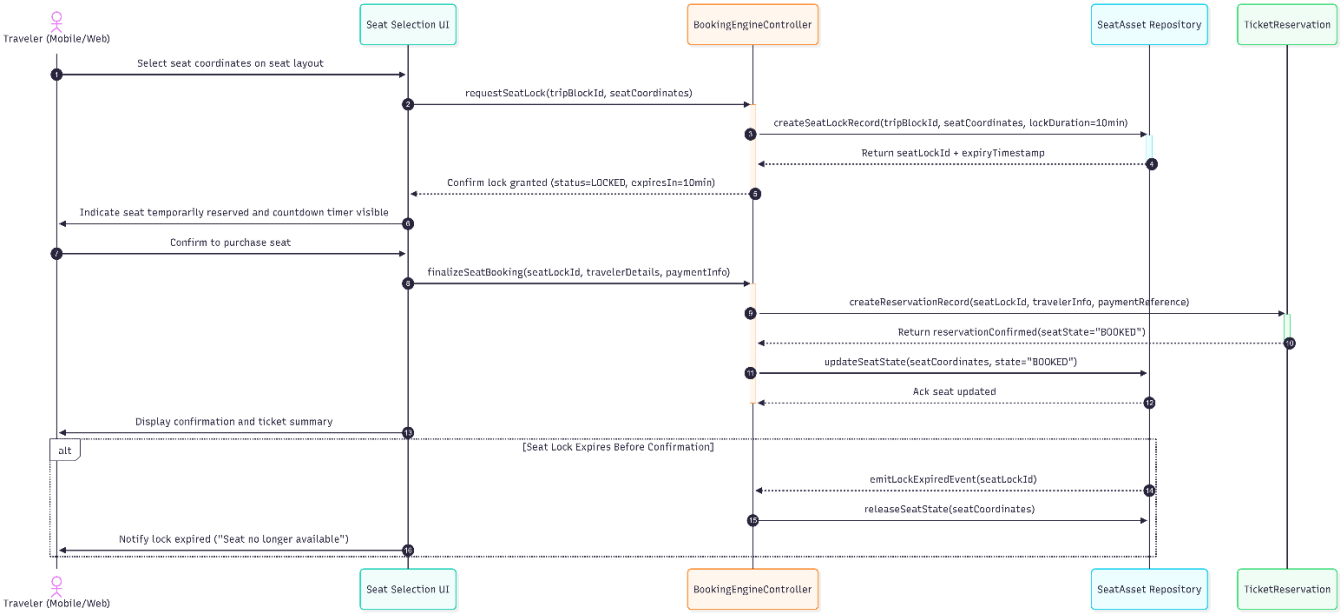
Category 1: Core Passenger & Transactional Diagrams

These trace the high-concurrency "Happy Paths" and "Exception Paths" driving your application's booking, locking, and external system connectivity.

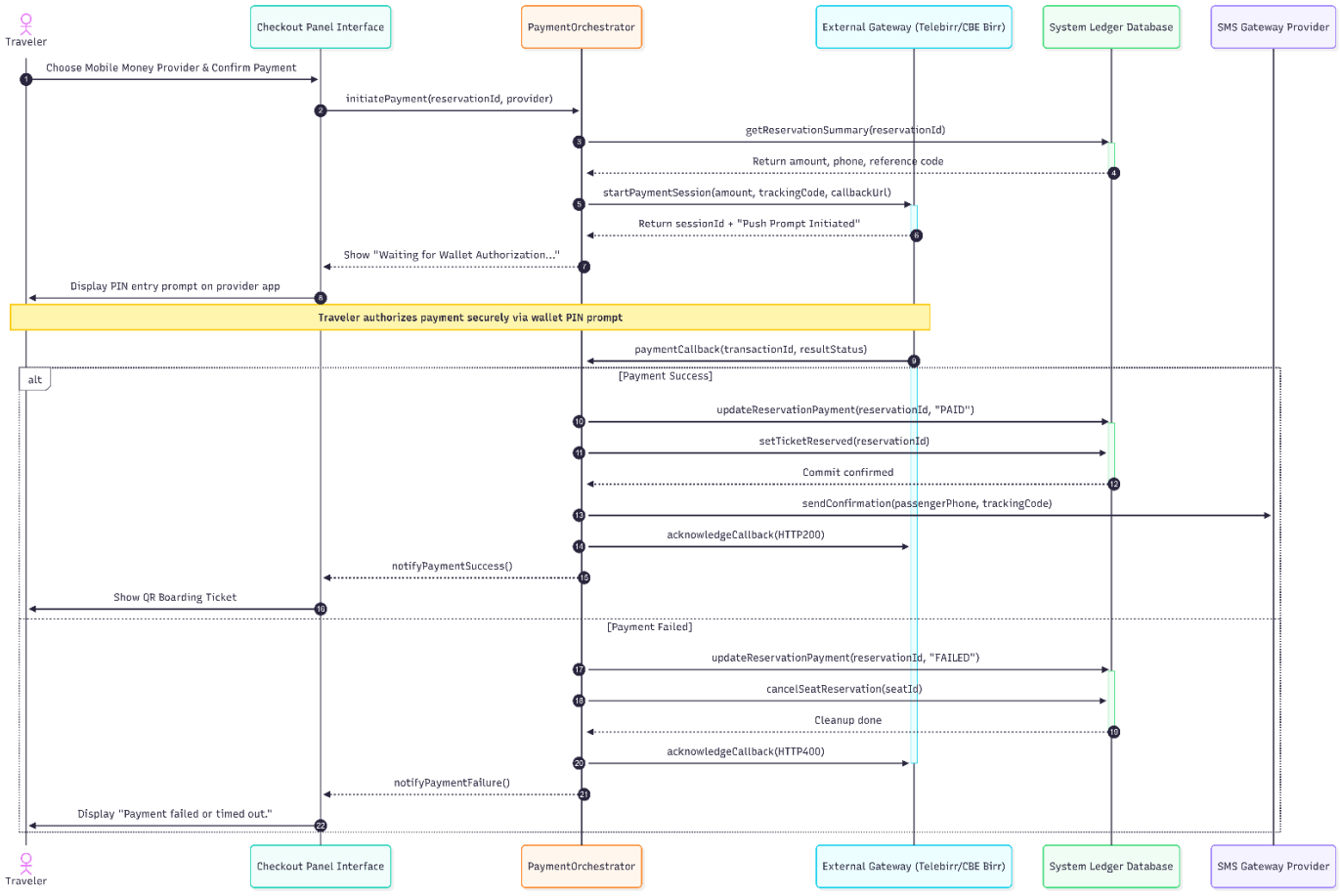
1. **SD-01: Search and Book Ticket (Primary Flow)**



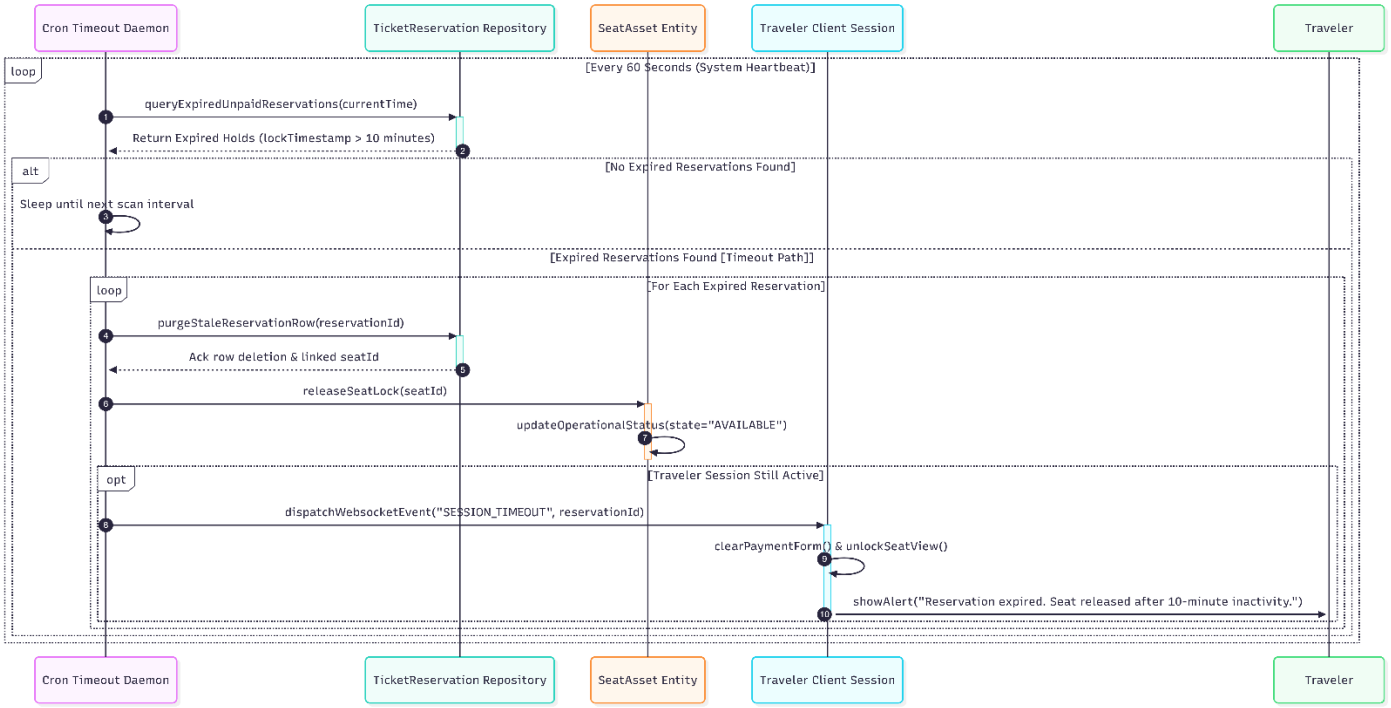
2. SD-02: Seat Reservation and 10-Minute Lock



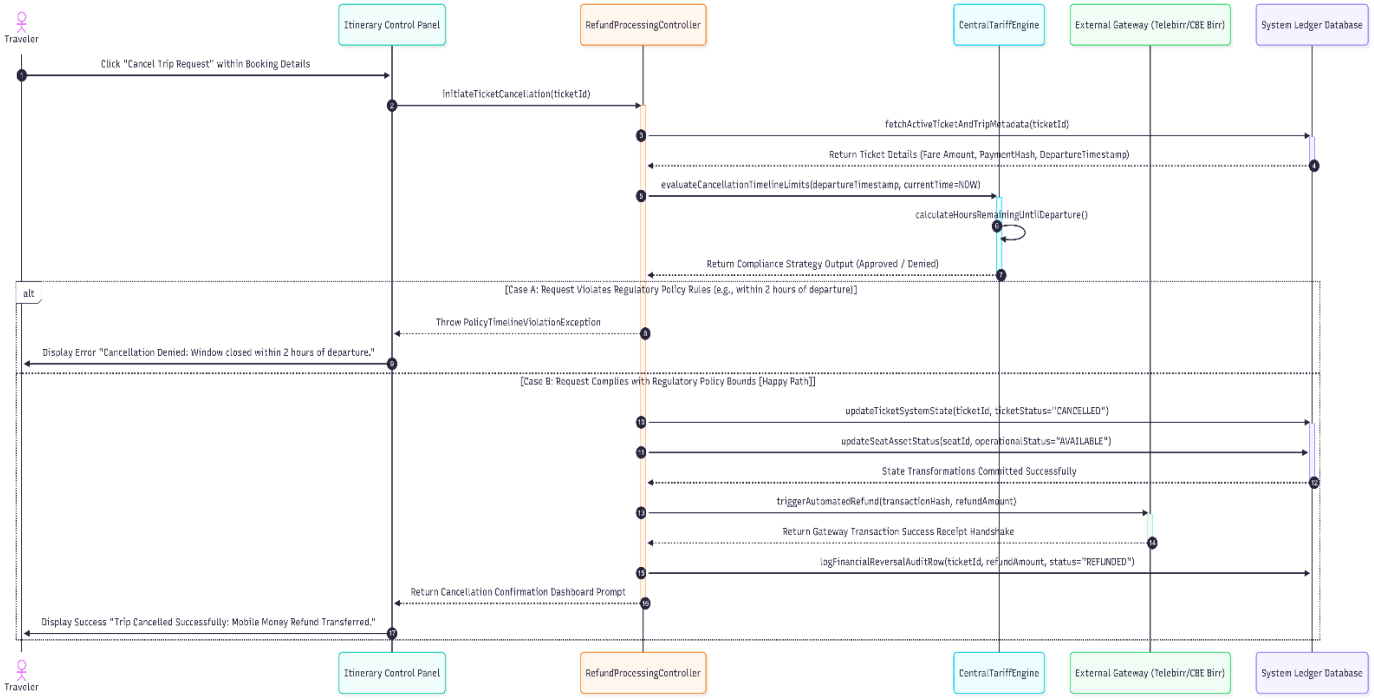
3. SD-03: Process Digital Payment and Ticket Generation



4. SD-04: Automatic Seat Release (Timeout)



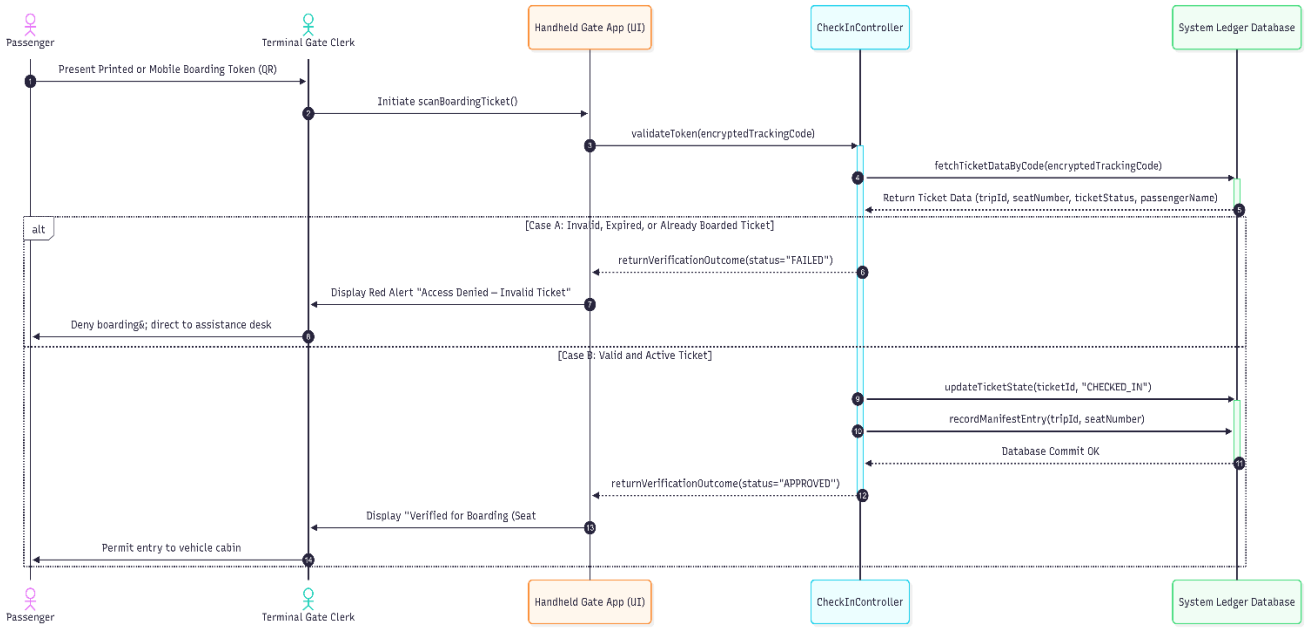
5. SD-05: Ticket Cancellation and Automated Refund



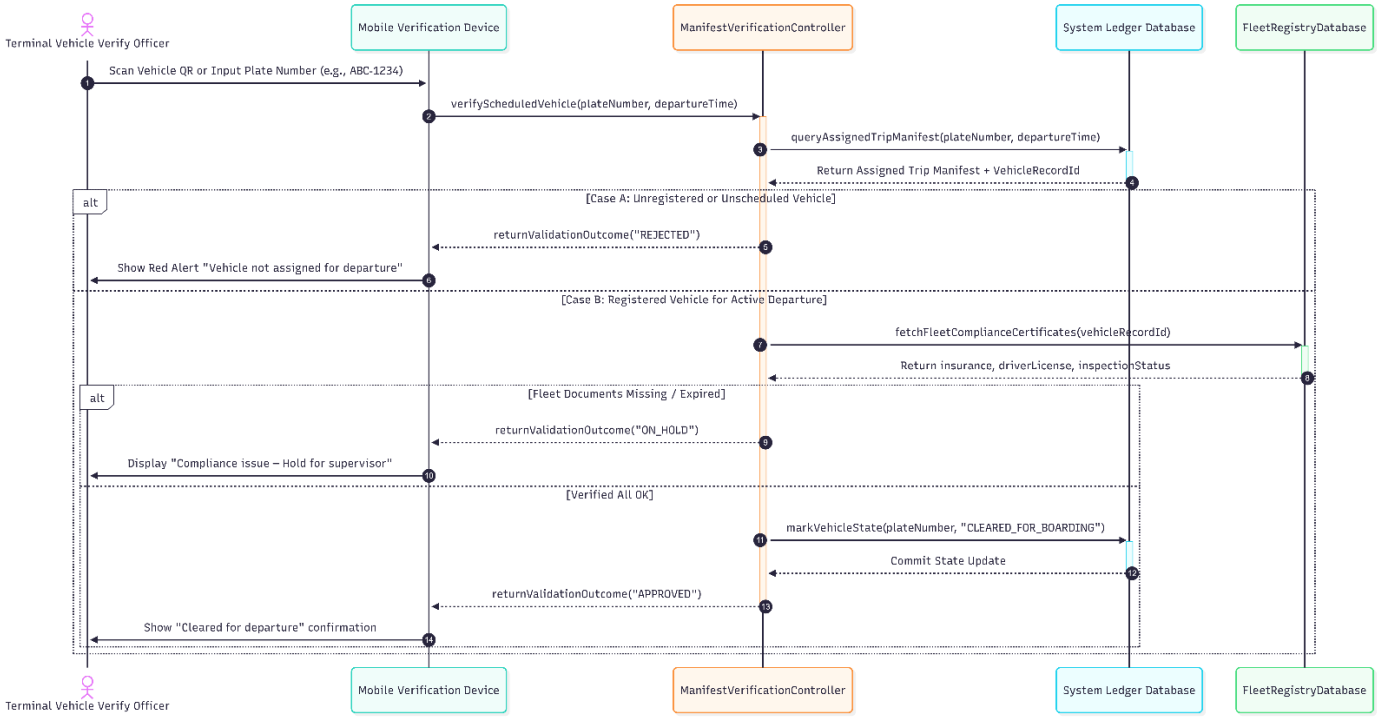
Category 2: Operational & Staff Diagrams

These model how terminal yard operations, physical station boundaries, and over-the-counter sales are securely digitized.

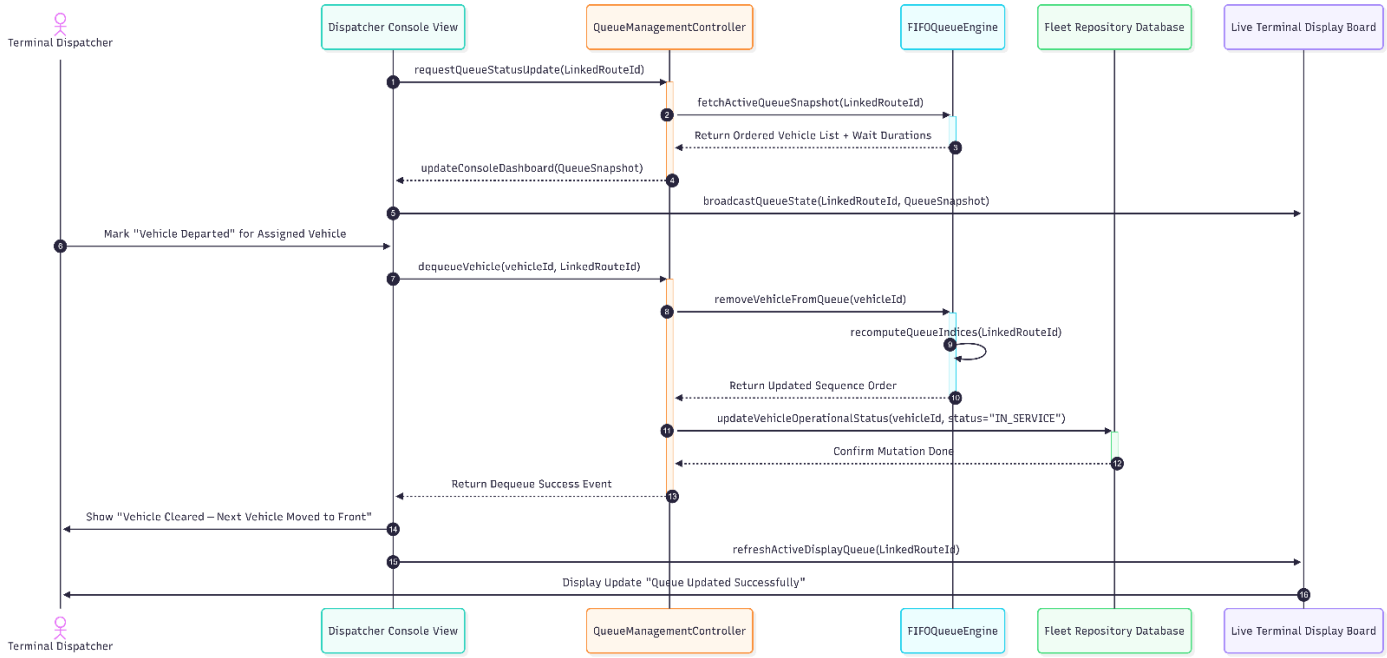
6. SD-06: Ticket Verification and Boarding



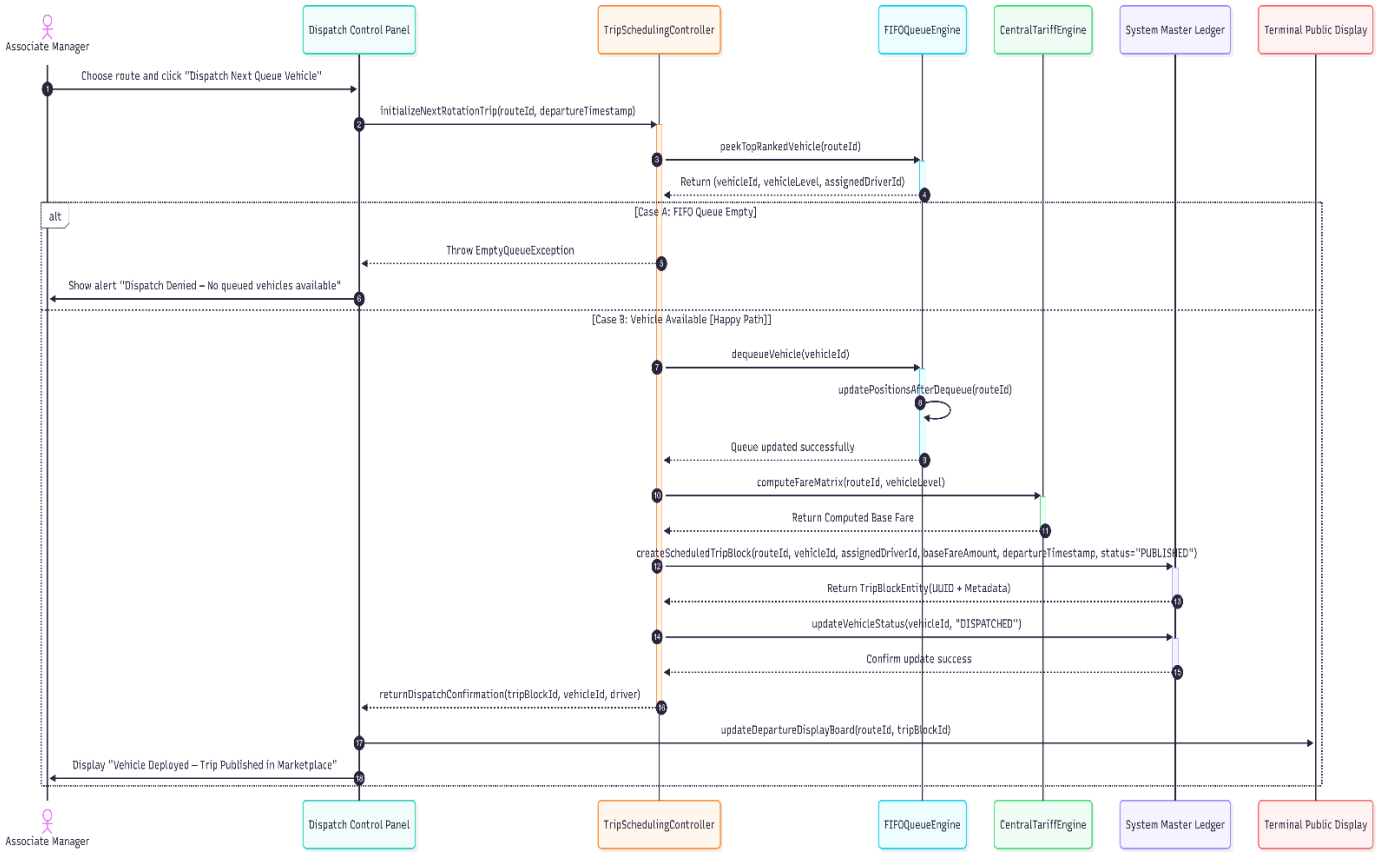
7. SD-07: Process Walk-in Booking



8. SD-08: Automated Vehicle FIFO Queue Enrollment



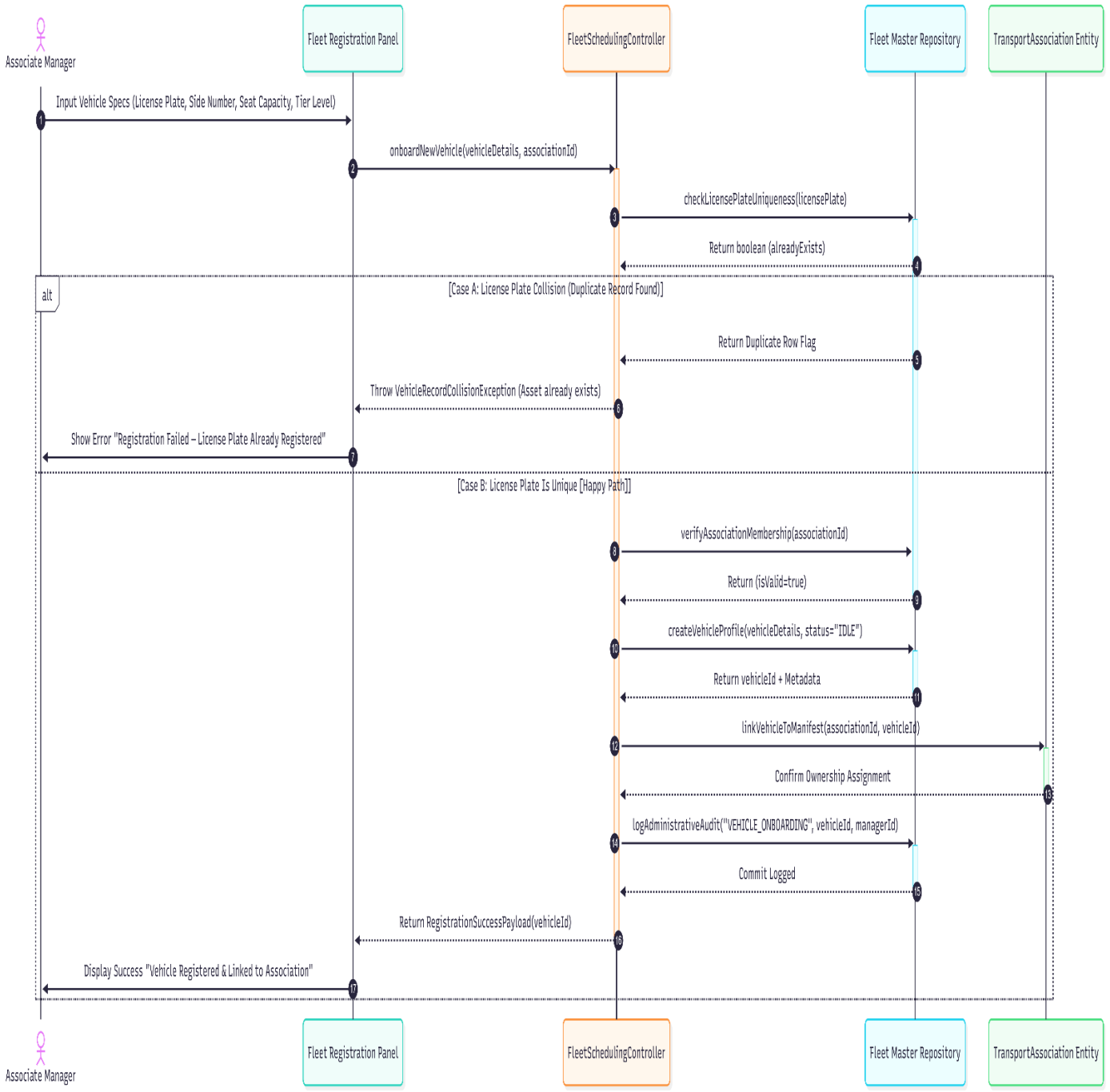
9. SD-09: Route Departure Rotation Assignment



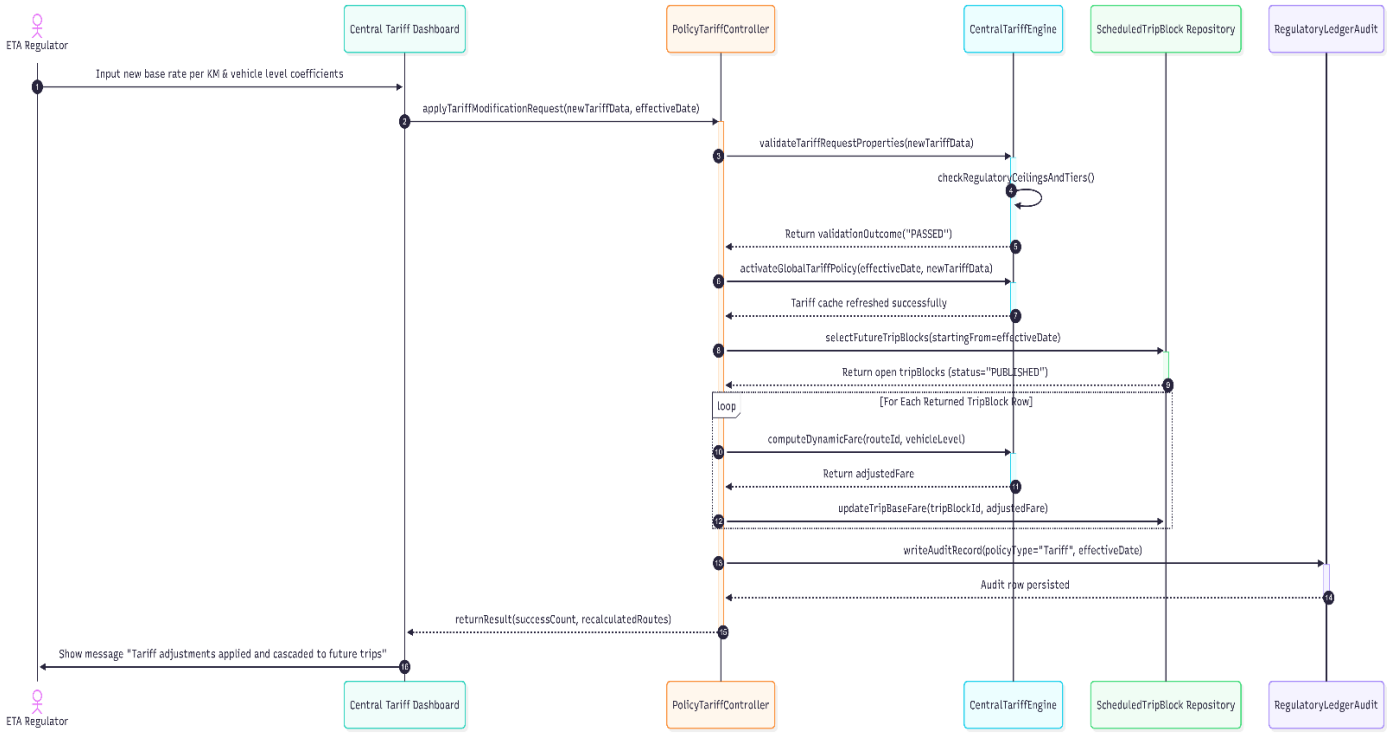
Category 3: Administrative, Policy & Exception Diagrams

These capture macro system management, regional tariff modifications, and unexpected hardware/fleet operational failures.

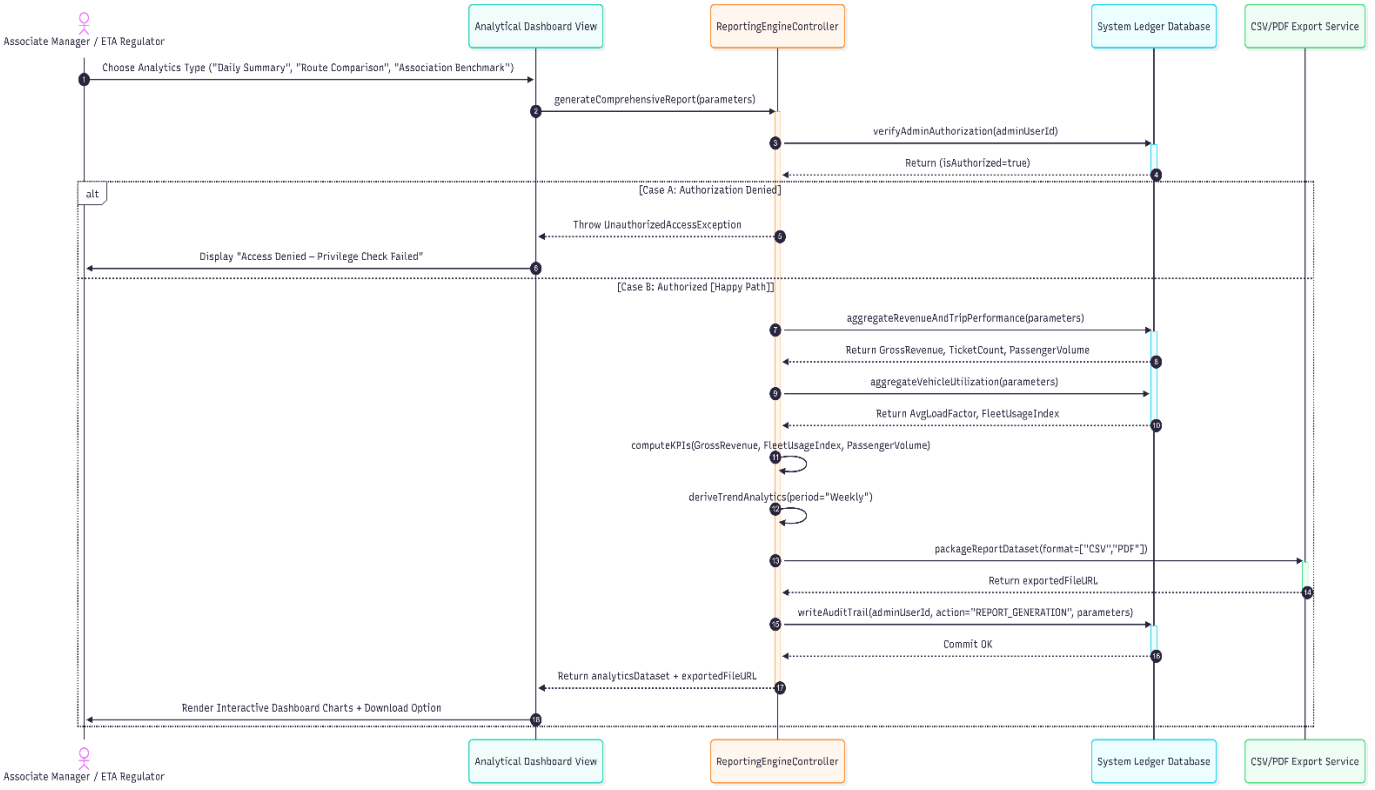
10. **SD-10: Add and Schedule a New Bus**



11. **SD-11: System Configuration (Tariff Update)**



12. **SD-12: Generate Performance Report**

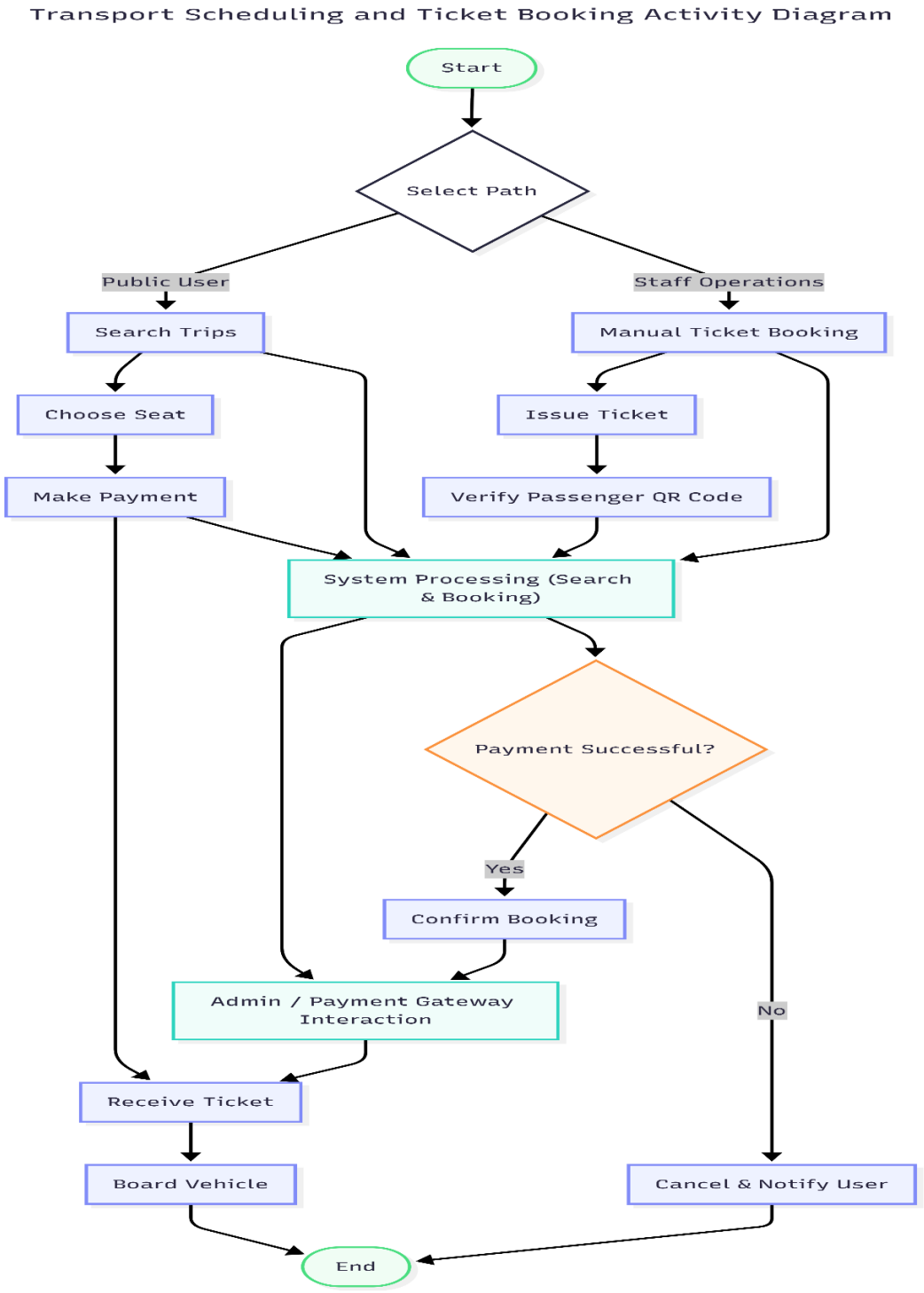


13. **SD-13: Mid-Trip Vehicle Breakdown & Emergency Handling**

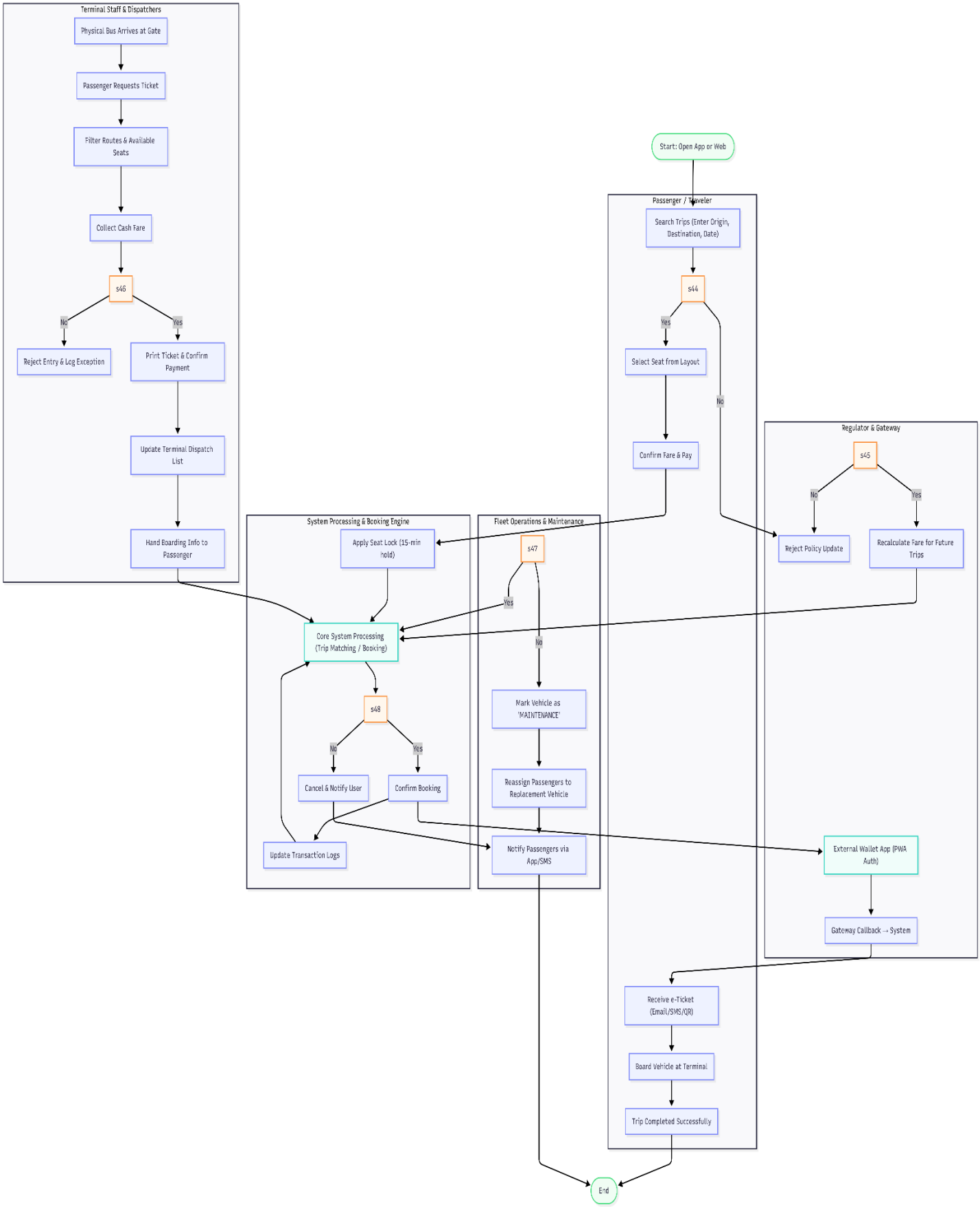


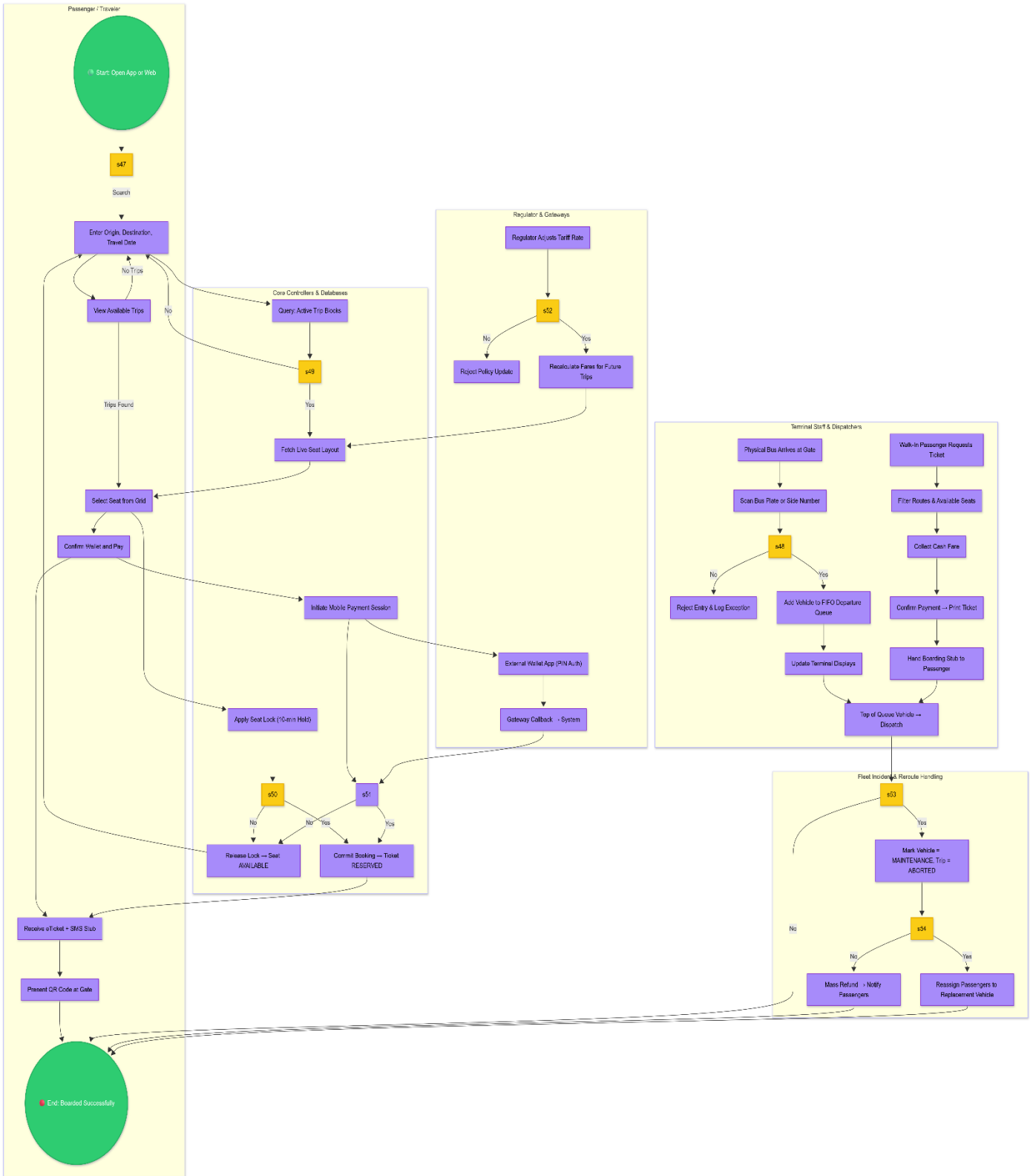
3.6.2 Activity Diagram

The activity diagram represents the procedural flow of activities within the system. It highlights the decision points and parallel workflows, specifically focusing on the passenger's journey from searching for a bus to receiving a digital ticket, as well as the system's logic for handling seat locking and payment timeouts.



Transport Scheduling and Ticket Booking – Integrated Activity Diagram




Passenger / Traveler
Start: Open App or Web
s47
Search
Enter Origin, Destination, Travel Date
No Trips
View Available Trips
No
Trips Found
Select Seat from Grid
Confirm Wallet and Pay
Initiate Mobile Payment Session
Apply Seat Lock (10-min Hold)
Receive eTicket + SMS Stub
Present QR Code at Gate
End: Boarded Successfully
Core Controllers & Databases
Query: Active Trip Books
s49
Yes
Fetch Live Seat Layout
s50
s51
No
No
Yes
Yes
Release Lock — Seat AVAILABLE
Commit Booking — Ticket RESERVED
Regulator & Gateways
Regulator Adjusts Tariff Rate
s52
No
Yes
Reject Policy Update
Recalculate Fares for Future Trips
External Wallet App (PIN Auth)
Gateway Callback → System
Terminal Staff & Dispatchers
Physical Bus Arrives at Gate
Scan Bus Plate or Side Number
s48
No
Yes
Reject Entry & Log Exception
Add Vehicle to FIFO Departure Queue
Update Terminal Displays
Top of Queue Vehicle → Dispatch
Walk-in Passenger Requests Ticket
Filter Routes & Available Seats
Collect Cash Fare
Confirm Payment → Print Ticket
Hand Boarding Stub to Passenger
Fleet Incident & Reroute Handling
s53
Yes
Mark Vehicle = MAINTENANCE, Trip = ABORTED
No
s54
No
Yes
Mass Refund + Notify Passengers
Reassign Passengers to Replacement Vehicle

3.6.3 Activity Diagram Descriptions

1. Passenger Booking and Seat Management Activity Workflow

This workflow focuses strictly on the customer-facing client interface layer (Web and Mobile applications). It outlines the core transactional loop required to take a user from initial route query to guaranteed seat allocation, serving as primary evidence of client-side application stability.

- Runtime Execution & Concurrency Safety: The diagram showcases a critical structural milestone via an execution fork. Once a passenger selects an open seat coordinate node within the interactive matrix interface, the system splits processing into two concurrent, asynchronous threads. The first thread interacts directly with the core database engine to acquire an immediate exclusivity hold on the row. The second thread triggers an API handshake with regional mobile money payment providers (Telebirr / CBE Birr), prompting a secure M-PIN processing view directly on the traveler's device.
- Exception Boundary & Timeout Rules: A prominent decision point monitors the lifecycle of the transaction. The system implements a strict 10-minute cron drop-timer to avoid inventory stagnation. If the external payment gateway fails to return a verified success webhook payload before the 10-minute window closes, the system activates an automated cleanup script. The temporary reservation record is completely purged, and the targeted coordinate attribute is reset to AVAILABLE, protecting the inventory from getting permanently stuck due to abandoned checkouts.
- Operational Outcome: Upon receipt of a valid transaction success token from the banking provider within the designated timeline, a join node merges the parallel processing paths. The system commits a permanent status mutation changing the ticket line to RESERVED, compiles a unique cryptographic tracking hash, and outputs a digital boarding pass featuring a scannable QR code alongside an out-of-band automated SMS receipt stub.

2. Administrative and Fleet Management Activity Workflow

This workflow details the heavy-duty operational backend managed via desktop browser views by authorized Transport Association Managers and ETA Regulators. It handles the data flow necessary to enroll infrastructure assets and turn static resource scheduling inputs into dynamic marketplace options.

- Data Validation & Onboarding Contours: The process initiates when a manager attempts to register a physical vehicle asset into the platform ecosystem. The form captures distinct engineering and logistics properties, including license plate strings, body side numbers, seat capacities, and level-tier classifications. To ensure data integrity across the fleet repo, a strict database constraint check runs. If a duplicate license plate is discovered, the process is intercepted, a record collision exception is thrown, and the administrative dashboard blocks database entry.

- Conflict Resolution & Schedule Release: When scheduling a voyage, the engine demands a vehicle ID, an active driver profile, a pre-mapped route corridor, and specific timestamp pairs. To eliminate scheduling conflicts, the scheduling controller evaluates active assignments. If the targeted bus or driver is caught overlapping with an existing block, the system halts execution and prompts manual error correction. Valid schedules are instantiated with an isolated state tag of DRAFT, ensuring they remain hidden from the consumer-facing marketplace during initial profiling.
- State Machine Transitions: The final transition from a localized DRAFT to a public PUBLISHED state is bound to physical terminal compliance. The system requires the designated vehicle to pass terminal gate check-in, where it is automatically appended to the tail-end of the route's FIFO Queue array. Once the vehicle's position rank is verified, the trip status updates to PUBLISHED, which triggers an immediate background price calculation loop using the regulatory tariff matrix and pushes the live trip block to passenger search streams in real time.

3. Unified Cross-Functional Swimlane System Diagram

This high-level integrated model captures the macro behavioral lifecycle of the entire system inside a unified blueprint. It uses clear horizontal and vertical boundaries (Swimlanes) to map out the exact separation of concerns between human operators, backend engines, and third-party infrastructure.

- Partitioning of Business Domains: The diagram maps out the boundaries between distinct system areas, dividing roles logically to match your system design:
    - Passenger/Traveler Zone: Handles client-side actions like entering destination criteria, selecting seats, and viewing digital QR passes.
    - Terminal Staff & Dispatchers Zone: Models physical counter tasks, cash banknote validation, thermal paper printing for walk-in tickets, and gate camera scanning.
    - System Processing & Tracking Engine: The core system layer that manages fast database locks, runs the 10-minute session countdowns, updates transaction tables, and keeps passenger lists accurate.
    - Regulator & Gateway Infrastructure: Tracks macro policy changes made by the ETA Regulator and monitors async callbacks coming from external banking webhooks.
- Cross-Boundary Edge Logic: The main value of this diagram is how it visualizes data moving between different layers of the application. For example, it traces how an offline over-the-counter cash transaction at a terminal ticket window instantly triggers a database lock in the central engine, protecting that seat from being booked by someone using the mobile app at the same time. It also details the complex connection between a passenger

hitting "Pay" on their phone, the external payment gateway processing the secure PIN, and the asynchronous webhook callback firing back to update the central ledger database.

- Mid-Trip Emergency Exception Recovery: The bottom sections of the swimlane chart display the system's defensive exception logic for real-world disruptions, specifically handling mid-trip vehicle breakdowns. When an incident is flagged, the application quickly isolates the damaged vehicle's status to MAINTENANCE and marks the trip block as ABORTED. The engine immediately pulls the active passenger list and checks the station yard for available backup buses.

If a backup bus is found, a batch migration process runs to map passengers to the new vehicle's seating grid, and the system sends an automated SMS alerting travelers to look for the replacement license plate. If no backup buses are available, the system automatically switches to a fail-safe path, moving all affected tickets to a REFUND_PENDING state and placing them in a queue for automatic mobile money reversals.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

The system design phase serves as the structural bridge between detailed requirements analysis and the physical implementation of the Ethiopian Transport Scheduling and Ticket Booking System. This chapter describes the high-level software architecture, system process decompositions, and strategic technical decisions made to ensure the platform is highly scalable, operationally secure, and executionally efficient. It outlines the methodologies used to translate the abstract functional profiles and non-functional quality attributes identified in Chapter 3 into a rigorous, authoritative technical blueprint.

By mapping abstract transit workflows onto a concrete multi-tier runtime framework, this section provides an engineering roadmap customized for the unique operational realities of the Ethiopian public transit sector. The design introduces targeted solutions to mitigate the risks of high-concurrency seat transaction bottlenecks, accommodates regional mobile banking gateway constraints, and provides programmatic error-handling exceptions tailored to withstand localized terminal connectivity drops and vehicle breakdown variations.

4.2 System Purpose

The primary purpose of this design is to establish an automated, reliable technical framework for managing the public transport lifecycle within the Ethiopian national transit network. This system architecture directly replaces fragmented manual workflows with an integrated digital ecosystem focused on three core operational transformations:

SYSTEM PURPOSE

1. CENTRALIZATION: Terminal Ledgers ──► Cloud DB Core

2. MOBILITY: Station Queues ──► Mobile App UI

3. ACCOUNTABILITY: Loose Slips ──► Digital Audit

- Centralization: Eliminating disconnected manual logbooks, localized paper ledgers, and terminal chalkboards. It replaces them with a single, authoritative, cloud-synchronized relational database cluster. This consolidates real-time seat allocations, active vehicle rosters, driver assignments, and official tariff structures across geographically distributed regional hubs managed by the Ethiopian Transport Authority (ETA) and local transit associations.
- Mobility: Enforcing a decoupled, multi-channel presentation layer that allows passengers to query transit streams via cross-platform mobile applications or responsive web views from any location. This remote accessibility eliminates the need to stand in long physical

queues at terminal ticket windows hours before departure. It brings route discovery, live visual seat maps, and secure ticket validation straight to the traveler's device.

- Accountability: Providing the ETA, regional station dispatch units, and independent transport associations with transparent, unalterable digital audit trails. Every electronic seat acquisition, walk-in cash ticket sale, passenger manifest modification, terminal yard queue update, and manual tariff change is cryptographically tracked to ensure financial clarity and administrative oversight.

4.3 Design Goals

To satisfy the academic criteria of Woldia University and meet the rigorous field requirements of the ETA and local transport operators, the system's software architecture is governed by four foundational design goals:

- Data Integrity: Enforcing strict ACID (Atomicity, Consistency, Isolation, Durability) parameters on all transactional inventory mutations. The platform uses targeted database isolation controls to completely block race conditions, stale metadata views, and double-booking bugs. This safeguards inventory precision even during high-traffic booking bursts across web and mobile channels.
- Scalability: Structuring the underlying application services to handle expanding data volumes and user traffic smoothly. The system architecture is built to remain responsive under standard daily traffic, with the ability to scale dynamically during major national holiday seasons (e.g., Fasika, Eid, Genna) when cross-regional passenger travel spikes exponentially across national transport corridors.
- Security: Deploying a multi-layered defense perimeter across all network endpoints. This includes stateless encryption token configurations, strict role-based access control (RBAC) middleware, data hashing algorithms, and Transport Layer Security protocols. These safeguards protect user identities, administrative records, and payment gateway connections from unauthorized access or alteration.
- User-Centricity: Engineering clean, accessible, and high-performance frontends tailored for varying levels of digital literacy. The layout configurations prioritize clear navigation paths, localized text patterns, and immediate visual feedback to accommodate both app-centric mobile users and walk-in passengers relying on printed thermal stubs.

4.4 Current Software Architecture

4.4.1 Overview

As established during the field research and manual system analysis phase, there is currently zero software architecture or digital infrastructure in place. The existing "architecture" relies entirely on traditional, paper-based operational patterns. System data is captured using hand-written ink

annotations across physical ledgers distributed across isolated ticket windows, regional association booths, and terminal dispatch stations. There are no computer networks, intranet systems, centralized servers, or digital interfaces linking these entities together.

MANUAL SYSTEM DECOMPOSITION

- INFORMATION LAYER ⟶ Hand-written Blackboards & Office Signage.
- TRANSACTION LAYER ⟶ Manual Banknote Counter & Paper Receipt Stubs.
- REPORTING LAYER ⟶ Hand-tallied Driver Passenger Manifests

4.4.2 Process Decomposition

The manual workflows observed across typical regional transport stations can be broken down into three completely isolated, non-synchronized layers:

- The Information Layer: Scheduled routes, vehicle assignments, departure timetables, and estimated pricing tables are manually written on terminal blackboards or pasted to office doors using paper signs.
- The Transaction Layer: Walk-in passengers present cash banknotes at a specific association ticket window. The terminal clerk checks an index card or paper sheet of available seats, crosses out a number by hand, takes the cash, and manually fills out a pre-printed paper receipt stub.
- The Reporting Layer: Once a vehicle reaches capacity or its scheduled departure time, the clerk hand-tallies the remaining paper receipt stubs. These numbers are used to type or write out a single physical passenger manifest, which is physically handed to the bus driver before departure.

4.4.3 Key Limitations

- High Error Proximity Rates: Hand-written ledger updates and manual data transfers across separate paper files cause record mismatch bugs, un-reconciled account balances, and math errors.
- Lack of Real-Time Visibility: The institution's global financial exposure—such as total active deposits, outstanding principal, and risk-weighted portfolio ratios—cannot be calculated dynamically. Data must be hand-tallied across records, delaying management decisions.
- Poor Audit Traceability: Reviewing account change histories or tracing specific financial transactions requires sorting through physical boxes and folders. This reliance on paper trails compromises internal controls and slows down regulatory audits.
- Absence of Role Access & Data Controls: The security of member data is limited to physical room locks. There are no technical logs tracking user activities, no field-level

validation rules, and no systems to prevent an entry error from updating historic ledger data.

- Low Growth Potential: The paper-centric workflow cannot handle expanding membership pools or higher daily transaction volumes without causing significant operational bottlenecks and increasing staffing costs.

4.5 Proposed System Architecture

4.5.1 Overview

The proposed system addresses historical operational limitations by deploying a decoupled, high-performance Three-Tier Architecture organized around a Service-Oriented Architecture (SOA) paradigm. This framework splits the application into independent Presentation, Application, and Data layers, allowing client interfaces to evolve without disrupting core business rules or transactional data repositories.

The system's technology stack is explicitly chosen for transactional durability, low-latency performance under concurrent loads, and cross-platform mobile ecosystem compatibility:

- **Backend Core Framework:** Employs the enterprise-grade **Spring Boot (Java)** framework to run the primary business logic engines, manage security filters, handle relational mappings via Hibernate Object-Relational Mapping (ORM), and enforce automated request processing.
- **Database Engine:** Utilizes an optimized, ACID-compliant **PostgreSQL Relational Database Management System (RDBMS)** configured to secure system-wide persistence, enforce strict referential integrity constraints, and execute row-level pessimistic locking (SELECT ... FOR UPDATE) during high-volume checkout spikes.
- **Frontend Implementations:** Enforces a dedicated user experience using modern single-page frameworks (**Next.js**) for web dashboard portals and cross-platform native compilation frameworks (**Flutter**) for multi-role mobile application environments.

4.5.1.1 Logical Three-Tier Architectural Mapping

To ensure structural separation of concerns and maintainable deployment boundaries, the interaction pathways between components are formally classified into three distinct, interconnected structural tiers:

I. Presentation Tier (User Experience Layer)

The presentation layer serves as the decoupled front-end boundary, completely isolated from direct data repository interactions. It handles user input capture, renders visual state indicators, and presents functional layout views depending on authenticated system permissions:

- **Cross-Platform Mobile Application (Flutter):** Provides the user interface for *Passengers* to execute remote route searches, visualize seating grids, and checkout single-seat selections, while simultaneously functioning as a handheld portal for *Drivers* to track assignments and *Terminal Clerks/Dispatchers* to execute fast ticket QR scanning validation workflows at terminal gates.
- **Responsive Web Management Portal (Next.js):** Functions as the point-of-sale (POS) interface for local terminal ticket booth clerks and delivers administrative command workspaces for *Associate Managers* and *Ethiopian Transport Authority (ETA) Regulators* to oversee vehicle management matrices, driver rosters, and automated financial accounting reports.

II. Application Tier (Core Logic & Routing Layer)

The application tier acts as the centralized computational processing center, executing transport logistics policies and managing data mapping between client channels and the underlying data store:

- **Reverse Proxy / Secure API Gateway:** Acts as the unified point of entry for all incoming Secure HTTPS REST API requests. The gateway handles TLS/SSL terminations, prevents Cross-Origin Resource Sharing (CORS) exceptions, blocks unauthorized headers, and intercepts inbound requests to run rate-limiting routines that protect backend services from denial-of-service spikes.
- **Core Business API Services (Spring Boot Core):** The centralized engine running your core application components. This includes the *User Authentication & RBAC Module* (signing state-free JWT session payloads), the *Scheduling Engine Module* (managing First-In, First-Out yard queue arrays), the *Booking & Seat Management Module* (enforcing the 10-minute pessimistic hold lock and the 1 ticket per passenger transaction limit), and the *Reporting & Analytics Module* (computing the automated $5\% / 95\%$ commission breakdown and locking passenger manifests when vehicle bookings cross the 80% capacity threshold).

III. Data Tier (Persistence Layer)

The data tier represents the physical storage and persistence layer of the system architecture, operating independently beneath the business logic services:

- **PostgreSQL Database Engine:** A relational database cluster running in full compliance with ACID (Atomicity, Consistency, Isolation, Durability) properties. Communication with this tier is driven entirely by Hibernate ORM database abstraction frameworks, completely eliminating manual sql vulnerabilities. The engine maintains relational data schemas in strict Third Normal Form (3NF) to support instant route indexing, prevent fleet inventory

duplication errors, and isolate active trip seating grids during high-concurrency reservation windows.

4.5.2 System Modules & Responsibilities

The system software is organized into four core functional modules that interact seamlessly through an internal application routing tier:

Module Index	Target Domain	Core Module Responsibilities & System Functions
1	**User Authentication & RBAC**	Manages user lifecycles, handles secure login/logout transitions, generates stateless JWT session tokens, and maps endpoints directly to strict Role-Based Access Control (**RBAC**) filters.
2	**Scheduling Engine**	Coordinates backend logic for registering physical route paths, managing vehicle assets within the terminal yard, establishing departure timetables, and updating real-time queue states.
3	**Booking & Seat Management**	Controls live graphical seat layouts, monitors inventory counts, processes database reservation entries, and handles the automated tracking parameters for temporary 10-minute seat isolation holds.
4	**Reporting & Analytics**	Aggregates real-time transaction records to compile digital passenger manifests for drivers and dispatchers, while generating financial split summaries and operational metrics for transport associations and the ETA.

4.5.3 Deployment & Infrastructure

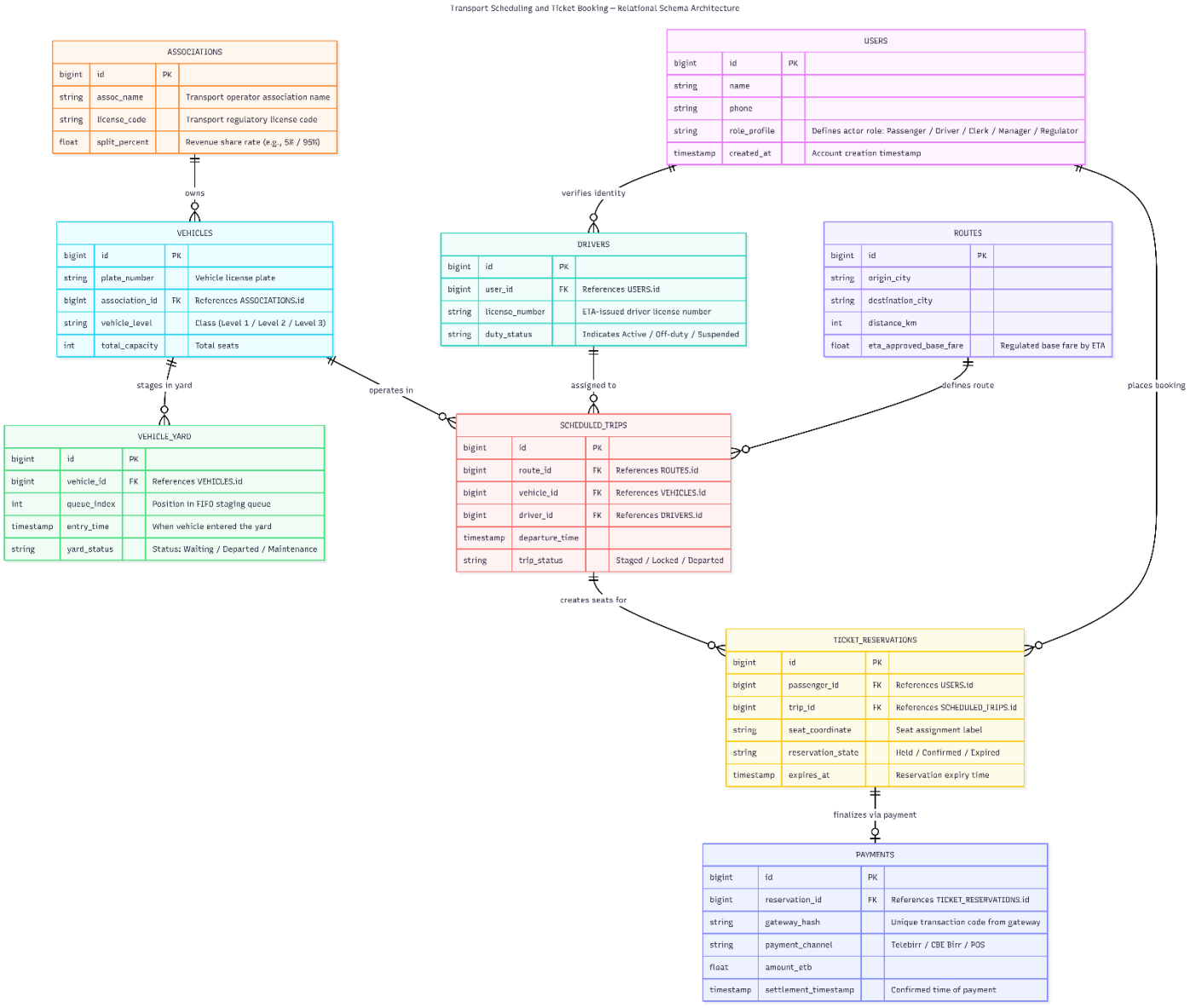
The system architecture uses a cloud-hosted Client-Server infrastructure design engineered to maximize platform availability, enforce horizontal scaling properties, and reduce network response latency:

- **Server-Side Architecture:** The application core and relational database engines are hosted on virtual private clouds running a secure Linux Ubuntu distribution. It leverages an **Nginx** configuration acting as a reverse proxy web server to manage inbound load-balancing, SSL/TLS certificate terminations, and API request acceleration.
- **Client-Side Frameworks:** Administrative panels, terminal association portals, and counter POS booking systems are built using a responsive **Next.js** web application. Passenger self-service operations and terminal gate-checking workflows run within a unified, cross-platform **Flutter** mobile application compiled natively for Android and iOS systems.

- **API Gateway Interception:** A secure API Gateway serves as the single point of entry for all incoming client traffic. The gateway interceptor handles reverse-proxy routing, enforces rate-limiting configurations to protect against denial-of-service threats, and strips unauthorized cross-origin resource sharing (CORS) blocks before payloads hit the lower services.

4.5.4 Database Design

The persistence layer relies on a highly optimized **PostgreSQL** Relational Database Management System (RDBMS). The relational table schemas are strictly normalized to **Third Normal Form (3NF)** to ensure absolute data consistency, eliminate structural redundancy, and maintain strict referential integrity across the system lifecycle.



Concurrency Control & Row-Level Locking

To prevent database row corruption and double-booking bugs during concurrent high-volume reservation spikes, the database tier avoids broad table locks. Instead, the platform uses a precise, row-level pessimistic isolation mechanism. When a user selects an open seat coordinate, the application issues an explicit row-level lock command:

This query isolates that specific seat row in the trip matrix table. Concurrent transaction requests seeking that same seat allocation node are forced to wait in line until the primary checkout thread either commits successfully or hits its 10-minute expiration timer, ensuring complete inventory accuracy across all point-of-sale paths.

4.5.5 Security & Access Control

- **Stateless Token Authentication:** User authentication states are entirely stateless and driven by JSON Web Tokens (JWT). Upon successful parameter validation at login, the server issues an encrypted token payload signed with a secure server key. Client applications store this token locally and inject it into the authorization header of every subsequent API request:
- **Role-Based Access Control (RBAC):** Request verification filters inspect incoming token claims, blocking unauthorized actors before they can access backend resources:
    - *Passengers* can access endpoint paths for trip searching, checking seat layout states, and executing digital mobile wallet checkouts.
    - *Terminal Clerks* can access local counter POS interfaces, execute ticket validation routines, and process manual cash ticket sales.
    - *Terminal Dispatchers* are authorized to update vehicle terminal arrival cues, edit yard ranks, and generate passenger manifests.
    - *Associate Managers* retain control over fleet profile configurations, driver roster assignments, and route scheduling tools.
    - *ETA Regulators* possess global system access to manage base tariff metrics, edit regional multipliers, and audit broad transaction ledgers.
- **Encryption Standards:** All user account passwords are encrypted using the computational **BCrypt** hashing algorithm with a standard workload cost factor of 10. Data in transit across public networks is wrapped inside **Transport Layer Security (TLS 1.3 / HTTPS)**, preventing credential eavesdropping and intercept attacks.
- **Immutable Activity Logs:** A background security engine logs system actions in a separate, append-only audit ledger. It automatically tracks every login attempt, seat override action,

and setting change, capturing the timestamp and user ID. This log is protected by database constraints that block modification or erasure commands.

4.5.6 Boundary Conditions

- **Initialization:** Upon cold system boot or container cluster startup, an initialization routine prepares the database layer: It reads the active data tables to pull all scheduled trips for the current calendar date and caches them in an in-memory database store to accelerate public search speeds. It also starts a background cleanup process that scans the persistence layer to identify and release any orphaned seat locks left behind during an unexpected server termination.
- **Critical Bulk Operations:** High-impact operations that process data across multiple entries simultaneously—such as running the nightly terminal queue reset—are wrapped within strict database isolation barriers:

If an unexpected hardware crash or power failure occurs mid-run, the database automatically drops the partial changes and rolls back to its last clean state, preventing balance mismatches.

- **Exceptions & Timeout Extensions:** If an external mobile money payment gateway API (**Telebirr / CBE Birr**) becomes completely unresponsive during an active checkout step, the core engine pauses the standard 10-minute inventory expiration clock, extending the inventory hold by a 5-minute safety buffer. It returns a retry warning notification back to the client interface, allowing the passenger to retry their payment without losing their chosen seat selection.

4.6 User Interface Design

4.6.1 Navigational Paths

To ensure maximum operational efficiency and minimize transaction execution times at busy terminal centers, the system maps clear, role-specific navigation paths. Each user pathway is programmatically decoupled and optimized around core functional objectives:

User Role	Operational Workflow Target	Target Navigational Sequence & Application Viewports
Passenger	Self-service remote search, seat selection, and digital checkout.	Open Flutter Mobile App $\rightarrow$ Route Search Form $\rightarrow$ Trip Selection Feed $\rightarrow$ Interactive Vehicle Cabin Grid $\rightarrow$ Select Single Seat Coordinate $\rightarrow$ Mobile Wallet Prompt

User Role	Operational Workflow Target	Target Navigational Sequence & Application Viewports
		(Telebirr/CBE Birr) $\rightarrow$ Secure Digital Ticket Pass Generation.
Terminal Clerk	Over-the-counter walk-in passenger booking and cash handling.	Secure Login $\rightarrow$ Web POS Control Interface $\rightarrow$ Route Selection Matrix $\rightarrow$ Manual Seating Selection Layout $\rightarrow$ Input Passenger Details $\rightarrow$ Collect Cash $\rightarrow$ Generate Valid Thermal Print Boarding Stub.
Terminal Dispatcher	Check-in gate verification and vehicle release management.	Launch Handheld App Gate View $\rightarrow$ Activate Device Camera $\rightarrow$ Scan Passenger Ticket QR Code $\rightarrow$ Asynchronous Core API Verification $\rightarrow$ Render Clearance Status $\rightarrow$ Update Master Passenger Manifest.
Associate Manager	Association management of fleets, routes, and drivers.	Authenticate Credentials $\rightarrow$ Association Hub Command Dashboard $\rightarrow$ Select Management Domain (Fleet Asset Logs / Driver Roster Sheet / Active Queue Traversal Matrix) $\rightarrow$ Save Configuration States.

4.6.2 Screen Mock-ups

The application's interfaces utilize a structured, clean layout style optimized for rapid data entry and visual scannability:

- **Administrative & POS Dashboard Viewport:** Employs a persistent top navigation bar tracking system health indicators, an expandable left sidebar for navigating across active functional modules, and a main container workspace. The central area displays real-time data grids tracking route allocations, vehicle terminal queue states, and dynamic booking velocities.

- **Vehicle & Driver Registry Interface:** Utilizes a clean, tabbed view that groups comprehensive metadata for a specific transport asset onto one layout grid. Authorized managers can quickly toggle between technical specs, active driver assignments, historical journey logs, and revenue generation profiles, removing the need to bounce between separate screens.
- **Bulk Operation Safety Interface (Trip Batch Generation):** Implements a strict two-step confirmation system to prevent administrative data errors during high-impact changes. Before committing a new schedule block, operators view a read-only preview listing all affected vehicle timelines and tariff calculations. Final execution requires confirming a secure modal overlay, completely avoiding accidental database overwrites.

4.7 Detailed Component Specifications

4.7.1 Financial Fare Calculation & Revenue Split Engine

The billing subsystem processes passenger ticket prices dynamically using an automated, rule-based calculation engine. Rather than relying on manual calculations by station staff, the core engine combines macro baseline tariffs regulated by the Ethiopian Transport Authority (ETA) with individual vehicle class modifiers to calculate final costs:

$$\text{Total Fare (ETB)} = \left(\text{Base Route Tariff} \times \text{Vehicle Level Modifier} \right) + \text{Terminal Station Overhead Fee}$$

1. Calculation Parameter Inputs

- **Base Route Tariff (B_t):** Computed by cross-referencing the explicit distance matrix between the origin and destination terminals with the national per-kilometer tariff rate enforced by the regulatory authority:

$$\text{Base Route Tariff} = \text{Distance (km)} \times \text{ETA Baseline Tariff Rate per km}$$

- **Vehicle Level Modifier (V_m):** A predefined multiplier score tied directly to the structural comfort classification and asset grade of the operating vehicle to accommodate varying operational expenditures:
    - **Level 1 Vehicles** (Luxury Fleet / High-Spec Air Conditioned Coaches): Multiplier = 1.30.
    - **Level 2 Vehicles** (Standard Mid-Tier Regional Fleets): Multiplier = 1.15.
    - **Level 3 Vehicles** (Economy Fleets / Standard Inter-City Minibuses): Multiplier = 1.00.

- **Terminal Station Overhead Fee (T_f):** A fixed administrative yard overhead price of 10.00 ETB appended onto every single issued ticket to support localized terminal station maintenance, security management, and sanitation workflows.

2. Automated Financial Routing Splitter

Upon receiving a successful payment confirmation webhook from integrated mobile money networks, the engine intercepts the gross transaction amount and programmatically distributes the funds into separate ledger sheets before updating the persistence layer, ensuring a clear division of resources:

$$\text{Regulatory Maintenance Split} = \text{Total Fare} \times 0.05 \quad (5\% \text{ Terminal Service Fee})$$

$$\text{Vehicle Operator Split} = \text{Total Fare} \times 0.95 \quad (95\% \text{ Association/Owner Balance})$$

This rule-based allocation maintains clear financial accountability, automatically routing the terminal's 5% administrative fee to the operating authority's ledger while securing the remaining 95% balance for the vehicle operator.

4.7.2 Dispatch Approval Workflow Engine

The system coordinates and tracks a vehicle's life cycle within the terminal yard across four sequential, highly ordered operational stages controlled by the dispatch engine:

4.7.2.1 Operational Lifecycle States

- **Stage 1: Terminal Queue Enrollment:** Upon physical arrival at the terminal yard checkpoint, the vehicle operator reports to the dispatcher kiosk. The system validates that the vehicle profile is active and confirms the linked driver is within legal duty limits to prevent operator fatigue. The engine then appends the vehicle's unique identification key to the back of the specific route's First-In, First-Out (FIFO) Queue Array.
- **Stage 2: Trip Instantiation:** When the vehicle reaches the front (Head-of-Line) of the FIFO queue array, the dispatch engine shifts its status tracking indicator. The engine extracts the vehicle asset from the waiting queue, attaches it to the next upcoming scheduled travel block, matches it to the calculated fare parameters, and publishes the active trip instance live to passenger search feeds.
- **Stage 3: Seating Manifest Locking:** The booking engine continuously updates the seat layout matrix as transactions are processed. As soon as total seat sales across online mobile apps and local walk-in counter POS tools reach or cross an 80% capacity threshold, the engine locks down the seating grid, generates the master passenger list, and marks the file as ready for dispatch on the workstation terminal.

- **Stage 4: Final Dispatch Authorization:** The terminal dispatcher completes a final physical check of the vehicle, reviews the passenger records, and executes the "Authorize Departure" action command on their terminal interface. This instantly changes the trip status code to DEPARTED, transmits the digital passenger manifest straight to the operator's mobile portal view, and clears the boarding lane for the next vehicle staged in line.

4.8 Integration Interfaces

The system architecture features clean, modern interface layers to coordinate financial settlements and deliver real-time travel alerts across system borders:

- **Regional Mobile Payment Gateway APIs (Telebirr & CBE Birr):** The financial system connects directly with regional mobile money banking infrastructure using secure REST APIs. The core engine passes an encrypted JSON billing payload to the provider endpoint to trigger the consumer's phone validation prompt. Once the user enters their PIN, the provider platform captures the funds and fires an asynchronous callback message back to the application's webhook listener to confirm payment success.
- **Cellular SMS Gateway Interfacing:** The platform integrates with a localized cellular gateway service to provide robust, text-based notifications. When a ticket is successfully issued or a schedule is disrupted by a vehicle breakdown, the notifications worker formats a text template and pushes it to the SMS gateway provider via a secure API connection, delivering critical travel updates straight to the passenger's mobile number.

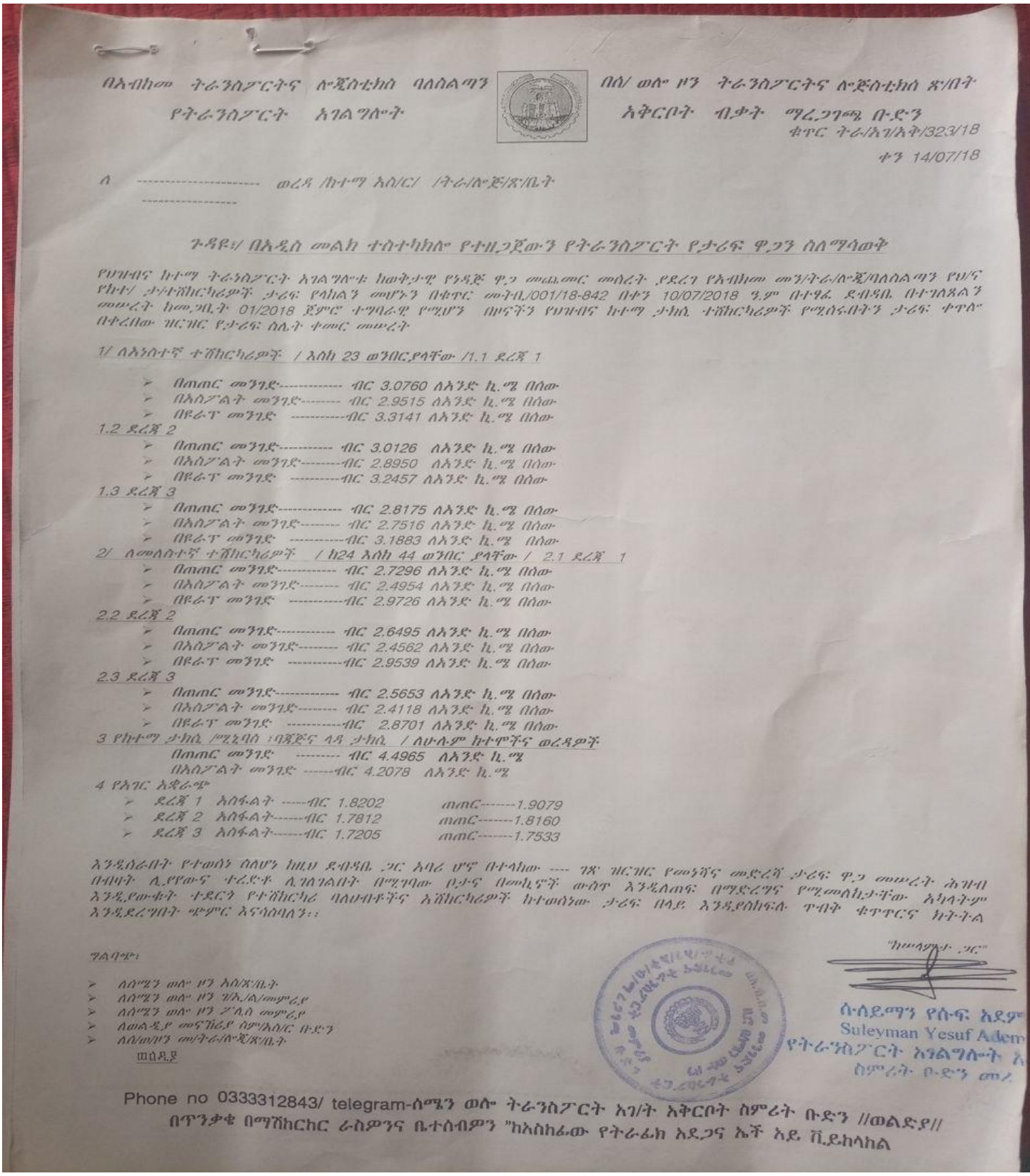
Core System API Endpoint Specifications

To maintain total consistency across the platform, client-side views and external integrations communicate with the central server via a standardized collection of secure API endpoints:

- POST /api/v1/auth/login - Validates user login attempts, verifies security roles, and returns a signed, state-free JSON Web Token (JWT) access token.
- GET /api/v1/trips/search?origin={id}&destination={id}&date={date} - Queries available active trips matching route criteria, returning classification levels, pricing matrices, and open seat counts.
- POST /api/v1/bookings/seat-lock - Initiates a row-level database lock on a chosen seat coordinate index, establishing an isolated, 10-minute countdown session block.
- POST /api/v1/payments/initialize - Securely outbounds structured JSON billing payloads directly to external mobile wallet APIs (Telebirr/CBE Birr) to launch merchant checkout processes.

- PUT /api/v1/dispatch/authorize/{trip_id} - Confirms the terminal dispatcher's approval signature, changes the trip operational status tracking flag to DEPARTED, and delivers the completed passenger manifest to the driver's interface.

Appendix



በአብክመ ትራንስፖርትና ሎጂስቲክስ ባለስልጣን
የትራንስፖርት አገልግሎት

በሰ/ ወሎ ዞን ትራንስፖርትና ሎጅስቲክስ ጽ/ቤት
አቅርቦት ብቃት ማረጋገጫ ቡድን
ቁጥር ትራ/አገ/አት/323/18
ቀን 14/07/18

ለ ---------- ወረዳ /ከተማ አስ/ር/ /ትራ/ሎጅ/ጽ/ቤት

ጉዳዩ:/ በአዲስ መልክ ተስተካክሎ የተዘጋጀውን የትራንስፖርት የታሪፍ ዋጋን ስለማሳወቅ

የህዝብና ከተማ ትራንስፖርት አገልግሎቱ ከወቅታዊ የነዳጅ ዋጋ መጨመር መሰረት ያደረገ የአብክመ መን/ትራ/ሎጂ/ባለስልጣን የህ/ና የከተ/ ታሪፍ ማሻሻያ ዋጋ የለከልን መሆኑን በቁጥር መ/ብ/001/18-842 በቀን 10/07/2018 ዓ.ም በተፃፈ ደብዳቤ በተገለጸልን መሠረት ከመ.ጋቢት 01/2018 ጀምሮ ተግባራዊ የሚሆን በሀገራችን የህዝብና ከተማ ታክሲ ተሸከርካሪዎች የሚሰሩበትን ታሪፍ ቀጥሎ በቀረበው ዝርዝር የታሪፍ ስሌት ቀመር መሠረት

1/ ለአንስተኛ ተሸከርካሪዎች / እስከ 23 ወንበር ያላቸው /1.1 ደረጃ 1

- በጠጠር መንገድ---------- ብር 3.0760 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ-------- ብር 2.9515 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 3.3141 ለአንድ ኪ.ሜ በሰው

1.2 ደረጃ 2

- በጠጠር መንገድ---------- ብር 3.0126 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ--------ብር 2.8950 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 3.2457 ለአንድ ኪ.ሜ በሰው

1.3 ደረጃ 3

- በጠጠር መንገድ---------- ብር 2.8175 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ-------- ብር 2.7516 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 3.1883 ለአንድ ኪ.ሜ በሰው

2/ ለመለስተኛ ተሸከርካሪዎች / ከ24 እስከ 44 ወንበር ያላቸው / 2.1 ደረጃ 1

- በጠጠር መንገድ---------- ብር 2.7296 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ-------- ብር 2.4954 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 2.9726 ለአንድ ኪ.ሜ በሰው

2.2 ደረጃ 2

- በጠጠር መንገድ---------- ብር 2.6495 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ-------- ብር 2.4562 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 2.9539 ለአንድ ኪ.ሜ በሰው

2.3 ደረጃ 3

- በጠጠር መንገድ---------- ብር 2.5653 ለአንድ ኪ.ሜ በሰው
- በአስፖልት መንገድ-------- ብር 2.4118 ለአንድ ኪ.ሜ በሰው
- በየራፕ መንገድ ----------ብር 2.8701 ለአንድ ኪ.ሜ በሰው

3 የከተማ ታክሲ /ሚኒባስ :ባጃጅና ላዳ ታክሲ / ለሁሉም ከተሞችና ወረዳዎች

በጠጠር መንገድ --------- ብር 4.4965 ለአንድ ኪ.ሜ
በአስፖልት መንገድ ------ብር 4.2078 ለአንድ ኪ.ሜ

4 የአገር አቋራጭ

- ደረጃ 1 አስፋልት -----ብር 1.8202 ጠጠር-------1.9079
- ደረጃ 2 አስፋልት-------ብር 1.7812 ጠጠር-------1.8160
- ደረጃ 3 አስፋልት-------ብር 1.7205 ጠጠር-------1.7533

እንዲሰራበት የተወሰነ ስለሆነ ከዚህ ደብዳቤ ጋር አባሪ ሆኖ ቀርቧለው ---- ገጽ ዝርዝር የመንግስና መድረሻ ታሪፍ ዋጋ መሠረት ሕዝብ በብዛት ሊያየውና ተረድቶ ሊገለገልበት በሚገባው ቦታና በመኪናዎች ውስጥ እንዲለጠፍ በማድረግና የሚመለከታቸው አካላትም እንዲያውቁት ተደርጎ የተሸከርካሪ ባለሀብቶችና አሽከርካሪዎች ከተመለከው ታሪፍ በላይ እንዳይስከፍሉ ጥብቅ ቁጥጥርና ክትትል እንዲደረግበት ሞያሮ እናሳስባለን::

"ከሠላምታ ጋር"

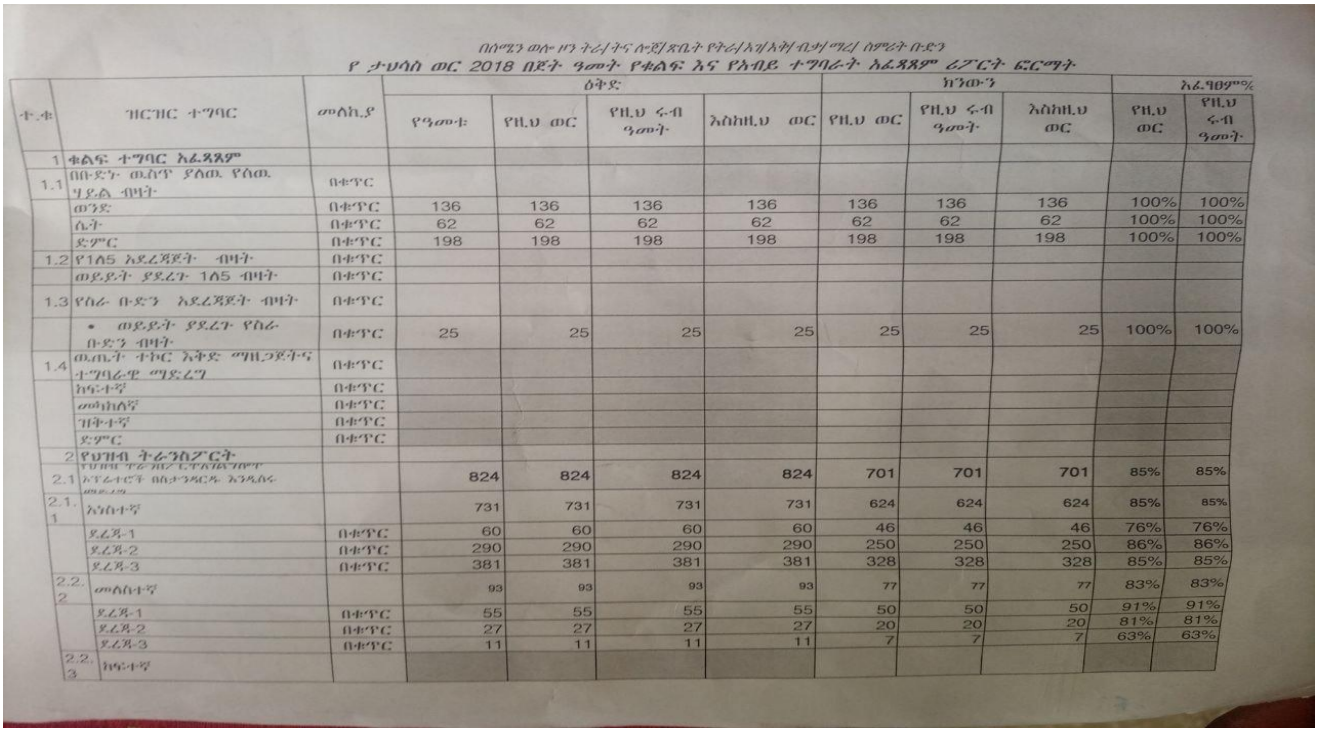
ሱሰይማን የሱፍ አደም
Suleyman Yesuf Adem
የትራንስፖርት አገልግሎት አ
ስምሪት ቡድን መሪ

ግልባጭ:

- ለሰሜን ወሎ ዞን አስ/ጽ/ቤት
- ለሰሜን ወሎ ዞን ፖሊስ/መምሪያ
- ለሰሜን ወሎ ዞን ፖሊስ መምሪያ
- ለወልዲያ መናኸሪያ ሱም/አስ/ር ቡድን
- ለስ/ወሀን መ/ትራ/ሎ/ጽ/ቤት

ጠቋሚያ

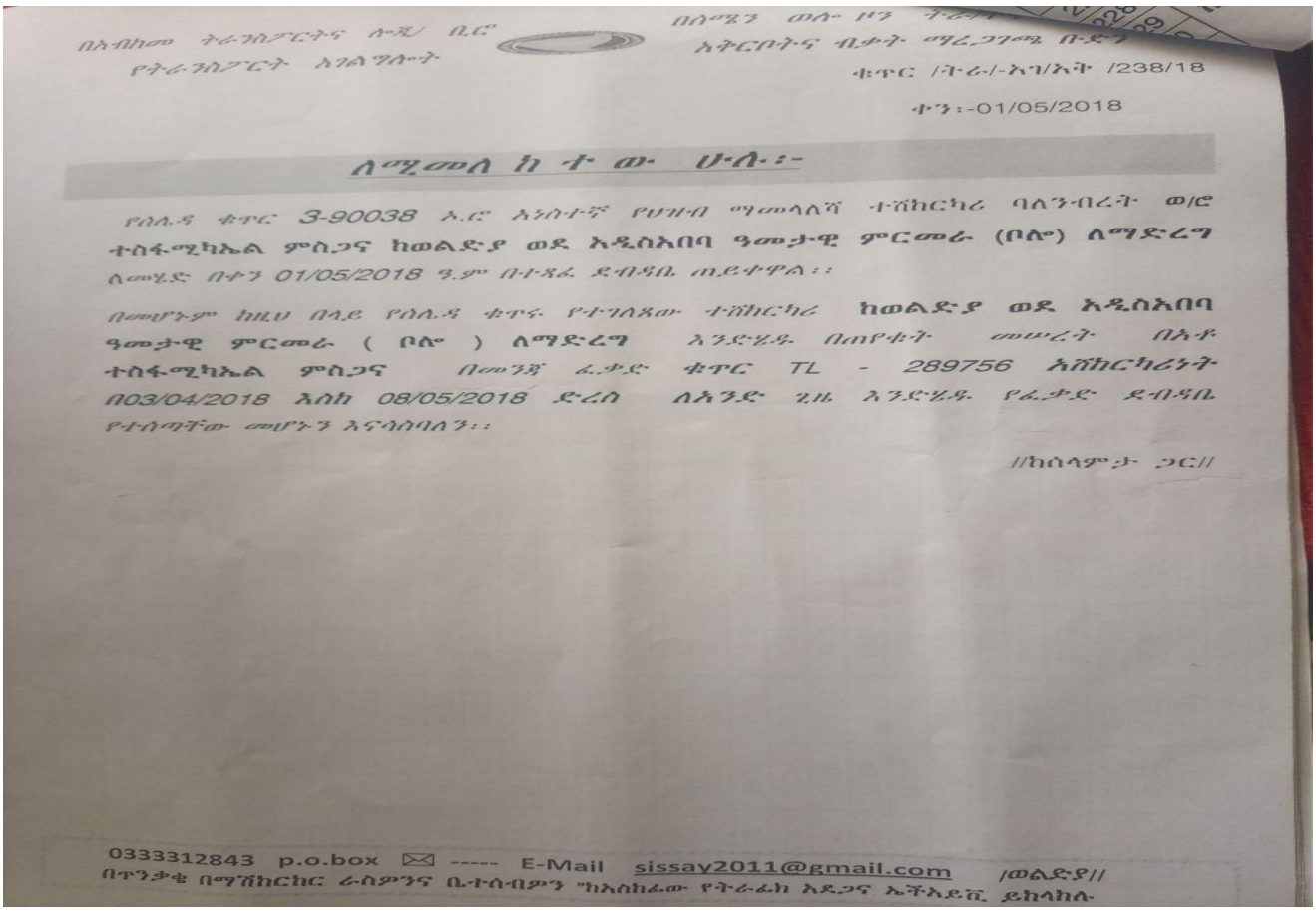
Phone no 0333312843/ telegram-ሰሜን ወሎ ትራንስፖርት አገ/ት አቅርቦት ስምሪት ቡድን //ወልድያ//
በጥንቃቄ በማሻሻርከር ራስዎንና ቤተሰብዎን "ከአስከፊው የትራፊክ አደጋና ኤች አይ ቪ.ይከላከል



በሰሜን ወሎ ዞን ትራንስፖርትና ሎጂስቲክስ የትራፊክ አዛዥነት ብቃት ማረጋገጫ ስሚሰጥ ቡድን

የ ታህሳስ ወር 2018 በጀት ዓመት የቁልፍ እና የአብይ ተግባራት አፈጻጸም ሪፖርት ፎርማት

ተ.ቁ	ዝርዝር ተግባር	መለኪያ	ዕቅድ				ክንውን			አፈፃፀም%	
			የዓመቱ	የዚህ ወር	የዚህ ሩብ ዓመት	እስከዚህ ወር	የዚህ ወር	የዚህ ሩብ ዓመት	እስከዚህ ወር	የዚህ ወር	የዚህ ሩብ ዓመት
1	ቁልፍ ተግባር አፈጻጸም										
1.1	በቡድኑ ውስጥ ያለዉ የለዉ ሃይል ብዛት	በቁጥር									
	ወንድ	በቁጥር	136	136	136	136	136	136	136	100%	100%
	ሴት	በቁጥር	62	62	62	62	62	62	62	100%	100%
	ድምር	በቁጥር	198	198	198	198	198	198	198	100%	100%
1.2	የ1ለ5 አደረጃጀት ብዛት	በቁጥር									
	ወይይት ያደረጉ 1ለ5 ብዛት	በቁጥር									
1.3	የሰራ ቡድን አደረጃጀት ብዛት	በቁጥር									
	• ወይይት ያደረጉ የሰራ ቡድን ብዛት	በቁጥር	25	25	25	25	25	25	25	100%	100%
1.4	ዉጤት ተኮር እቅድ ማዘጋጀትና ተግባራዊ ማድረግ	በቁጥር									
	ከፍተኛ	በቁጥር									
	መካከለኛ	በቁጥር									
	ዝቅተኛ	በቁጥር									
	ድምር	በቁጥር									
2	የህዝብ ትራንስፖርት										
2.1	የህዝብ ትራንስፖርት አገልግሎት አገልግሎት አቅራቢዎች በስታንዳርዱ እንዲሰሩ መደረጉ		824	824	824	824	701	701	701	85%	85%
2.1.1	አነስተኛ		731	731	731	731	624	624	624	85%	85%
	ደረጃ-1	በቁጥር	60	60	60	60	46	46	46	76%	76%
	ደረጃ-2	በቁጥር	290	290	290	290	250	250	250	86%	86%
	ደረጃ-3	በቁጥር	381	381	381	381	328	328	328	85%	85%
2.2.2	መለስተኛ		93	93	93	93	77	77	77	83%	83%
	ደረጃ-1	በቁጥር	55	55	55	55	50	50	50	91%	91%
	ደረጃ-2	በቁጥር	27	27	27	27	20	20	20	81%	81%
	ደረጃ-3	በቁጥር	11	11	11	11	7	7	7	63%	63%
2.2.3	ከፍተኛ										



በአብክመ ትራንስፖርትና ሎጂ/ ቢሮ
የትራንስፖርት አገልግሎት

በሰሜን ወሎ ዞን ትራንስፖርት
አትርቦትና ቢቃት ማረጋገጫ ቡድን

ቁጥር /ት-ራ/-አገ/አት /238/18

ቀን፡-01/05/2018

ለሚመለ ከ ተ ው ሁሉ፡-

የሰሌዳ ቁጥር 3-90038 አ.ሮ አነስተኛ የህዝብ ማመላለሻ ተሽከርካሪ ባለንብረት ወ/ሮ **ተስፋሚካኤል ምስጋና ከወልድያ ወደ አዲስአበባ ዓመታዊ ምርመራ (ቦሎ) ለማድረግ** ለመሄድ በቀን 01/05/2018 ዓ.ም በተጻፈ ደብዳቤ ጠይቀዋል፡፡

በመሆኑም ከዚህ በላይ የሰሌዳ ቁጥሩ የተገለጸው ተሽከርካሪ **ከወልድያ ወደ አዲስአበባ ዓመታዊ ምርመራ (ቦሎ) ለማድረግ** እንድሄዱ በጠየቁት መሠረት በአቶ **ተስፋሚካኤል ምስጋና** በመንጃ ፈቃድ ቁጥር TL - 289756 **አሽከርካሪነት** በ03/04/2018 እስከ 08/05/2018 ድረስ ለአንድ ጊዜ እንድሄዱ የፈቃድ ደብዳቤ የተሰጣቸው መሆኑን እናሳስባለን፡፡

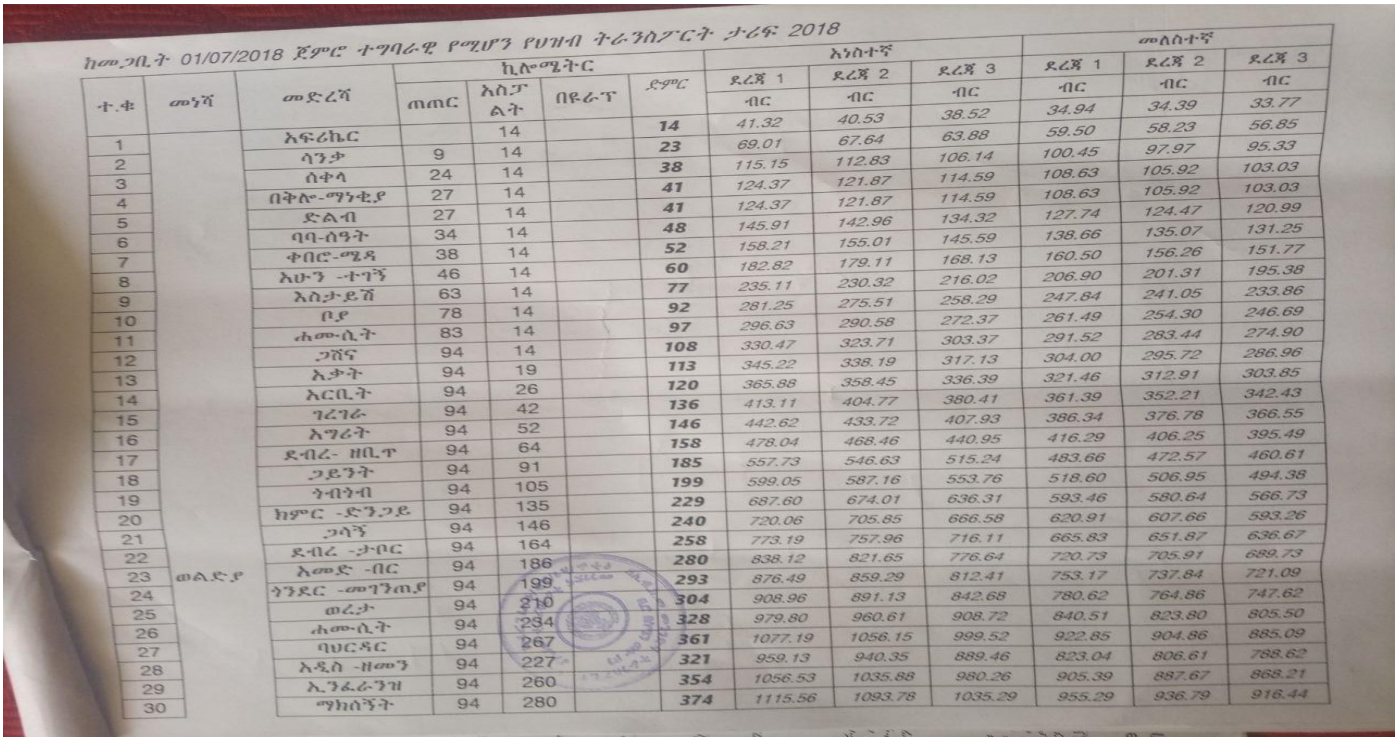
//ከሰላምታ ጋር//

0333312843 p.o.box ✉ ----- E-Mail sissay2011@gmail.com /ወልድያ//

በጥንቃቄ በማሽከርከር ራስዎንና ቤተሰብዎን "ከአስከፊው የትራፊክ አደጋና ኤችአይቪ ይከላከሉ

7.1	የተከበረ የመናኸሪያ ስታንዳርድ	በ%	87	87	87	87	82	82	94	94%	94%
7.2	ያልተከበረ የመናኸሪያ ስታንዳርድ ብዛት	በቁጥር									
7.3	ነባር መናኸሪያዎች ብዛት	በቁጥር	25	25	25	25	25	25	25	100%	100%
7.4	ርክክብ የተደረገባቸው አዲስ መናኸሪያዎች ብዛት	በቁጥር									
7.5	የሚሃን አገልግሎት ተግባራዊ የሆነባቸው መናኸሪያች ብዛት	በቁጥር	2	2	2	2	2	2	2	100%	100%
7.6	የትኬት ኮሪደር ተግባራዊ የሆነባቸዉ መናኸሪያዎች ብዛት	በቁጥር	3	3	3	3	6	6	6	100%	100%
7.7	በመናኸሪያ ዉስጥ ለመንገደኞችና ለኦፕሬተሮች በየቀኑ የ15 ደቂቃ የግንዛቤ ማስጨበጫ ትምህርት	በቁጥር	5	5	5	5	8	8	8	100%	100%
8	በኢ-ቲኬቲንግ አገልግሎት የሚሰጡ መናኸሪያዎች ብዛት		3	3	3	3	6	6			
8.1	አዲስ	በቁጥር	2	2	2	2	5	5	5	100%	100%
8.2	ነባር	በቁጥር	1	1	1	1	1	1	1	100%	100%
9	በኢ-ቲኬቲንግ የተፈጠረ ስራ ዕድል										
9.1	ወንድ	በቁጥር	6	6	6	6	29	29	29	100%	100%
9.2	ሴት	በቁጥር	3	3	3	3	18	18	18	100%	100%
	ድምር	በቁጥር	9	9	9	9	47	47	47	100%	100%
10	በህዝብ ትራንስፖርት የተሰጠ ስምሪት፣ የተጓዝዝ መንገደኛ ፣ የተሸፈነ ኪ.ሜ	በቁጥር									
10.1	የተሰጠ ስምሪት በምልልስ	በቁጥር									
	አነስተኛ	በቁጥር	906466	75539	226617	453,234	78250	232048	465,281	100%	100%
	መሰስተኛ	በቁጥር	323738	26978	80934	161,868	28050	83114	168,672	100%	100%
	ከፍተኛ	በቁጥር	64748	5396	16188	32376	5568	16496	35,224	100%	100%
	ድምር	በቁጥር	**1294952**	**107913**	**323739**	**647,478**	**111868**	**331658**	**669,177**	**100%**	**100%**
10.2	የተጓዝዝ መንገደኛ በምልልስ	በቁጥር									
10.2.1	አነስተኛ	በቁጥር									
	ወንድ	በቁጥር	4500000	375000	1125000	2,250,000	378250	1134698	2,446,976	100%	100%
	ሴት	በቁጥር	1743754	145313	435939	871,878	150200	450444	775,075	100%	100%

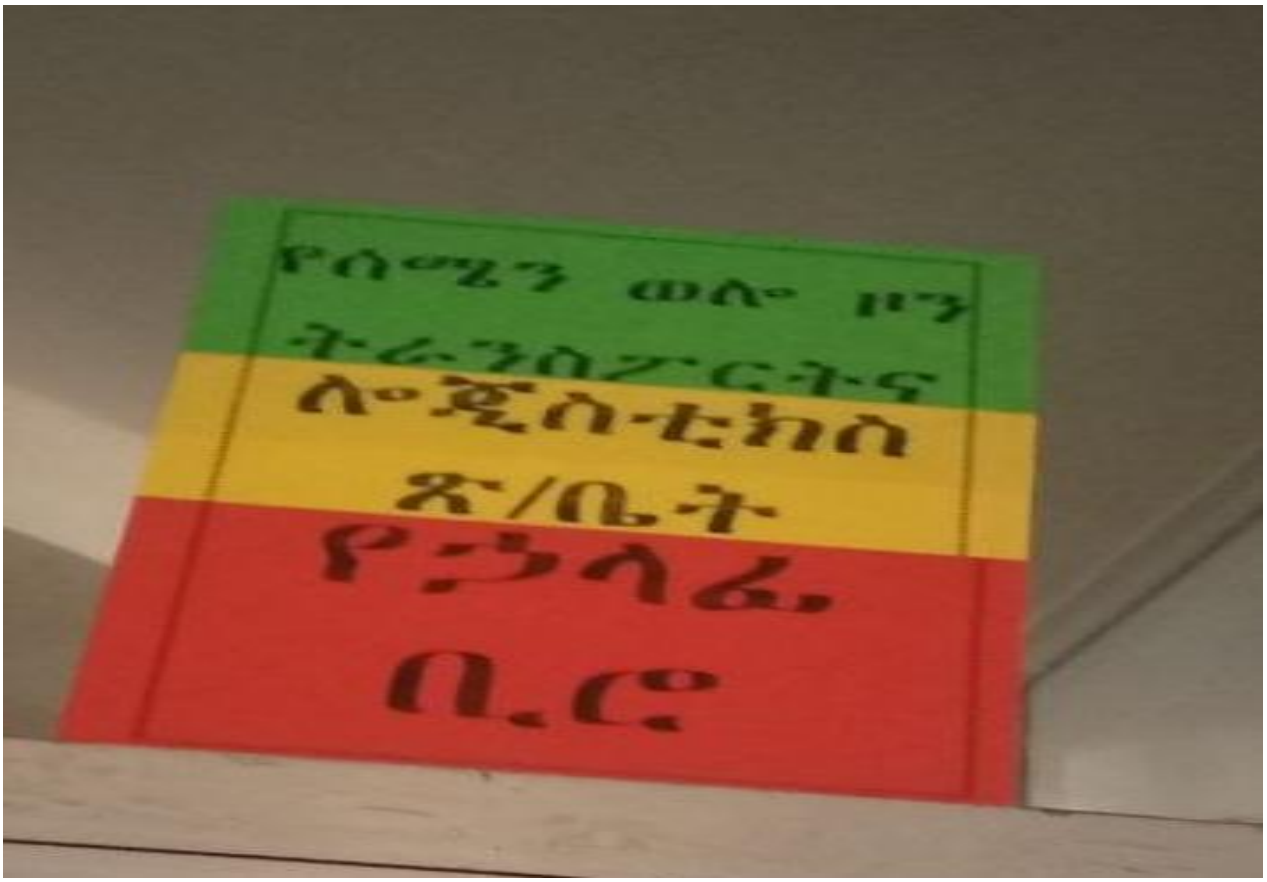
65		ላቦ ሚሌ		87		**87**	*256.78*	*251.87*	*239.39*	*217.10*	*213.69*	*209.83*
66		ሐይቅ		92		**92**	*271.54*	*266.34*	*253.15*	*229.58*	*225.97*	*221.89*
67		ሱሉሳ		102		**102**	*301.05*	*295.29*	*280.66*	*254.53*	*250.53*	*246.00*
68		ጢጣ		111		**111**	*327.62*	*321.35*	*305.43*	*276.99*	*272.64*	*267.71*
69	ከወልዲያ	ደሴ		120		**120**	*354.18*	*347.40*	*330.19*	*299.45*	*294.74*	*289.42*
70		ኮምቦልቻ		145		**145**	*427.97*	*419.78*	*398.98*	*361.83*	*356.15*	*349.71*
71		ከሚሴ		196		**196**	*578.49*	*567.42*	*539.31*	*489.10*	*481.42*	*472.71*
72		ካራ -ቆሪ		238		**238**	*702.46*	*689.01*	*654.88*	*593.91*	*584.58*	*574.01*
73		አጣየ		252		**252**	*743.78*	*729.54*	*693.40*	*628.84*	*618.96*	*607.77*
74		ሸዋ-ሮቢት		299		**299**	*882.50*	*865.61*	*822.73*	*746.12*	*734.40*	*721.13*
75		ደብረ -ሲና		331		**331**	*976.95*	*958.25*	*910.78*	*825.98*	*813.00*	*798.31*
76		ደብረ -ብርሃን		391		**391**	*1154.04*	*1131.95*	*1075.88*	*975.70*	*960.37*	*943.01*
77		ሸኖ		443		**443**	*1307.51*	*1282.49*	*1218.96*	*1105.46*	*1088.10*	*1068.43*
78		አዲስ አበባ		521		**521**	*1537.73*	*1508.30*	*1433.58*	*1300.10*	*1279.68*	*1256.55*
79		ልደታ			24	**24**	*79.54*	*77.90*	*76.52*	*71.34*	*70.89*	*71.28*
80		ገብሬ -እምባ			20	**20**	*66.28*	*64.91*	*63.77*	*59.45*	*59.08*	*59.40*
81		ዋንዛዬ			16	**16**	*53.03*	*51.93*	*51.01*	*47.56*	*47.26*	*47.52*
82		ወጠቅ			25	**25**	*82.85*	*81.14*	*79.71*	*74.32*	*73.85*	*74.25*
83		ጃርሳ		10	3	**13**	*39.46*	*38.69*	*37.08*	*33.87*	*33.42*	*32.73*
84		ቀይ-አንባ			21	**21**	*69.60*	*68.16*	*66.95*	*62.42*	*62.03*	*62.37*
85		ጭፍራ		72		**72**	*212.51*	*208.44*	*198.12*	*179.67*	*176.85*	*173.65*
86		ላስቴ			15	**15**	*49.71*	*48.69*	*47.82*	*44.59*	*44.31*	*44.55*
87		ድሀ-ወዲሀ			11	**11**	*36.46*	*35.70*	*35.07*	*32.70*	*32.49*	*32.67*
88		ጉባርጃ			12	**12**	*39.77*	*38.95*	*38.26*	*35.67*	*35.45*	*35.64*
89		እዜት- በር			29	**29**	*96.11*	*94.13*	*92.46*	*86.21*	*85.66*	*86.13*
90		ኮረታ			31	**31**	*102.74*	*100.62*	*98.84*	*92.15*	*91.57*	*92.07*
91		ሚጤ	9	14		**23**	*69.01*	*67.64*	*63.88*	*59.50*	*58.23*	*56.85*
92		አፍሪኬር- ቃሊም	5	14		**19**	*56.70*	*55.59*	*52.61*	*48.58*	*47.63*	*46.59*
93	ከአፍሪኬ	ቃሊም			5	**5**	*16.57*	*16.23*	*15.94*	*14.86*	*14.77*	*14.85*
94	ር	ሚጤ			9.2	**9.2**	*30.49*	*29.86*	*29.33*	*27.35*	*27.18*	*27.32*
95		አፍሪኬር	9			**9**	*27.68*	*27.11*	*25.36*	*24.57*	*23.85*	*23.10*
96		ቃሊም	9		5	**14**	*44.25*	*43.34*	*41.30*	*39.43*	*38.47*	*37.44*
97		ሚጤ	9		9	**18**	*57.51*	*56.32*	*54.05*	*51.32*	*50.16*	*48.92*
98		ወልዲያ	14	9		**23**	*69.63*	*68.23*	*64.21*	*60.67*	*59.20*	*57.62*

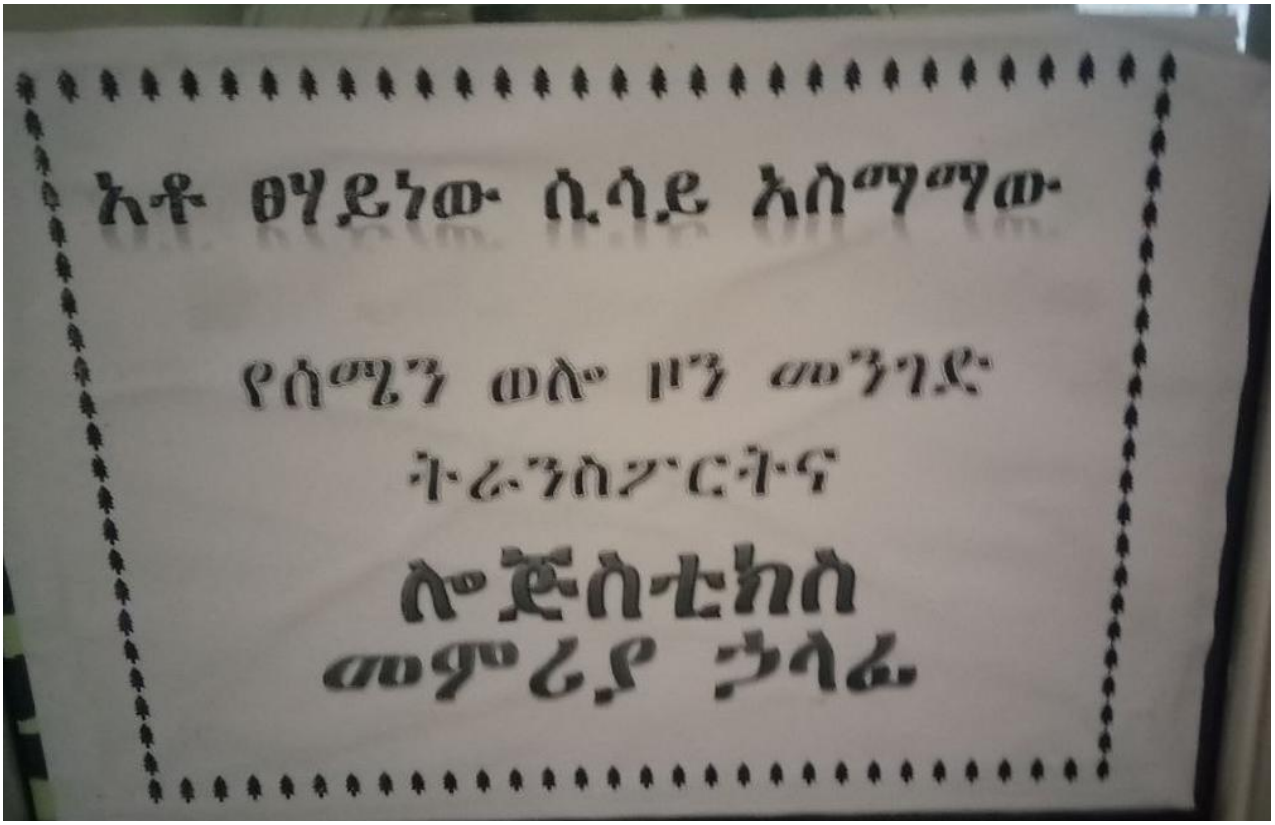


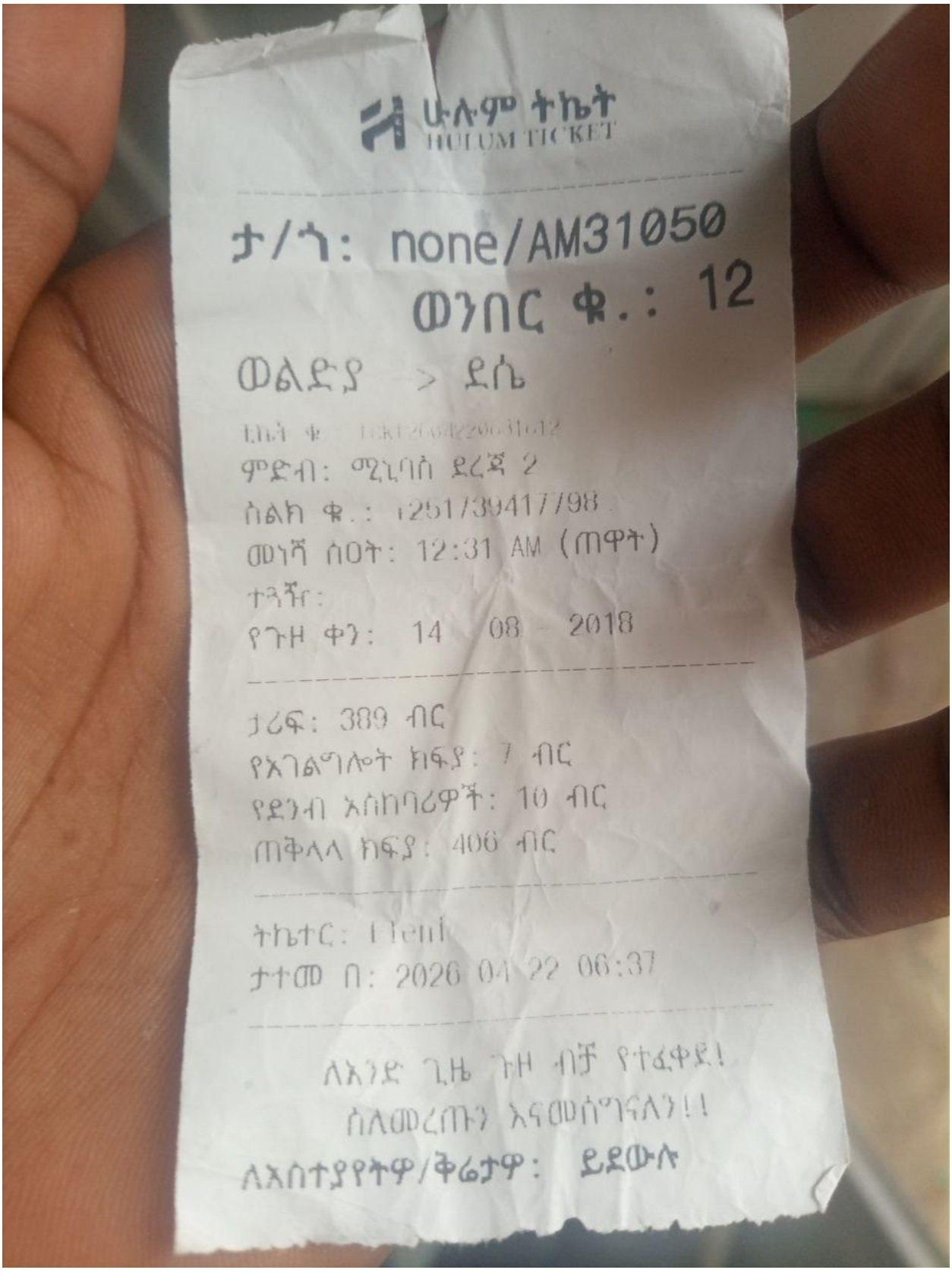
ከመጋቢት 01/07/2018 ጀምሮ ተግባራዊ የሚሆን የህዝብ ትራንስፖርት ታሪፍ 2018

ተ.ቁ	መነሻ	መድረሻ	ኪሎሜትር			ድምር	አነስተኛ			መለስተኛ		
			ጠጠር	አስፓልት	በዩራፕ		ደረጃ 1 ብር	ደረጃ 2 ብር	ደረጃ 3 ብር	ደረጃ 1 ብር	ደረጃ 2 ብር	ደረጃ 3 ብር
1		አፍሪኬር		14		14	41.32	40.53	38.52	34.94	34.39	33.77
2		ሳንቃ	9	14		23	69.01	67.64	63.88	59.50	58.23	56.85
3		ሰቀላ	24	14		38	115.15	112.83	106.14	100.45	97.97	95.33
4		በቅሎ-ማነቂያ	27	14		41	124.37	121.87	114.59	108.63	105.92	103.03
5		ድልብ	27	14		41	124.37	121.87	114.59	108.63	105.92	103.03
6		ባባ-ሰዓት	34	14		48	145.91	142.96	134.32	127.74	124.47	120.99
7		ቀበሮ-ሜዳ	38	14		52	158.21	155.01	145.59	138.66	135.07	131.25
8		አሁን -ተገኝ	46	14		60	182.82	179.11	168.13	160.50	156.26	151.77
9		እስታይሽ	63	14		77	235.11	230.32	216.02	206.90	201.31	195.38
10		ቦያ	78	14		92	281.25	275.51	258.29	247.84	241.05	233.86
11		ሐሙሲት	83	14		97	296.63	290.58	272.37	261.49	254.30	246.69
12		ጋሸና	94	14		108	330.47	323.71	303.37	291.52	283.44	274.90
13		አቃት	94	19		113	345.22	338.19	317.13	304.00	295.72	286.96
14		አርቢት	94	26		120	365.88	358.45	336.39	321.46	312.91	303.85
15		ገረገራ	94	42		136	413.11	404.77	380.41	361.39	352.21	342.43
16		አግሪት	94	52		146	442.62	433.72	407.93	386.34	376.78	366.55
17		ደብረ- ዘቢጥ	94	64		158	478.04	468.46	440.95	416.29	406.25	395.49
18		ጋይንት	94	91		185	557.73	546.63	515.24	483.66	472.57	460.61
19		ጎብጎብ	94	105		199	599.05	587.16	553.76	518.60	506.95	494.38
20		ክምር -ድንጋይ	94	135		229	687.60	674.01	636.31	593.46	580.64	566.73
21		ጋሳኝ	94	146		240	720.06	705.85	666.58	620.91	607.66	593.26
22		ደብረ -ታቦር	94	164		258	773.19	757.96	716.11	665.83	651.87	636.67
23	ወልድያ	አመድ -በር	94	186		280	838.12	821.65	776.64	720.73	705.91	689.73
24		ጎንደር -መገንጠያ	94	199		293	876.49	859.29	812.41	753.17	737.84	721.09
25		ወረታ	94	210		304	908.96	891.13	842.68	780.62	764.86	747.62
26		ሐሙሲት	94	234		328	979.80	960.61	908.72	840.51	823.80	805.50
27		ባህርዳር	94	267		361	1077.19	1056.15	999.52	922.85	904.86	885.09
28		አዲስ -ዘመን	94	227		321	959.13	940.35	889.46	823.04	806.61	788.62
29		ኢንፈራንዝ	94	260		354	1056.53	1035.88	980.26	905.39	887.67	868.21
30		ማክሰኝት	94	280		374	1115.56	1093.78	1035.29	955.29	936.79	916.44

166		ዘዋ			7	7	23.20	22.72	22.32	20.81	20.68	20.79
167		ከአዩብ-አማያ			5	5	16.57	16.23	15.94	14.86	14.77	14.85
168		ኩኩፍቱ		62		62	182.99	179.49	170.60	154.71	152.28	149.53
169		ዋጃ		16.5		16.5	48.70	47.77	45.40	41.17	40.53	39.79
170		አማያ -ሚካኤል	9	10		19	57.20	56.06	52.87	49.52	48.41	47.21
171		በአራዶም-አርበት		9	13	22	69.65	68.25	66.21	61.10	60.51	59.02
172	ቆቦ	አዲስ- አለም	10.3			10.3	31.68	31.03	29.02	28.11	27.29	26.43
173		በገደባ- ተኩለሽ			22	22	72.91	71.41	70.14	65.40	64.99	65.34
174		ዲኖ			32	32	106.05	103.86	102.03	95.12	94.52	95.04
175	ኩተኩለሽ	ዲኖ			10	10	33.14	32.46	31.88	29.73	29.54	29.70
176		አፋፍ	16			16	49.22	48.20	45.08	43.67	42.39	41.06
177		አላማጣ		49		49	144.62	141.86	134.83	122.27	120.35	118.18
178	ሮቢት	ደሴ		151		151	445.68	437.15	415.49	376.81	370.89	364.18
179		ጎብዬ		9		9	26.56	26.06	24.76	22.46	22.11	21.71
180		ወርቁ		7	31	38	123.40	120.88	118.10	109.62	108.76	105.86
181		ሃራ		29		29	85.59	83.96	79.80	72.37	71.23	69.94
182		ቆቦ		26		26	76.74	75.27	71.54	64.88	63.86	62.71
183		ወርቁ ገበያ		2	31	33	108.64	106.41	104.34	97.14	96.48	93.80
184	ጎብየ	ሀራ		20		20	59.03	57.90	55.03	49.91	49.12	48.24
185		ደሴ		142		142	419.11	411.09	390.73	354.35	348.78	342.48
186		አፋፍ	9			9	27.68	27.11	25.36	24.57	23.85	23.10
187		ከጀነቶ - ጉባርጃ			7	7	23.20	22.72	22.32	20.81	20.68	20.79
188	ጉባላፍቶ	ጀነቶ ድሃ - ወዲህ			6	6	19.88	19.47	19.13	17.84	17.72	17.82
189		ከሰቀላ - ውራፍ			11.5	11.5	38.11	37.33	36.67	34.18	33.97	34.16
190		ከመቻሬ - ልዶታ			13.6	13.6	45.07	44.14	43.36	40.43	40.17	40.39
191		አላማጣ		78		78	230.22	225.81	214.62	194.64	191.58	188.12
192		ሐሮ		11		11	32.47	31.85	30.27	27.45	27.02	26.53
193		ደሴ		144		144	425.02	416.88	396.23	359.34	353.69	347.30
194		አረሪት	29	24		53	160.04	156.85	147.75	139.05	135.78	132.28
195		አላሌ	31	24		55	166.19	162.87	153.38	144.51	141.08	137.41
196	ከሐራ	ቆቦ		46		46	135.77	133.17	126.57	114.79	112.99	110.94
197		መርሳ		54		54	159.38	156.33	148.59	134.75	132.63	130.24
198		ጭፍራ		48		48	141.67	138.96	132.08	119.78	117.90	115.77
199		ዲሌ ሮቃ		23		23	67.88	66.59	63.29	57.39	56.49	55.47
200		አዳሚ			9	9	29.83	29.21	28.69	26.75	26.59	26.73


የሰሜን ወሎ ዞን
ትራንስፖርትና
ሎጂስቲክስ
ጽ/ቤት
የኃላፊ
ቢሮ


አቶ ፀሃይነው ሲሳይ አስማማው
የሰሜን ወሎ ዞን መንገድ
ትራንስፖርትና
ሎጅስቲክስ
መምሪያ ኃላፊ



ሁሉም ትኬት
HULUM TICKET

ታ/ጎ: none/AM31050

ወንበር ቁ.: 12

ወልድያ -> ደሴ

ቲኬት ቁ: [illegible]

ምድብ: ሚኒባስ ደረጃ 2

ስልክ ቁ.: +251739417798

መነሻ ሰዐት: 12:31 AM (ጠዋት)

ተጓዥ:

የጉዞ ቀን: 14 - 08 - 2018

ታሪፍ: 389 ብር

የአገልግሎት ክፍያ: 7 ብር

የደንብ አስከባሪዎች: 10 ብር

ጠቅላላ ክፍያ: 406 ብር

ትኬተር: [illegible]

ታተመ በ: 2026-04-22 06:37

ለአንድ ጊዜ ጉዞ ብቻ የተፈቀደ!

ስለመረጡን እናመሰግናለን!!

ለአስተያየትዎ/ቅሬታዎ: ይደውሉ